MoE 训练手册·中文全译

这是 Hugging Face 与 NovaSky 的《MoE Training Handbook》全文中文翻译。原文是一篇可以动手照做的工程手册：所有数字都来自 8×H100 上的真实运行，讲的是「这一层的每个部件怎么选、每个开关到底把时间花在哪」。

- 原文：https://huggingface.co/spaces/huggingface/moe-blogpost
- 代码基准：Megatron commit `d6316eb45`
- 规模：5 章 + 附录，正文约 5.1 万英文词

怎么读

这篇手册的组织方式是一条「从外到内、再从内到外」的路线：

1. 先建直觉（第 1 章）：为什么 MoE 值得做，参数里有多少是「总参」、多少是「激活」。
2. 先跑起来（第 2 章）：测硬件、建镜像、分词，先训一个稠密基线，再换成一个 MoE。
3. 再调每一层（第 3 章，最长）：路由器怎么打分、怎么均衡、专家多少个多宽、块怎么接、优化器和 MTP 怎么配、最后系统工程怎么把时间拿回来。
4. 然后服务出去（第 4 章）：训练时的选择在推理时哪些还成立、哪些要翻案。
5. 最后收口（第 5 章）：一份可以直接抄的推荐参数清单。

章节

章	中文	内容
1	引言：为什么是 MoE	稀疏性的直觉、参数量与激活量、三个数字（总参 / 激活 / batch）
2	起步：环境与首次运行	测硬件（DCGM + nccl-tests）、NGC 镜像与 EFA、分词 FineWeb-Edu、稠密基线 1.22B、首个 MoE 6.86B
3-1	第 3 章（上）：监控与路由	3.0 先打开监控、3.1 路由器打分的四要素、3.2 谁来做选择
3-2	第 3 章（中）：负载均衡	3.3 负载不均的代价、辅助损失、scope、无损失偏置、分位数均衡
3-3	第 3 章（中）：专家	3.4 数量与宽度、LatentMoE、共享专家、可变算力、激活函数
3-4	第 3 章（下）：块布局与训练	3.5 稠密层与块接线、3.6 upcycling、优化器、MTP、稳定性
3-5	第 3 章（下）：系统工程	3.7 剖析、互联、重叠、内存、组限路由、容量与副本、精度
4	从训练到服务	不同的 k、低位宽专家、服务端 EP、注意力-FFN 解耦、RL 路由
5	结语与推荐参数	按节整理的推荐配置，可直接抄的 bash 块
A	附录：监控细节与排障索引	A.1 每条命令与算法、A.2 42 条「症状 → 该读哪节」

合并版 PDF：2026-11-moe-handbook.pdf

关于图

原站的图都是 React 交互组件（可以拖动、切换），无法在 Markdown 与 PDF 里复现。本译本的处理方式是：保留图号与图注，正文里标注「（原站为交互图，此处从略）」；凡是有数字的结论，都在正文里用文字说清楚了，不依赖看图。

一个提醒

手册里的数字几乎都来自单节点 8×H100（少数是两节点）。它给的不是「标准答案」，而是「同一套测量方法下，不同选择之间的相对差距」。照抄参数之前先按第 2 章把硬件测一遍、按第 3 章把监控打开——不同集群、不同互联，最优点会移动。

相关概念：[[moe]]（混合专家）

第 1 章 引言

> 原文：*The MoE Handbook*（Hugging Face），https://huggingface.co/spaces/huggingface/moe-blogpost
> 阅读时间：1–2 天。为了最好的阅读体验，官方建议不要用手机看。

前沿模型是稀疏的。

在 Artificial Analysis 榜单上，得分最高的 20 个开源权重模型里有 18 个是混合专家（MoE）。剩下两个不是——Nanbeige4.3-33B 和 Qwen3.8 27B，排在第十二、十三名，都是稠密模型。而这 18 个里，包含了五家领先实验室各自的最强模型：DeepSeek V4.1 Flash 1017、GLM-5.3、MiMo-V2.6-Pro、Ling 3.1 Flash 1019、MiniMax-M3.1-Flash。

> 图 1.1 稠密与稀疏模型的发布时间与得分分布（原站为交互图，此处从略）

MoE 的好处，本质上在于它把推理算力花在了哪里。每个 token 只经过少数几个专家，于是模型可以携带比「单 token 实际用到」多得多的参数：你在第 2 章要动手训练的那个小 MoE 有 68.6 亿参数，其中每个 token 只激活 12.2 亿。难的地方在于，怎么把这笔推理上的节省，换算成训练上的加速。

我们上一本讲训练的深度长文 Smol Training Playbook 覆盖了稠密模型的训练，但只是顺带提了 MoE 的设计。因为那本书的主角 SmolLM3 是个稠密 Transformer，它的训练压根用不到 MoE 专属的技术，比如专家并行。即便如此，我们还是在书中「Going Sparse: MoE」一节里扫了一遍高层设计选择：稀疏度、粒度、共享专家、负载均衡。那一节点出了分布式 MoE 训练为什么复杂——专家放置、负载均衡、All-to-All 通信——但没有往细节里钻。这本手册就从那张清单开始。

从稠密模型走到 MoE，你会多出三件必须操心的事：

1. 一个路由器，决定每个 token 走哪些专家；
2. 一份必须始终均衡地压在各专家上的负载；
3. 一旦专家被拆到不同 GPU 上，每一层 MoE 都要在这些 GPU 之间交换 token。

每一件都会带来稠密训练从未有过的配置项，以及全新的失败方式。接下来的章节会把这些配置一个一个拆开，配上能跑起来的代码，而且大多数情况下还给出我们自己集群上的实测结果与数据。

一次跑崩的训练只会告诉你「症状」，不会告诉你「原因」。下面这张图是一张高层地图，帮你顺着症状回到源头：先看它是怎么崩的，再找异常行为的第一个信号（矿井里的金丝雀），然后去读正文里解释成因的那一节。留着手边，等哪天真的跑炸了再用。

> 图 1.2 故障排查地图：从症状 → 首个信号 → 对应章节（原站为交互图，此处从略）

> 图 1.3 全书路线图（原站为交互图，此处从略）

最后说一句读者定位。这本手册写给谁？如果你已经在 GPU 集群上训过稠密模型，下一个想训 MoE，那你就是目标读者。第 2 章会把需要的东西铺好，带你在单节点上把那个小 MoE 训出来。如果你更关心「MoE 到底怎么工作、前沿模型怎么选型」，而不是亲手跑一个，可以直接跳到第 3 章「MoE 训练的解剖」。

等专家被放到不同 GPU 上，每一层 MoE 都要在它们之间倒腾 token——这是稠密训练里从未出现过的通信模式。在你改模型的第一行代码之前，你真的信得过集群的互联吗？

导航：[[2026-11-moe-handbook|全书目录]] · 第 2 章 环境与第一次跑通 →

第 2 章 环境准备与第一次跑通

2.0 你需要什么

这一章里的每一条命令，都是按同一种机器写的，目的是让所有人跑在同一个环境里：一个由八卡节点组成的 Slurm 集群。本节说明这套环境长什么样，你有 1 张卡甚至没有卡时能跟到哪一步，以及怎么在 Hugging Face Jobs 上租一个八卡节点。如果你的机器不一样，讲解照样能读，命令自己改。

命令默认的集群形态

这些命令假定你有一个 Slurm 集群，每个 GPU 节点八张卡。容器通过 Enroot 与 Pyxis 启动，这样 `srun` 就能在镜像里拉起一个任务。2.1 节的节点自检用的是 NVIDIA 的 DCGM 诊断。我们自己的实验跑在 AWS P5 节点上，节点之间用 EFA（Elastic Fabric Adapter）互联，每台八张 80 GB 的 H100。

这一章大部分内容一个节点就能跑完。稠密基线和第一个 MoE 各要一个八卡、每卡约 80 GB 的节点。只有网络检查需要更多机器：2.1 节的 All-to-All（all-to-all）测试会跑 2、4、8 个节点，2.2 节的 EFA 镜像要在两个节点上验证。

软件来自两个钉死的来源。一是 NGC PyTorch 镜像 `26.06-py3`，2.2 节会导入它并补上集群的网络库。二是 Megatron 的 `d6316eb45` 这个 commit——它是挂载进容器，而不是打进镜像里。你还需要一个 CPU worker 和 GPU 节点都能读的共享文件系统，因为 2.3 节在 CPU 分区上做分词，训练任务再从那里读结果。

只有一张卡、或者一张卡都没有

没有集群也能读懂大半篇。第 3 章是拆解 MoE 层：每个部件做什么、前沿模型各自怎么选、以及每个选择背后的证据。那一章可以完全不跑代码读完，把里面的参数先记下来，等有硬件了再用。第 4 章把这些选择从预训练一路带到推理服务和强化学习。

在这一章里，2.3 节的分词跑在 CPU 上。它的小样本模式单机就能跑、不需要 Slurm，所以你可以亲手产出并检查训练要读的那些文件。2.4 与 2.5 的训练配方假定八张约 80 GB 的卡，而且 MoE 会把专家铺满八张卡。卡少的话，就读方法：两次实验之间什么保持不变、专家宽度是怎么定的、该比什么。2.1 的 All-to-All 测试与 2.2 的 EFA 配置，按原文跑需要多节点。

在 Hugging Face Jobs 上租一个节点

如果你想跑单节点那部分又不想买机器，Hugging Face Jobs 可以在 Hugging Face 的机器上跑一个 Docker 镜像加一条命令，按分钟计费。任何余额为正的个人或组织都能用，PRO / Team / Enterprise

订阅的月度额度也能抵扣。`hf jobs hardware` 列出的最大规格就是八卡，正好够 2.4 与 2.5 用；比如 `h200x8` 是八张 141 GB 的 H200。

一个 job 由三样东西定义：来自 Docker Hub 或 Hugging Face Space 的镜像、一条命令、一个硬件规格。所以想在上面跑本章，得把 2.2 节产出的镜像打包成 Docker 镜像，再把 Slurm 启动脚本换成 Jobs 命令。job 默认 30 分钟超时，除非你另设。第一条命令先确认它真能看到八张卡：

```
hf jobs run --flavor h200x8 --timeout 10m \
    pytorch/pytorch:2.6.0-cuda12.4-cudnn9-devel \
    python -c "import torch; print(torch.cuda.device_count())"
```

不管你在自己集群还是租的节点上跑，发给你的机器未必有规格书上写的那么能打。那么问题来了：一张卡或一条链路在偷偷降速，你怎么发现？

2.1 测你的硬件

在开始调模型之前，先摸清机器能干多少活。用 Megatron 的 `alltoall` 分发器时，每一层 MoE 都要把 token 送到持有对应专家的卡上，再通过另一次 All-to-All 把结果收回来。所以卡与卡之间的链路，本身就是你要测的工作负载之一：一次健康的 all-reduce 结果，并不能说明这些 token 交换会有多快。

下面这些检查要在你的作业分配里、启动训练之前跑。用 Pyxis 时，`srun --container-image` 就能起容器；下一节讲怎么准备训练镜像。

逐个节点自检

先对每个分配到的节点做一次就绪检查。NVIDIA 的 DCGM 提供几个诊断级别，1 级用来做就绪检查，更高级别会加上帮助定位故障的测试。下面这条命令每节点跑一个任务，这样启动训练前你能逐个看结果。

```
NNODES=$SLURM_JOB_NUM_NODES
srun --nodes="$NNODES" --ntasks="$NNODES" --ntasks-per-node=1 \
    dcgmi diag -r 1 --fail-early
```

全部通过就去做通信测试。有节点挂了，把它的诊断输出留下来，查清楚再往下走。举例：我们有一台节点上，2 级诊断在三张卡上报了温度告警，同时 PCIe 和显存测试也失败。

更长时间的诊断应该另开维护窗口。NVIDIA 把 3、4 级定位成管理员事后取证，1 级才是就绪检查。

准备诊断镜像

如果你的集群已经提供了 `nccl-tests` 镜像，把 `NCCL_TESTS_IMAGE` 指向它就行。否则，在带 EFA 的 AWS 上可以导入那张配合 AWS NCCL 测试配方维护的公开镜像。目标路径要落在分配节点能访问的文件系统上：

```
NCCL_TESTS_IMAGE=/shared/images/nccl-tests.sqsh
mkdir -p "$(dirname "$NCCL_TESTS_IMAGE")"
enroot import -o "$NCCL_TESTS_IMAGE" \
    'docker://public.ecr.aws#hpc-cloud/nccl-tests:latest'
```

把导入出来的 SquashFS 文件和你的基线数据放在一起，因为再导一次 `latest` 可能拿到不一样的镜像。想要可重复，就从 AWS 配方里挑一个带版本号的 tag，替掉 `latest`。

AWS 的配方也提供 Dockerfile，需要自己构建时可以用。换别的互联网络，就用它支持的 CUDA、NCCL、MPI 和网络插件栈；NVIDIA 的 nccl-tests 构建说明讲了怎么指定这些库的路径。多节点测试要求带 `MPI=1` 构建。下面的基准假定 MPI 能通过 Slurm 的 PMIx 集成拉起，所以镜像要和集群的 MPI 配置对得上。

测你实际要用的那个集合通信

节点都过了，就在整个分配上测 All-to-All 带宽。下面这条命令每卡一个 rank，每个 rank 交换 8 GiB 缓冲，对应我们基线用的大消息测试。用上面准备好的 `NCCL_TESTS_IMAGE`；如果你的镜像把 `nccl-tests` 装到了别处，改一下可执行文件路径。

```
srun --nodes="$NNODES" --ntasks="$((8 * NNODES))" \
    --ntasks-per-node=8 --gres=gpu:8 --mpi=pmix --cpu-bind=none \
    --container-image="$NCCL_TESTS_IMAGE" --no-container-mount-home \
    /opt/nccl-tests/build/alltoall_perf \
    -b 8G -e 8G -g 1 -c 1 -n 20
```

看结果之前先核对 rank 列表：两个八卡节点应该列出 16 个 rank。然后看目标尺寸那一行的 `busbw` 和 `#wrong` 两列。下面是我们双节点基线的 out-of-place 结果，其他列略去：

```
size (B)      busbw (GB/s)    #wrong
8589934592    77.14           0
```

`#wrong` 为 0，说明这次结果通过了正确性校验。这个带宽可以当作后续同样节点数、同样卡数、同样测试参数下的参照。把节点列表一起存下来，将来出现慢的结果，才能追回到具体是哪几台机器。另一次双节点分配里，我们打开 NCCL 调优变量跑同样的消息大小，只拿到 39.84 GB/s——所以每份基线都要记清楚是哪些节点、什么设置跑出来的。

all-reduce 作为对照仍然有价值，但同一个集合通信操作在不同机器上的表现可能差很远。上面那次偏慢的双节点测试，all-reduce 拿到 398.52 GB/s，而 all-to-all 只有 39.84 GB/s。对一个要跨这些节点交换 token 的 MoE 来说，后者才是测对了通信模式的那个数。

每种分配规模各定一条基线

节点加得越多，All-to-All 里跨节点网络的比例就越高。我们的多节点扫描说明了基线为什么必须跟节点数绑定：双节点 77.14 GB/s、四节点 39.25 GB/s、八节点 30.91 GB/s。每次分配都要跟相同节点数、相

同消息大小、相同结果列的基线比。

定参照之前，先在一套已知良好的配置上把硬件、软件、测试参数都固定住，重复几次，用得到的区间来判断一份新分配够不够格开训。一个实用的起步策略是：带宽不低于基线的 90%，且正确性校验通过。按上面那次双节点结果，阈值大约 69 GB/s。阈值要按你自己集群的波动来调。举个反例：如果钉死 33 GB/s，那次偏慢的双节点结果会过关，而八节点扫描的结果会被判死。

追查慢分配的时候，别动这条基线。如果你想试 NCCL 调优变量，就在同样的节点、同样的测试参数上对比。我们默认组和调优组用的是不同的节点对，所以那点带宽差归因不到变量头上。

训练期间也要盯硬件变化

训练期间的周期性硬件检查，用 Megatron 的 `--gpu-sniff-test-interval <n>`。它会在训练开始前跑一次，之后每 `n` 步跑一次（`training.py#L4850-L4852`、`training.py#L4484-L4489`），其中包含一次专家并行组内的 All-to-All（`gpu_sniff_test.py#L5-L21`）。它给出的离群告警，就是变慢时该去查的 rank（`gpu_sniff_test.py#L104-L118`）。

诊断镜像让你测过了卡和链路。训练镜像需要访问同一批硬件，那接下来就把它做出来，顺便验一下它的网络库。

2.2 构建你的镜像

训练环境有两点必须弄对：跑 GPU 计算的库，和把这些卡跨节点连起来的网络库。第一部分从 NVIDIA 的 PyTorch 容器起步，再补上集群需要的网络支持。把 Megatron 放在独立的 checkout 里，你就能在不动镜像的前提下改训练代码。

这里的例子用 Slurm 上的 Enroot 与 Pyxis。基础镜像是 NGC PyTorch `26.06-py3`；网络那部分指令针对我们带 EFA 的 AWS P5 节点。换集群就用它支持的网络栈来做这一块。

导入基础镜像

Enroot 把容器镜像导成 SquashFS 文件，Pyxis 才能通过 `srun` 启动它。选一个计算节点能访问的输出路径，然后导入：

```
IMAGE=/path/to/images/pytorch-26.06.sqsh
enroot import -o "$IMAGE" docker://nvcr.io#nvidia/pytorch:26.06-py3
```

Megatron 的 checkout 放在镜像外面，启动训练时挂进去。本文里的参数针对 commit `d6316eb45`；显式 checkout 到它，能保证你读到的代码和跑起来的代码是同一份。每个测量结果对应的镜像与 commit，都在正文的脚注里列了。

把这个 checkout、以及后面的数据和分词器，都放在 CPU worker 与 GPU 节点都能读的文件系统上。下面的命令里，`SHARED_ROOT` 就是这个目录，`PROJECT_DIR` 是你的项目目录，配套脚本克隆到它的

`scripts/` 子目录。

```
export SHARED_ROOT=/path/to/shared/work
export PROJECT_DIR="$SHARED_ROOT/moe-blogpost"
git clone https://gist.github.com/nmntz/5de952d686f66d7bce5bce8cba9d7139.git "$PROJECT_DIR/script
s"
git clone https://github.com/NVIDIA/Megatron-LM.git "$SHARED_ROOT/Megatron-LM"
cd "$SHARED_ROOT/Megatron-LM"
git checkout d6316eb45bf1bfbb6f1f848c0b56e55166564d80
```

验证 GPU 可用

补网络库之前，先确认基础镜像能 import PyTorch 与 Transformer Engine、能看到卡。在分配好的八卡节点里，用上面的 `IMAGE` 路径跑：

```
srun --nodes=1 --ntasks=1 --gres=gpu:8 \
    --container-image="$IMAGE" --no-container-mount-home \
    python -c '
import torch, transformer_engine
print("torch", torch.__version__)
print("transformer_engine", transformer_engine.__version__)
print("device_count", torch.cuda.device_count())
x = torch.randn(8).cuda().sum()
print("cuda sum ok", x.item())
'
```

原生镜像报出来的版本与卡数如下。最后一行还在 GPU 上算了个和，所以别只看 import，把那个结果也一起看了。

```
torch 2.13.0a0+8145d630e8.nv26.06
transformer_engine 2.16.0+4220403e
device_count 8
```

CUDA 那个结果只验证了本节点内的 GPU 执行。要跨节点走 EFA，还得加上下面说的网络插件与依赖。

补上 EFA 支持

在我们 AWS 节点上，网络层由 EFA 用户态库和针对镜像内 NCCL 编译的 `aws-ofi-nccl` 插件组成。构建用的是 EFA installer `1.49.0` 与 `aws-ofi-nccl` `v1.19.2`。这两个版本号要和基础镜像一起写进你的构建配方。

EFA 镜像构建脚本从基础镜像起一个可写的 Enroot 容器，用 `--skip-kmod` 装 EFA 用户态库，再针对镜像里的 NCCL 编译插件，然后导出新的 SquashFS 并删掉临时容器。在一个有 Enroot 与 GPU 权限的分配节点上跑它，指定新的输出路径：

```
EFA_IMAGE=/path/to/images/pytorch-26.06-efa.sqsh
bash "$PROJECT_DIR/scripts/build_efa_image.sh" "$IMAGE" "$EFA_IMAGE"
```

之后启动训练，`--container-image` 都用 `EFA_IMAGE`。脚本只负责把网络层做进镜像；下面这几条环境变量要和镜像一起带着，任务才能找到并选中正确的库。

这里的库路径指向容器内装好的副本，`NCCL_NET_PLUGIN=ofi` 是按名字选插件。我们的环境里，如果给插件传一个文件系统路径，NCCL 会拼出一个非法的库名。

```
export LD_LIBRARY_PATH=/opt/amazon/ofi-nccl/lib:/opt/amazon/efa/lib:${LD_LIBRARY_PATH:-}
export NCCL_NET_PLUGIN=ofi
export FI_PROVIDER=efa
export FI_EFA_USE_DEVICE_RDMA=1
export NCCL_SOCKET_IFNAME='^lo,docker0,veth_def_agent'
export NCCL_DEBUG=INFO
```

排除网卡这一项，针对的是 NCCL 建立通信时那次初始连接。在我们的节点上，NCCL 选中了 `veth_def_agent`，试图连它的 link-local 地址，然后 `Connection refused`。把这个网卡排掉，启动才过得去。复用到你自己集群前，先按实际的网卡名改排除列表。

验证网络通路

第一次多节点开训时，先看 NCCL 初始化日志，再去解读训练性能。这一步检的是训练镜像自己的网络通路。对本节这个 EFA 镜像，要找的是这一行：

```
NCCL INFO Using network Libfabric
```

如果看到的是 `Using network Socket`，就去读它上面紧挨着的插件加载信息。插件名写错、或者库版本缺失，都会让 NCCL 退回 socket。举例：`EFA_1.4` 这个报错就是在说某个库版本找不到，得先解决再试。

日志只能确认任务选了哪个传输层，带宽还得单独测。把训练镜像、网络设置连同 2.1 节的诊断结果记在一起，将来比较两者性能时环境才说得清。

硬件验过了、镜像也备好了，就可以开始第一次训练。不过得从数据开始：把原始文本变成 token 序列，并落成 Megatron 认识的那种格式。

2.3 分词

Megatron 从成对的索引文件里读训练文本：`.bin` 存 token ID，`.idx` 告诉 dataloader 每条序列从哪儿开始到哪儿结束。这些文件只做一次，稠密和 MoE 两次实验读同一批 token，换模型就不会顺带换了数据。

我们用 FineWeb-Edu 和 Llama 3.2 分词器。Datatrove 的 `MegatronDocumentTokenizer` 直接写这种格式，剩下的活就是选输出目录、验一个小样本、提交大的分词任务。

> 图 2.3.1 两条示例文档以 token ID 形式存储，各自以文档结束符收尾（原站为交互图，此处从略）

准备分词器

数据和分词器都放在 `SHARED_ROOT`，也就是 2.2 节那个共享目录。分词脚本在本地跑小样本，大任务则提交成 Slurm 数组作业。

```
export TOKENIZER_DIR="$SHARED_ROOT/tokenizer"
python3.11 -m venv "$SHARED_ROOT/tokenize-venv"
source "$SHARED_ROOT/tokenize-venv/bin/activate"
python -m pip install \
    'datatrove @ git+https://github.com/huggingface/datatrove.git@2c895af7a95010a96fa53e2dc77cc0ce
92799e3f' \
    pyarrow tokenizers huggingface_hub
```

原始分词器仓库 `meta-llama/Llama-3.2-1B` 需要申请权限。你的账号有权限就 `hf auth login`，然后在下载命令里用那个仓库。没有的话，`unsloth/Llama-3.2-1B` 提供了一个不需要门禁的副本：我们比对过，词表和 token ID 一致。分词和训练读的是同一批下载下来的文件。

```
TOKENIZER_REPO=unsloth/Llama-3.2-1B
hf download "$TOKENIZER_REPO" \
    --include 'tokenizer*' --include 'special_tokens_map.json' --include 'config.json' \
    --local-dir "$TOKENIZER_DIR"
```

Datatrove 从这个目录读 `tokenizer.json`，并在每篇文档末尾追加 `<|end_of_text|>`。Megatron 之后通过 `--tokenizer-model` 拿到这个目录，同时也拿到分词器的配置。

验一个小分片

提交整个数组之前，先在 CPU 计算节点上分一个小样本。下面这条读到 1000 篇为止、全部保留，结果写进另一个目录，免得样本被当成训练语料。

```
python "$PROJECT_DIR/scripts/tokenize_first_run.py" \
    --tokenizer "$TOKENIZER_DIR/tokenizer.json" \
    --output "$SHARED_ROOT/data-smoke" --limit 1000
```

输出前缀是 `fineweb-edu_00000_tokens`。用 2.2 节的训练镜像里 Megatron 自己的读取器验一遍，节点要能让 Pyxis 起容器。先退出分词环境，否则它的 Python 会盖掉镜像里的解释器。`TRAIN_IMAGE` 在配了 EFA 的机器上指向 `EFA_IMAGE`，否则指向你集群准备好的训练镜像。

```
deactivate
export TRAIN_IMAGE="$EFA_IMAGE"
export MEGATRON_PATH="$SHARED_ROOT/Megatron-LM"
srun --nodes=1 --ntasks=1 \
    --container-image="$TRAIN_IMAGE" \
    --container-mounts="$SHARED_ROOT:$SHARED_ROOT" \
    --container-workdir="$MEGATRON_PATH" --no-container-mount-home \
    env PYTHONPATH="$MEGATRON_PATH" python -c '
import sys
from megatron.core.datasets.indexed_dataset import IndexedDataset
data = IndexedDataset(sys.argv[1])
assert len(data) > 0
print("sequences", len(data))
for i in range(min(3, len(data))):
    tokens = data[i]
    assert len(tokens) and tokens[-1] == 128001
    print("length", len(tokens), "last token", int(tokens[-1]))
' "$SHARED_ROOT/data-smoke/fineweb-edu_00000_tokens"
```

你应该看到非零的序列数，以及每条的 `last token 128001`。那个尾 token 就是我们让分词器追加的文档边界。用 Megatron 读一遍，同时也验证了它在 GPU 训练开始前就能解析索引、从 token 文件里读出最前面几条序列。

提交分词任务

小样本验的是格式，大任务是为了准备一份能随训练规模一起长大的语料。在登录节点上、同一个虚拟环境里提交。`CPU_PARTITION` 填你集群的 CPU 分区；调度器要 QOS 就加 `--qos`。脚本每个任务要 12 个 CPU、24 GB 内存，而且这些 worker 上同一路径必须能访问到那个虚拟环境。

```
source "$SHARED_ROOT/tokenize-venv/bin/activate"
export OUT_DIR="$SHARED_ROOT/data/fineweb-edu"
CPU_PARTITION=your_cpu_partition
python "$PROJECT_DIR/scripts/tokenize_first_run.py" \
    --tokenizer "$TOKENIZER_DIR/tokenizer.json" --output "$OUT_DIR" \
    --tasks 100 --limit -1 --sample-rate 0.074 --partition "$CPU_PARTITION"
```

Datatrove 会打印提交上去的 job ID。用 `squeue -j <job-id>` 跟这个数组，失败的任务到 `$OUT_DIR/logs` 下看，看清楚了再重跑同一条命令。

每个任务分一部分输入文件，按 7.4% 随机抽样后再分词。因为文档长度不一，这个比例并不能固定输出的 token 数。中断后重启时，任务数、采样率、输出目录都别改——Datatrove 靠日志目录下的完成标记跳过已经做完的任务。

把分片交给 Megatron

等数组跑完再做训练用的清单。下面这段检查要求 100 个任务完成标记和对应的分片文件都在，然后把分片前缀写进 `data-args.txt`，一行一个。每个前缀同时指代那两个文件，所以清单里既不该有 `.bin` 也不该有

`.idx`。

```
export DATA_ARGS="$OUT_DIR/data-args.txt"
python - <<'PY'
import os
from pathlib import Path
root = Path(os.environ["OUT_DIR"])
prefixes = [root / f"fineweb-edu_{rank:05d}_tokens" for rank in range(100)]
for rank, prefix in enumerate(prefixes):
    assert (root / "logs/completions" / f"{rank:05d}").is_file(), rank
    for suffix in (".bin", ".idx"):
        path = Path(str(prefix) + suffix)
        assert path.is_file() and path.stat().st_size > 0, path
Path(os.environ["DATA_ARGS"]).write_text("\n".join(map(str, prefixes)) + "\n")
print(f"wrote {len(prefixes)} prefixes")
PY
deactivate
```

训练时通过 `--data-args-path` 读这个文件。它给的数据和 `--data-path` 一样，只是把一长串分片挪出了启动命令。启动器还会把 `--data-cache-path` 指到一个持久目录，存 Megatron 的样本索引，之后同配置的实验可以直接复用。

头几次实验我们用 `--split 100,0,0`，全给训练。等你打算用这次实验比模型质量了，就先留出一个留出集。

数据能读、清单也指向了正确的分片，就可以开稠密基线了。它的 loss、梯度范数和每步耗时，会成为引入专家之前的参照系。

2.4 稠密基线

MoE 表现异常时，稠密实验给了你一个具体可比的对象。两个模型用同样的数据、batch size 和优化器，于是 loss 尖峰或者某一步变慢，都能先跟那个更简单的模型对一对。这条基线还顺便把训练镜像和 dataloader 一起跑通了——这时候路由还没登场。

模型 16 层 Transformer，hidden 2048，前馈宽 8192。词表 128,256，合计约 12.2 亿参数。配方针对一个八卡、每卡约 80 GB 的节点。启动脚本后面给 MoE 复用。

在单节点上启动

训练指标我们用 Trackio 记录。它是 Hugging Face 开源的轻量实验追踪器，API 对齐 `wandb`，通常现有代码只要把 `import wandb` 换成 `import trackio as wandb`。实验记录写进你自己机器上的本地数据库，可以在本地面板里翻，也可以同步到一个 Hugging Face Space，跟团队一起看实时曲线，不需要账号也不花钱。

先在训练镜像里、在共享存储上建一个 Python 环境。它继承镜像的 GPU 库，再补一个钉死版本的追踪包：

```
srun --nodes=1 --ntasks=1 \
    --container-image="$TRAIN_IMAGE" \
    --container-mounts="$SHARED_ROOT:$SHARED_ROOT" --no-container-mount-home \
    bash -c 'python -m venv --system-site-packages "$1/train-venv" &&
        "$1/train-venv/bin/python" -m pip install "trackio==0.42.0"' _ "$SHARED_ROOT"
export TRACKIO_PROJECT=moe-first-run
```

启动器通过 Trackio 入口把 Megatron 的标量指标送进每个 job 一个 run。两个 run 共用上面这个 project，数据都落在 `$SHARED_ROOT/trackio` 下。这个适配器复用了 Megatron 现有的标量日志钩子，所以像 `--log-memory-to-tensorboard` 这类开关，仍然决定哪些指标会进 Trackio。

首次运行脚本把训练参数分组塞进 Bash 数组，用前面几节准备好的训练镜像、钉死的 Megatron checkout、分词器目录和数据清单。挂载路径保持在 `SHARED_ROOT` 下，`TRAIN_IMAGE` 指向前面的 SquashFS 镜像。

在共享的博客 checkout 里，把稠密任务提交到你的 GPU 分区。提交时 2.3 节那些变量要保持导出状态。脚本会把镜像路径、Megatron commit 和数据清单一起记进输出。

```
cd "$PROJECT_DIR"
GPU_PARTITION=your_gpu_partition
sbatch --partition="$GPU_PARTITION" scripts/first_run.sbatch dense
```

Slurm 会占下一个八卡节点。节点里一个 `srun` 任务起 `torchrun`，每张卡一个训练进程。运行输出在 `$SHARED_ROOT/runs/dense-<job-id>`，训练日志在提交目录下的 `first-run-<job-id>.out`。

这次实验跑 120 步就停。这个阶段关掉了存 checkpoint 和验证——它只是用来检查环境。日志和 Trackio 数据都留着，下一节要拿它们当参照。

想看曲线，就在一台能读到那个目录的机器上装 `trackio==0.42.0`，然后起面板。在 project 里把稠密和 MoE 两个 run 都选上对比。

```
TRACKIO_DIR="$SHARED_ROOT/trackio" trackio show --project "$TRACKIO_PROJECT"
```

看懂这套配置

张量并行和流水并行都设成 1，所以每张卡上都有一份完整的稠密模型。数据并行把 batch 摊到八张卡上，两次实验都开了分布式优化器和通信重叠。

```
--tensor-model-parallel-size 1 --pipeline-model-parallel-size 1
--expert-model-parallel-size 1 --context-parallel-size 1
--use-distributed-optimizer --overlap-grad-reduce --overlap-param-gather
--micro-batch-size 8 --global-batch-size 512
```

每卡一个 micro-batch 是 8 条序列。八张卡一个 micro-batch 就是 64 条，再累积 8 次梯度，得到 global batch 512。序列长度 4096，于是每个优化器步吃掉 2,097,152 个 token。这些量固定住，稠

密和 MoE 的每步耗时才可以直接比。

学习率走 warmup－稳定－衰减，由 `--lr-decay-style WSD` 选择：先升到 `5e-4`，保持，然后在预算末段衰减。120 步的实验结束在稳定段，它的停止条件并不会截短学习率曲线。

开启完整的激活重算，才能让初始配置塞进我们基线用的显存预算。在 micro-batch 8 加原生交叉熵实现下，不开重算的那次在算 loss 时就把显存吃爆了。两个模型都保留重算，这样第一次比较不会把「改模型」和「省显存」混在一起。

```
--recompute-granularity full --recompute-method uniform --recompute-num-layers 1
--cross-entropy-loss-fusion --cross-entropy-fusion-impl native
```

把稠密实验的 loss、梯度范数、每步耗时留在 Trackio 里当参照。等你把前馈层换成专家，这些数就能告诉你这次改动影响了什么。

2.5 你的第一个 MoE

稠密模型对每个 token 都套同一个前馈网络。MoE 里，路由器给每个 token 从 64 个更小的专家里挑 8 个，于是模型可以揣着比「单个 token 实际用到」多得多的参数。专家宽度是倒着算的：让被选中的 8 个专家加起来，前馈参数量正好等于被替换掉的那层稠密网络。

这样得到 68.6 亿总参数、每 token 约 12.2 亿激活参数的模型。多出来的容量带着一笔新开销：token 得先跑到持有对应专家的卡上，专家的输出还得再跑回来。

替换前馈层

传 `moe` 时启动器会切到 MoE 配置。用和稠密实验相同的导出路径与 GPU 分区提交，镜像、checkout、分词器、数据清单都不变：

```
sbatch --partition="$GPU_PARTITION" scripts/first_run.sbatch moe
```

专家并行从 1 调到 8，64 个专家铺满这个节点的八张卡：每卡 8 个专家，而路由器是从全集里给每个 token 挑 8 个。

```
--expert-model-parallel-size 8 --expert-tensor-parallel-size 1
--num-experts 64 --moe-ffn-hidden-size 1024 --moe-router-topk 8
--moe-token-dispatcher-type alltoall --moe-grouped-gemm
--moe-router-load-balancing-type aux_loss --moe-aux-loss-coeff 1e-3
--moe-router-dtype fp32 --moe-per-layer-logging
```

`alltoall` 负责在卡之间搬运被路由的 token，grouped GEMM 负责把本地专家的矩阵乘批量算掉。辅助损失以 `1e-3` 的系数鼓励路由器把活摊开。路由 logits 用 fp32 算，并打开逐层日志——这些初始设置和测量结果，第 3 章会逐个翻。

其余都留在公共配置里，包括完整激活重算。特别注意：脚本里稠密的 `--ffn-hidden-size 8192` 还在，而 `--moe-ffn-hidden-size 1024` 给的是每个专家的宽度。

数一下激活参数和总参数

无偏置的 SwiGLU 专家有三个权重矩阵。hidden 2048、专家宽 1024 时，它们合起来是 `3 × 2048 × 1024 = 6,291,456` 个参数。选 8 个专家，每 token 就用掉 50,331,648 个前馈参数，正好等于稠密层 `3 × 2048 × 8192` 的量。

> 图 2.5.2 稠密前馈层与 64 个专家，同一比例尺。一个 token 的 8 个专家首尾相接，宽度恰好等于稠密层：激活参数一样，而存储的参数量是 8 倍（原站为交互图，此处从略）

64 个专家合起来，前馈参数是 8 倍。整个模型的倍数小得多，因为注意力、embedding 和其他非专家参数没变，每层只多了一个不大的路由矩阵。把这些部件在 16 层上加起来，总参数 68.6 亿、激活参数 12.21 亿，而稠密模型是 12.19 亿。

结果是总参数约 5.6 倍，激活参数几乎不变。但激活参数对齐并不保证每步耗时相同：把 token 摊给专家会改变 GPU 上的工作组织方式，还引入了通信。

留一条稠密参照

只有一个专家、top-1 路由的 MoE，算的函数和它来源的稠密模型一模一样，所以训练行为也应该一样。在我们的实验里，从一个稠密 checkpoint 上采样（upcycle）出 1 专家 MoE，和稠密实验并排跑，15 个对齐的步里两者差不超过 0.00087。反过来让 MoE 自己初始化再比，就没什么意义了——两边在第 4 步就差了 2.72。MoE 那边还记录到负载均衡损失恒等于 1，梯度里有约 32,770 个零，而稠密实验最多只有 17 个：那是路由权重，只有一个专家时它拿不到梯度。这次对齐实验走的是 upcycle 稠密 checkpoint 这条路，3.6 节还会回来讲它。

在我们钉的这个 commit 上，upcycle 这条路要打两个本地补丁才跑得起来，都在 `training.py` 里。第一处是往 checkpoint loader 传稠密模型时用的关键字是 `model`（`training.py#L3074-L3078`），而 `load_checkpoint` 里那个参数叫 `ddp_model`。第二处在为加载而构建的稠密模型上：upcycle 的代码是在 `args` 里把稠密形状打开（`training.py#L3056-L3062`），但模型构建器读的是启动时存下来的那份配置（`training.py#L2917-L2926`），于是那个模型仍然是 MoE 层的布局，加载时报缺少 `pre_mlp_layernorm`。

确定性模式能让一次实验可复现，这样两次实验之间的差异才有意义。在这个 commit 上，`--deterministic-mode` 要求关掉交叉熵 loss 融合、TP 通信重叠和融合版 aux-loss kernel（`determinism.py#L28-L37`、`determinism.py#L195-L210`），并且只接受 `NVTE_ALLOW_NONDETERMINISTIC_ALGO=0`（`determinism.py#L85-L90`）；在用户没设的时候，它会自己设上 `NCCL_ALGO=Ring`、`NCCL_EP_HT_EM_AG_SCAN_MODE=1`、`NVTE_ALLOW_NONDETERMINISTIC_ALGO=0` 和 `CUBLAS_WORKSPACE_CONFIG=:4096:8`（`determinism.py#L40-L50`）。2.4 节的启动器传了 `--cross-entropy-loss-fusion` 并导出 `NVTE_ALLOW_NONDETERMINISTIC_ALGO=1`，加这个开关前两样都要去掉。按这套设置，这个 MoE 缩到 micro-batch 2、global batch 64、跑 12 步，两次实验每一步打印的 lm loss、负载

均衡损失、梯度范数和梯度零个数都完全一致；不设这些参数的话，lm loss 从第一步就开始漂，最大差到 0.00063。

要比两次实验、或者追一个 NaN，先把确定性模式打开。

开发 MoE 的过程中，把稠密实验和它的配置一直当参照。一旦某个改动让训练变慢、或者 loss 不降了，就回到 Trackio 里的稠密实验，在同样的 batch size 与序列长度下比每步耗时、loss 和梯度范数。两次实验都有的问题，指向它们共用的那套环境；只在 MoE 里出现的问题，才有理由去查路由和专家执行。

> 图 2.5.4 Trackio 里的稠密与 MoE 示例实验。这两条曲线只是示意界面，不是真实测得的性能（原站为交互图，此处从略）

有了稠密参照，你就知道该从哪儿开始查。第 3 章会把 MoE 层一块一块拆开，把理解它行为所需的那些测量逐个介绍一遍。

导航：[[2026-11-moe-handbook|全书目录]] · ← 第 1 章 引言 · 第 3 章（上）监控与路由 →

第 3 章 解剖 MoE 训练（上）

> 说明：原站里的图都是交互式组件，这里无法复现，一律保留图注并标注「（原站为交互图，此处从略）」。原站未给图编号，本文按节内顺序编号（如 图 3.1-a），便于对照。

2.5 节那个 MoE，比稠密实验多出了几样东西：一个给每个 token 挑 8 个专家的路由器、工作量可能差得很远的专家、以及在卡之间搬运 token 的 All-to-All。每一样都能让训练变慢或者把 loss 弄坏，而「比稠密基线差了」这个现象本身，并不会告诉你是哪一样造成的。

下面这张图画了一个比我们小一号的 MoE 层：16 个专家、top-4（我们是 64 与 top-8），另外多一个我们没有的共享专家。它展示了 batch 里的 token、路由器、专家，以及把专家输出合并起来的那次求和。后面每一节都会拿同一张图的一个局部视角来开场，放大到它要讲的部分，所以你可以直接跳到需要的那节。有几个视角还带着该节的实验结论，比如 3.6 节里 Step-3 Flash 的训练 loss 没有暴露出来的那些失控专家。

> 图 3.0-a 一层 MoE 的全景：token、路由器、专家、输出合并（原站为交互图，此处从略）

3.0 节把监控打开之后，我们从路由器内部开始：它的分数怎么变成被选专家的权重，以及到底是 token 选专家还是专家选 token。然后是专家之间负载不均、用多少专家每个多大、哪些层要放专家、以及怎么起步并保持稳定。最后一步讲一个训练步的时间都花在 GPU 的哪儿。

3.0 监控：一次性打开

动任何设置之前，先把「它到底有没有起作用」这件事打开。你现在不必知道每个数字的含义：每节在第一次需要某个指标时再介绍（3.1 节的 z-loss、3.2 节的 MaxVio、3.7 节的耗时与显存）。附录 A.1 给出每个测量对应的采集命令、计算公式和局限。

该打开什么

2.5 节那次实验已经在逐层记录路由损失（`--moe-per-layer-logging`），你可以看出是哪一层在闹脾气（3.1、3.3、A.1 节）。2.4 节的 Trackio 启动器留着，把下面这些参数加到 2.5 节的参数里：

```
--save <dir> --save-interval <m>
--save-tokens-per-expert-interval <n>
--log-throughput --log-params-norm
--log-num-zeros-in-grad --log-max-attention-logit
```

Megatron 会把逐专家的 token 数和激活 dump 写到 `--save` 下面（`training.py#L3398-L3401`），并要求在它旁边给出 `--save-interval`（`arguments.py#L1321-L1323`），所以这次实验也会把 checkpoint 存在那里。下一行参数记录每个专家收到多少 token（3.3 节、A.1 节）。再往后四个盯的是运行健康度：吞

吐、参数范数、梯度里的零、以及最大的注意力 logit（3.6 节、A.1 节）。标量指标会进到你基线所在的同一个 Trackio project，而路由抓取则留在自己的文件里。NaN 检查本来就开着（`arguments.py#L2825-L2827`）。

需要时才抓的东西

还有两项测量是有界抓取：有问题要问的时候再开。

路由追踪记录每个 token 选了哪些专家（3.1、3.3、A.1 节）。它挂在路由器上的 hook 在 CUDA graph 重放下不会触发（`router_trace.py#L23`），而且默认每步都追（`training_config.py#L363-L367`），所以要把 graph 关掉并给它设边界：

```
--moe-routing-trace-path <dir> --moe-routing-trace-max-training-iters <n>
--cuda-graph-impl none
```

激活 dump 把专家的激活存到同一个 `--save` 目录下，这样你就能离线算逐专家的激活范数（3.4 节、A.1 节）：

```
--save-activations-interval <n>
```

监控开好了，就从每个 token 都要经过的地方开始：路由器。它挑专家，但每个专家在一个 token 的输出里到底占多大分量？

3.1 路由器打分：logits 怎么变成权重

路由器把每个 token 变成每个专家一个 logit，在朴素的 top-k 路由里，top-k 的 logit 决定这个 token 走哪些专家。在这些 logit 和「用来加权专家输出的权重」之间，还夹着四个选择。门控函数、输出缩放（乘到被选专家输出的加权和上的那个常数）、锐度（路由分数的尖锐程度，也就是头号专家压过别人多少）决定每个被选专家在一个 token 输出里占多少分量、以及这个输出有多大。在朴素 top-k 路由下，给定一个 token 的 logits，这四个都不改变「谁被选中」：它们要么保持专家的相对顺序，要么作用在 top-k 之后。

精度是那个例外。一个 bf16 路由器可能把两个很接近的 logit 圆整成同一个值，于是改变了胜者——这就是为什么这里唯一永远该用的设置是 fp32 路由器。

下表列出当前 10 个开源 MoE 在这四项上各自的选择，一行一个模型。

模型	门控函数	输出缩放	锐度	路由器精度：训练中 / 推理代码
DeepSeek-V4.1-Flash	sqrt(softplus)	重归一化，k=6 时 1.5（√k=2.45）	一个温度设置，保持 1.0	未说明；fp32
Gemma 4 26B-A4B	softmax	重归一化，再接一个可学的逐专家缩放；128 选 8（√k=2.83）	归一化路由器输入：RMSNorm + 可学缩放 + 1/√d	未说明；bf16，softmax 在 fp32
GLM-5.3	sigmoid	重归一化，k=8 时 2.5（√k=2.83）	无	未说明；fp32
Inkling	sigmoid，在 top-6 与其 2 个共享专家上联合归一化	8.0 倍一个可学的全局缩放，覆盖 6 个路由专家 + 2 个共享专家（√8=2.83）	无	未说明；bf16
Kimi K3	sigmoid	重归一化，896 里选 16 时 1.0（√k=4）	无	后训练 QAT 中高于 MXFP4；fp32
MiMo-V2.6-Pro-RL	sigmoid	重归一化，k=8 时无系数（√k=2.83）	无	未说明；fp32
North-Mini-Code	sigmoid	不重归一化：权重就是原始分数；128 选 8	无	未说明；bf16
Qwen3.8-Flash-Next	softmax	重归一化，k=10 时无系数（√k=3.16）	无	未说明；bf16，softmax 在 fp32
Step-5-Preview	sigmoid	重归一化，352 里选 8 时 3.0（√k=2.83）	无	未说明；fp32
ZAYA1-8B	对 MLP 路由器的 logits 做 softmax	不重归一化，16 选 1，外加一个 skip 专家	无	RL 期间 fp32 softmax；bf16

先看门控函数。如果它从不改变胜者，那它改变的是什么？

门控函数

softmax、sigmoid 和 sqrt(softplus) 给专家的排序是一样的，所以差别只在「每个被选专家拿到多少权重」。下面这张图把同一个 token 的 logits 从不自信推到自信：softmax 让头号专家占得越来越多，而 sigmoid 在 logits 变大时会饱和，于是一堆自信的专家最后拿到权重相同。当选择只依赖这些分数本身时，门控函数不改变排序。3.3 节的专家偏置是在门控之后加到分数上再去选的（`moe_utils.py#L936-L938`），所以它把一个专家推移多远取决于分数的大小，换个门控函数就可能改掉 top-k。分组受限路由（3.7 节）也会，因为它按各组最好分数的和来选组（`moe_utils.py#L712-L715`）。

> 图 3.1-a 同一个 token 从不自信路由器推到自信路由器：softmax、sigmoid、sqrt(softplus) 的权重曲线（原站为交互图，此处从略）

在图中，sqrt(softplus) 落在 softmax 与 sigmoid 之间：对正的 logits，随着尺度变大，两个权重之比趋向于它们 logits 之比的平方根。DeepSeek-V4 从 sigmoid 换成了 sqrt(softplus)。在 Megatron 里，3.3 节的专家偏置要求 sigmoid 或 sqrt(softplus)：启动时它会拒绝 softmax 配偏置（`transformer_config.py#L3139-L3147`），而它的 softmax 分支从不加这个偏置。提出该偏置的 Wang 等人解释过它为什么和 softmax 不搭：softmax 的归一化让两个专家之间的差距取决于其他每个专家的分数，于是偏置更难调。他们仍然用它在 softmax 路由器上做到了均衡。辅助损失三种门控都能用（`moe_utils.py#L998-L1014`）。

```
# megatron/core/transformer/moe/moe_utils.py#L913-L927（softmax，在 top-k 之前或之后）
if score_function == "softmax":
    if use_pre_softmax:
        scores = torch.softmax(logits, dim=-1, dtype=torch.float32)
    else:
        scores, top_indices = compute_topk(logits, topk, num_groups, group_topk)
        probs = torch.softmax(scores, dim=-1, dtype=torch.float32)
# 同文件 L928-L941（sigmoid / sqrt(softplus)；偏置只参与选择）
elif score_function in ("sigmoid", "sqrtsoftplus"):
    scores = torch.sigmoid(logits.float()) if score_function == "sigmoid" \
             else torch.nn.functional.softplus(logits.float()).sqrt()
    if expert_bias is not None:
        scores_for_routing = scores + expert_bias.float()
        _, top_indices = compute_topk(scores_for_routing, topk, num_groups, group_topk)
```

```
# megatron/core/transformer/transformer_config.py#L3139-L3147（softmax 配偏置直接报错）
if self.moe_router_enable_expert_bias and self.moe_router_score_function not in (
    "sigmoid", "sqrtsoftplus",
):
    raise ValueError(
        "Expert bias for aux-loss-free routing only supports 'sigmoid' and 'sqrtsoftplus' ..."
    )
```

sigmoid 在被选专家的 logits 都很大且为正时，会给它们相同的权重——这也正是模拟里那个被放大后的 token 落到的位置：一个自信的路由器，配上「锐度」那节所说的、平坦的门控。想知道你这次实验的路由器有没有走到那里，就带着 logits 打开路由追踪（A.1 节），逐层跟两个数：头号专家的权重，以及路由熵（一个专家吃掉大部分权重时熵低，权重摊开时熵高）。

什么时候用它：当你用专家偏置做均衡时（3.3 节），用 sigmoid（或 sqrt(softplus)），因为 Megatron 不允许 softmax 配偏置。其余情况用 softmax，也就是 Megatron 的默认值。

路由权重干两件事。它挑专家，同时它又是混合专家输出的那些系数：这一层返回的是「每个被选专家的输出乘权重」再求和。门控函数决定的是「被选专家之间谁的份量更大」，它不决定「它们合起来有多大」：用 softmax 时 top-k 的权重和小于 1，而且每个 token 少多少还不一样，于是这一层输出的量级会跟着晃。那么这个合起来的输出该有多大？

输出缩放

有两个设置决定被选专家合起来的输出有多大。第一个是重归一化：把 k 个被选权重重新缩放到和为 1，输出的量级就不再随这个和摇摆。在 Krajewski 等人的实验里，粒度 8 下重归一化把验证 loss 从 2.219 降到 2.183（粒度 8：专家小 8 倍、数量多 8 倍、每 token 选 8 倍，于是参数和算力不变；3.4 节比较不同粒度）。Megatron 默认就重归一化（`moe_utils.py#L913-L942`），`--moe-router-pre-softmax` 把重归一化关掉。top-1 的 softmax 必须有这个开关：只选一个专家时，对它单个 logit 做 softmax 恒等于 1，路由器就拿不到梯度（`transformer_config.py#L3213-L3221`）。

> 图 3.1-b 模拟：从不自信到自信路由器：不重归一化时输出量级跟着 top-k 概率质量走，重归一化把它压在「一个专家 RMS 的 $1/\sqrt{k}$」到「一个专家 RMS」之间（原站为交互图，此处从略）

第二个设置是路由缩放系数，那个乘在结果上的常数。它之所以必要，是因为即便重归一化，路由输出的量级也取决于每个专家占多少。全部权重压在一个专家上时，输出就是那个专家的均方根（RMS）；权重平均分给 k 个互不相关、RMS 相同的专家时，它掉到前者的 $1/\sqrt{k}$，一个接近 $\sqrt{k}$ 的缩放系数能把量级找回来。DeepSeek-V3 在 k=8 时用 2.5，Kimi K2 用 2.827 $\approx \sqrt{8}$。已发布的模型在取值上并不一致：上表里只有 GLM-5.3 与 Step-5-Preview 贴着 $\sqrt{k}$，而 Kimi 从 K2 的 $\sqrt{8}$ 变成了 K3 在 k=16 时的 1.0。缩放路由输出也会改变它相对于共享专家输出的权重（3.4 节）。

图里最后一行是缩放专家的输入而不是输出：Llama 4 把每个 token 只送给一个专家，并在专家执行前先把这个 token 乘上该专家的 sigmoid 分数。它的 SwiGLU 专家没有偏置，于是输出大致按分数的平方缩小，同时这个分数也把 SiLU 门软化。

```
# megatron/core/transformer/moe/moe_utils.py#L942-L947（先重归一化，再缩放）
probs = scores / (scores.sum(dim=-1, keepdim=True) + 1e-20) if topk > 1 else scores
if scaling_factor:
    probs = probs * scaling_factor
```

```
# megatron/core/transformer/transformer_config.py#L3213-L3221（top-1 必须 pre-softmax）
if (
    self.moe_router_topk == 1
    and self.moe_router_score_function == "softmax"
    and not self.moe_router_pre_softmax
    and self.moe_router_load_balancing_type != "sinkhorn"
):
    raise ValueError("Please use --moe-router-pre-softmax when topk is 1.")
```

不重归一化时，路由输出的量级会逐 token 跟着 top-k 概率质量走——也就是上面那个 loss 更高的设置。所以该盯的是：每个 token 的路由输出有多大，即它权重平方和的平方根。用路由追踪离线算（A.1 节）。

什么时候用它：top-k > 1 时保持重归一化；top-1 的 softmax 要加 `--moe-router-pre-softmax`，否则路由器没梯度。路由缩放系数没有定论：$\sqrt{k}$ 只在「互不相关、权重相等」时恢复路由输出的 RMS，而已发布模型在取值上并不一致。

权重和输出的量级都取决于 logits 有多尖。路由器可能太尖（一个专家吃掉全部权重），也可能太平（把 top-k 平均掉）。是什么决定了它有多尖？

锐度

用 softmax 时，线性路由器会随着 token 的范数或路由器权重的增长变得更尖：logits 被拉开，头号专家吃掉更多权重。下面这张图比较了三种控制手段与线性路由器。logit 软截断主要压缩靠近上限的那些 logits。归一化路由器输入去掉来自 token 范数的那部分。门控 logit 归一化在专家维度上把 logits 标准化，把它们的展布定到一个选定的尺度。

> 图 3.1-c 线性路由器 vs logit 软截断 vs 输入归一化 vs 门控 logit 归一化（原站为交互图，此处从略）

图里那个朴素线性路由器会随 token 范数或路由矩阵增长而变尖；软截断（Grok 2 在上限 30、不重归一化的条件下用它）只有在 logits 接近上限之后，才把大范数 token 的头号权重拉回来。归一化输入（Gemma 4 这么做）让两个 token 走同样的路由，但路由矩阵长大时它仍然变尖；门控 logit 归一化对两者都免疫。Skywork-MoE 用过门控 logit 归一化去治一个平到「这一层把 top-k 专家平均掉」的门，结果训练 loss 适度地低了。归一化路由器输入这一项只与专家 embedding 一起被消融过。

```
# megatron/core/transformer/moe/router.py#L1150-L1152（先加抖动，再走朴素线性门控）
input = self.apply_input_jitter(input)
logits = self.gating(input)
# 同文件 L824-L831（乘性输入抖动）
if self.config.moe_input_jitter_eps is not None:
    eps = self.config.moe_input_jitter_eps
    self.input_jitter = torch.distributions.uniform.Uniform(
        torch.tensor(1.0 - eps, ...), torch.tensor(1.0 + eps, ...)
    ).rsample
    return input * self.input_jitter(input.shape)
```

一个平掉的门（把一个层的 top-k 专家平均掉，让模型学得更差）会表现为 Max1/Max2 趋向 1，也就是头号专家的权重除以第二名的权重——Skywork-MoE 在不做归一化时就是这样。把它和头号专家权重一起逐层盯，数据来自路由追踪（A.1 节）。

什么时候用它：门控变平了，Skywork-MoE 那边是用门控 logit 归一化治好的。在路由器输入这一侧，Megatron 提供 Switch 式的乘性输入抖动 `--moe-input-jitter-eps`。

锐度活在 logits 之间的差距里，而 softmax 对「所有 logit 共同平移」是不敏感的。bf16 不是：如果每个 logit 都一起往上漂，bf16 能表示的值之间间隔就变大，靠得近的专家可能并列。这些并列会改变谁被选中吗？

精度

top-k 对并列专家的排序方式，PyTorch 并没有规定。下面这张图模拟了用 bf16 路由器给同样的 token 在 64 个专家上做 top-8，分别在有 z-loss 和没有 z-loss 的情况下——z-loss 是对 logits 的

logsumexp 的惩罚项。模拟里所有 logits 共同上漂一个偏移量，然后统计有多少 token 的专家与「用 fp32 路由器选」不同。

> 图 3.1-d 共同偏移量上升时，bf16 路由器选出的专家与 fp32 的差异（有/无 z-loss）（原站为交互图，此处从略）

随着模拟偏移量上升，bf16 路由器的 logits 落到更粗的格点上，越来越多 token 的专家与 fp32 不同。z-loss 通过把 logits 往负方向推来压低 logsumexp。在 64 选 8 时，第 8 个 logit 大约比 logsumexp 低 3.5，于是模拟里带 z-loss 时它落在约 −2.2、不带时落在 +3.6，两个量在 bf16 里被按同样长度的步长圆整，改变的 token 数也差不多；只有 fp32 路由器没有翻转，因为 fp32 的圆整细 65,536 倍。DeepSeek-V3.2、GLM-4.5 和 Kimi K2.5 在它们的推理代码里用 fp32 算路由器。fp32 的理由建立在训练证据上，而那件事关稳定性：在 Switch Transformers 里，bf16 路由器发散了，而 fp32 路由器以 bf16 的速度训完了。在 ST-MoE 里，路由器 z-loss 让每次实验都稳住了。各家实验室很少说他们用什么精度训路由器：上表 10 份报告里只有 2 份提到，而 10 份里有 5 份在推理代码里用 bf16 做路由器那次矩阵乘。在这一层其余部分能降到多低位宽、同时路由器保住精度，是 3.7 节「精度」会回来讲的问题。

```
# megatron/core/transformer/moe/router.py#L124-L130（路由器精度）
router_dtype = input.dtype
if self.config.moe_router_dtype == 'fp32':
    router_dtype = torch.float32
elif self.config.moe_router_dtype == 'fp64':
    router_dtype = torch.float64
logits = router_gating_linear(input, self.weight, self.bias, router_dtype)
# megatron/core/transformer/moe/moe_utils.py#L171-L184（z-loss）
logsum = torch.logsumexp(logits, dim=-1)
z_loss_values = torch.square(logsum)
```

bf16 路由器能让训练发散（Switch Transformers 就是），所以有两个信号值得跟：z-loss，它追踪 logits 上漂了多少；以及翻转率，即用 fp32 重算路由器时有多少 token 换了专家。设了 `--moe-z-loss-coeff` 时 Megatron 会记录 z-loss，且已经除过系数（`router.py#L805-L807`），所以它的平方根就是 logsumexp 的 RMS：我们那次实验记到 20.9，对应 RMS logsumexp 约 4.6。翻转率需要同一批输入、同一个当前路由器状态下的配对抓取；A.1 节给了一个有界的实时 hook，因为追踪里那份初始权重快照跟不上后来的训练更新。

什么时候用它：永远设 `--moe-router-dtype fp32`。对 logits 上漂的 softmax 路由器，加上 `--moe-z-loss-coeff 1e-3`（ST-MoE 的取值）。DeepSeek-V3 的报告从没为它的 sigmoid 路由器提过 z-loss。

该盯什么

部位	症状	该盯的信号
门控函数	—	逐层的头号专家权重与路由熵，来自带 logits 的路由追踪（A.1 节）
输出缩放	「学不好」：不重归一化时，路由输出随 top-k 概率质量缩放；重归一化把 loss 降低了	每个 token 的路由输出有多大，$\sqrt{(\Sigma w^2)}$，来自路由追踪（A.1 节）
锐度	「学不好」：门控变平时，这一层把它的 top-k 专家平均掉	逐层的头号专家权重与 Max1/Max2（头号权重除以第二名），来自路由追踪（A.1 节）
精度	「炸了」：bf16 路由器在 Switch Transformers 里发散	记录的 z-loss 已除过系数，平方根即 RMS logsumexp：20.9 对应约 4.6；fp32 翻转率用 A.1 节那个配对实时 hook 量，同一批输入 + 当前路由器状态（追踪里的初始快照无法精确重放后来的前向）

到这里我们塑造了「一个 token 的 logits 怎么变成权重」：门控函数决定每个被选专家占多少，重归一化和缩放系数决定它们合起来多大，锐度与精度决定分数有多尖、多准。这一切都假定了一种设定：一个学出来的路由器，由 token 挑它的 top k。两者都能改：专家可以反过来挑 token，固定的表也可以替掉学出来的选择。那么该由谁来做选择，token 还是专家？这个选择一定要学吗？

3.2 路由策略：谁来选，以及怎么选

token 选 top-k 是默认做法：每个 token 给所有专家打分，留下最好的 k 个。本节看两种改法。让专家去挑它的 token，而不是 token 挑专家；或者干脆用一个固定哈希来挑，不要学出来的路由器。

当前开源 MoE 怎么挑专家，以及哪些会按 token id 查表（附 Zhou 等人的研究模型作为 expert choice 的对照）：

模型	谁选	按 token id 查表
DeepSeek-V4-Pro	token choice	前 3 个 MoE 层用 hash 路由
DeepSeek-V4.1-Flash	token choice	第 1、14 层有 Engram n-gram 记忆（2–4 gram）；没有 hash 路由
EC-CF2 8B/64E（Zhou 等）	expert choice	无
GLM-5.3	token choice	无
Kimi K3	token choice	无
Llama 4 Maverick	token choice	无
MiMo-V2.6-Pro-RL	token choice	无
Qwen3.8-Flash-Next	token choice	第 1 层有 n-gram embedding（2–3 gram）
Step-5-Preview	token choice	无

在问「这个选择需不需要学」之前，得先定「由谁来做」：因为 token choice 下，很多 token 可能都想要同样的专家，负载就歪了。到底该让每个 token 继续挑它的专家，还是让每个专家去挑它的 token？

谁来选

token choice 下，很多 token 可能同时想要同一批专家，负载于是歪掉。expert choice 把角色反过来：每个专家挑它最好的那些 token，于是每个专家的负载一样。代价是未来 token 泄漏：一个 token 能不能拿到某个专家，取决于它后面的 token。推理时那些 token 还不存在，于是路由和训练时不一样。

图里的第三条规则「专家阈值」在不去看未来 token 的前提下把负载压得接近均匀，代价是每个 token 拿到的专家个数不固定。在一个 5.75 亿参数的模型上，它的 loss 比 token choice 更低——不管有没有均衡（2.844 对约 2.89）。

> 图 3.2-a token choice / expert choice / expert threshold 三者对比：序列偏向专家 1 时各自的负载（原站为交互图，此处从略）

图里这条序列偏向专家 1，token choice（画成不配均衡器）把它压爆了。expert choice 把负载摊平了，但后面的 token 会把前面已经选中的挤出去；而专家阈值——它的截断值来自此前同样偏向的批次——不修改任何选择就把负载保持在目标附近，代价是每 token 的专家数不定。

```
# megatron/core/transformer/moe/moe_utils.py#L891-L897（逐 token top-k，只有 token choice）
# Sorting top-k turned off during inference
return torch.topk(
    scores, k=topk, dim=1,
    sorted=torch.is_grad_enabled() or is_batch_invariant_mode_enabled(),
)
```

token choice 留下的偏斜会让这一步变慢。它体现在 token 计数上，也就是逐层逐专家的 token 数，`--save-tokens-per-expert-interval` 会记录（A.1 节），最好读成 MaxVio：最忙的专家比均值高出多少（MaxVio 是 maximal violation 的缩写：最忙专家的负载减均值，再除以均值。0 是完全均匀；1 表示最忙的专家拿到平均的两倍。提出 3.3 节那个专家偏置的 Wang 等人就是这么定义的）。

什么时候用它：用 token choice，这也是 Megatron 训练的那种，k 由 `--moe-router-topk` 定。

不管由 token 还是专家来挑，到目前为止这个选择都来自一个学出来的路由器，于是路由本身需要训练。路由真的需要学吗，还是更简单的规则就能挑专家？

哈希层

在 DeepSeek-V4 的前 3 个 MoE 层里，一张固定的表把每个 token id 映射到它的专家，路由器只保留用来算权重。这张表是 token id 的固定哈希，在我们的探测里，它对高频 token 的摊开效果不比随机表好。选择这件事没有什么可学的，而 hash 层在最大到 45 亿参数的模型上，与 Switch 层、BASE 层（Lewis 等人提出的均衡分配，靠解一个线性指派问题让每个专家分到相同数量的 token）打平。下面这张图把学出来的路由与 hash 路由并排，上面再放第三种按 token id 查表的方式：Engram，它把一个 token 前面 2–4 个 token 哈希进若干 embedding 表，再把读到的行加到隐状态上。DeepSeek-V4.1-Flash 只在两层保留 Engram，而每一层的专家仍然由学出来的路由器挑。

> 图 3.2-b 学出来的路由 / hash 路由 / Engram 三种按 token 找专家的方式（原站为交互图，此处从略）

图里两个路由器对「的」这个字的处理不一样：学出来的路由器按上下文逐个选，而 hash 层把每一个「的」都送到它表里那一行指定的 6 个专家，哪怕路上有个分数更高的专家。两行都把 6 个专家的权重归一化到和为 V4-Flash 的路由缩放系数 1.5，图里用平均 Jaccard 相似度衡量四个「的」的专家集合有多像。hash 层放弃的是上下文：同一个 token id 永远选同一批专家，周围是什么都一样。它的扩展性也比学出来的路由差。Clark 等人改变了一个模型里有多少层是 MoE 层：只有一层 MoE 时，hash 路由在他们比较过的方法里 loss 最低；但随着 MoE 层变多，它不再变好，而他们学出来的路由器还在继续变好。而且没有任何偏置能把它掰回来：高频 token 能把分配给它们的专家压爆，没人能动它们。V4.1-Flash 因此放弃了 hash 层，只留下 Engram——而 Engram 不挑任何专家。

在 Megatron 里，`--moe-num-hash-layers` 让前几层 MoE 变成 hash 层（`router.py#L304-L308`）。它们从一张 token-to-expert 表（`tid2eid`）里读每个 token 的专家，表里词表每个条目一行；如果你不加载，Megatron 会用轮转的占位表填上（`router.py#L313-L329`）。这些层里专家偏置是关的，训练中没有东西去纠正一张坏表，所以表本身很要紧。Roller 等人比较过几种建表方式：随机表和「把最高频的

token 贪心地摊到最空的专家上」那种，困惑度差不多，后者更均衡；而按 embedding 聚类把相似 token 分到一组，明显更差。

```
# megatron/core/transformer/moe/router.py#L304-L308（前 N 层走哈希）
self.is_hash_layer = (
    not self.is_mtp_layer
    and hash_moe_layer_threshold > 0
    and layer_number <= hash_moe_layer_threshold
)
# 同文件 L322-L326（轮转占位表）
token_ids = torch.arange(self.config.hash_moe_vocab_size, device=self.weight.device)
expert_offsets = torch.arange(self.topk, device=token_ids.device)
self.tid2eid = ((token_ids[:, None] + expert_offsets) % self.num_experts).to(torch.int32)
```

> 另外还需要几行：从 Hub 加载 V4-Flash 的 `layers.0.ffn.gate.tid2eid`，并统计逐专家 token 数。

一张把某些专家压爆的表会让这一步变慢，而训练中没有任何东西去重新均衡它，所以要在实验早期就去查：把 hash 层里的逐专家 token 数与学习层里的比一比，数据来自 `--save-tokens-per-expert-interval` 写出的 token 计数（A.1 节）。

什么时候用它：只用来复现 DeepSeek-V4 的头几层。像 V4 那样加载一张固定的哈希表，或者一张按频率均衡的表，而不是 Megatron 那个轮转占位表。

该盯什么

部位	症状	该盯的信号
谁来选	「慢」：token choice 下有些专家分到的 token 比别人多。「学不好」：expert choice 下推理时的路由与训练时不同	逐层的 MaxVio——最忙专家比均值高多少，来自 token 计数（A.1 节）
哈希层	「慢」：高频 token 能把哈希分给它们的专家压爆，而偏置动不了它们	hash 层的逐专家 token 数，与学习层对比，来自 token 计数（A.1 节）

expert choice 与专家阈值能让负载均匀或接近均匀，而偏置动不了一张哈希表的负载。Megatron 训的是学出来的 token choice，在这里，迄今为止还没有什么东西能阻止大量 token 挤到同几个专家上。专家挤爆是 MoE 第一个真正的失效模式，那么在路由器还在学的过程中，怎么让 token 均匀地摊到各个专家上？

（负载均衡见下一节。）

导航：[[2026-11-moe-handbook|全书目录]] · ← 第 2 章 环境与第一次跑通 · 第 3 章（中）负载均衡 →

第 3 章 解剖 MoE 训练（中）：负载均衡

> 说明：原站里的图都是交互式组件，这里无法复现，一律保留图注并标注「（原站为交互图，此处从略）」。原站未给图编号，本文按节内顺序编号（如 图 3.3-a）。

3.3 负载均衡

当大量 token 都挑了同一批专家，会坏两件事。一是装着最忙专家的那块 GPU 活最多，别的 GPU 得等它，于是它决定了整个步的时间；二是如果路由器一直把 token 发给同几个专家，剩下的专家几乎收不到 token，参数就基本白放着。第一种代价更好量：把路由强制摊平之后，同一个模型在初始化时就快了 1.5 倍。

强制摊平只是把「该追求的速度」摆出来，因为它把路由器学到的东西全扔了。要在路由器继续学习的前提下逼近那个速度，有两类办法：辅助损失（aux loss）和免损失偏置（loss-free bias），两者均衡负载的方式不同。辅助损失加一项奖励均匀负载的损失，于是路由器的梯度会把它的概率往「摊开 token」的方向推，代价是这个梯度会和语言模型损失（LM loss）抢方向。偏置则只改变「哪些专家被选中」。不管你选哪种，它在多大范围上均衡（一条序列、一个 micro-batch、还是全局批次）决定了「均衡」到底是什么意思。

各家模型作者报告的均衡方案。辅助损失与选择偏置可以同时用。「范围」指的是被均衡的那些 token；MiMo 那一行讲的是 RL 而不是预训练。

模型	辅助损失	范围（scope）	选择偏置	分位数均衡
DeepSeek-V3	小的 sequence-wise loss，系数 0.0001	批次级偏置 + sequence-wise loss	固定步长的专家偏置	否，固定步长更新
DeepSeek-V4.1-Flash	小的 sequence-wise loss，系数 0.0001	文本与图像的负载分别均衡，外加 sequence-wise loss	每种模态各自一个偏置，更新速率 0.001	用的是无辅助损失的偏置策略
Kimi K3	描述为「无辅助损失」	全局批次	由分数间距推出的偏置，下一步生效	是，全局批次直方图估计
LongCat-Flash	设备级均衡损失	全局批次 FFN 目标 + 序列内设备级均衡	PID 控制的专家偏置；恒等专家不参与偏置更新	否，PID 更新
MiMo-V2.6-Pro-RL	RL 期间路由器冻结	保留 RL 之前的路由器	报告评估的是冻结，而非新的偏置更新规则	RL 没有报告新的均衡更新
Qwen3 MoE	全局批次均衡损失	全局批次	公开的路由器里没有选择偏置	否，用辅助损失路线

在逐一比较这些均衡器之前，得先知道「负载不均可劲儿到掉多少速度」，因为下面每个均衡器都该拿「强制摊平时的吞吐」当尺子。那个 1.5 倍是从哪来的，又怎么在我们自己的机器上量？

负载不均的代价

为了看清不均的代价，我们把同一个模型按三种方式路由。强制均摊（Megatron 的 `--moe-router-force-load-balancing`）把路由器的 logits 换成随机值，于是每个专家分到的 token 差不多，这就是最好的情况：149.4 TFLOP/s。

强制偏斜（`--moe-router-force-biased`）在这些随机 logits 上加一个所有人共享、逐渐变大的随机偏置，于是少数专家分到越来越多的 token——相当于一个「不均衡程度」的旋钮。拧到头吞吐掉到 100.5 TFLOP/s；把那次路由回放出来（它的辅助损失与日志里的对上），最忙的专家接走了每一个 token，MaxVio 为 7，这已经是 64 选 8 能达到的最歪。还能更歪：回放里最忙的几个专家分散在几块 GPU 上，所以最忙的那块只承担了约两倍的平均负载；如果它们全坐在一块 GPU 上，那块 GPU 就得干绝大部分专家活。

学出来的路由器——也就是你真正训练时会用的那个——离这个最好情况很远：一个刚初始化的路由器，头 40 个迭代跑得和最差的强制偏斜一样慢。下图把三者放在一起比。训练要做的，就是在不强制路由器选择的前提下去补上这段差距，本节余下部分就用「把一个学出来的路由器带到多接近强制均摊的吞吐」来比较各种均衡方法。

> 图 3.3-a 强制均摊 / 强制偏斜（偏置逐步变大）/ 学出来的路由器：吞吐（原站为交互图，此处从略）

图里吞吐随强制偏置变大而下降，因为少数专家吃掉了更多 token；而学出来的路由器从最歪那条强制曲线的水平起步。最忙的专家决定了步长：All-to-All 会把每个 GPU 本地专家收到的 token 全送过去（我们这次是 64 个里的 8 个），下一次 All-to-All 要等最慢的那块 GPU。一块 GPU 的负载是它本地那些专家之和，所以它可以比逐专家的负载更均衡。这也是为什么均衡损失可以按 GPU 算而不是按专家算：DeepSeekMoE 的设备级均衡损失把每块 GPU 的专家并成一项，下一张图把它和专家级损失放在一起比。

> 图 3.3-b 玩具批次（每块 GPU 两个专家，其中一个是 LM 偏爱的四个之一）：专家级损失 vs 设备级损失（原站为交互图，此处从略）

在一个玩具批次上（每块 GPU 两个专家，其中一个属于 LM 偏爱的四个），两种损失都能把 GPU 摊平，而设备级损失把路由器拉离「LM 想要的」更少，因为它只看每块 GPU 的总量，而这个摆法让这个总量更接近均匀。问题在于每块 GPU 内部的专家仍然不匀，单个专家仍可能占掉这块 GPU 的大多数 token。DeepSeekMoE 给的也是同一个理由去给两种损失不同的权重：严格的专家级均衡会伤模型质量，所以它把设备级的系数设大、把专家级的系数留小，用来防路由崩塌。DeepSeek-V2 用的是 0.05 对 0.003。DeepSeek-V3 的报告里已经不再提设备级损失：它用一个偏置加一个小小的 sequence-wise 损失来均衡每个专家，两者本节后面都会讲。

同样的不均在服务端也是时间成本，所以生产环境里的前沿 MoE 会把它压得很小：DeepSeek 在服务 DeepSeek-V4.1-Flash 时，最慢的那块 GPU 最多只到平均的 1.09 倍。

```
# megatron/core/transformer/moe/router.py#L1185-L1193（前向里切换强制路由）
if self.config.moe_router_force_load_balancing:
    # Apply force load balancing with random logits for benchmark
    logits = apply_random_logits(logits)

if self.config.moe_router_force_biased is not None:
    # Apply biased logits with shared random bias across all ranks
    logits = apply_biased_logits(
        logits, self.config.moe_router_force_biased, self.layer_number
    )
```

```
# megatron/core/transformer/moe/moe_utils.py#L1301-L1314（每个 rank 各自的随机 logits）
@staticmethod
def forward(ctx, logits: torch.Tensor) -> torch.Tensor:
    with get_cuda_rng_tracker().fork(get_expert_parallel_rng_tracker_name()):
        random_logits = logits.clone().normal_()
    return random_logits
```

```
# megatron/core/transformer/moe/moe_utils.py#L1353-L1364（每层共享的随机偏置）
@staticmethod
def forward(ctx, logits, std, layer_number):
    """Forward pass: apply random bias to logits."""
    if std < 0 and layer_number in RandomSTEShared._cache:
        return logits + RandomSTEShared._cache[layer_number]

    # Generate random bias with shared seed across all ranks
    with get_cuda_rng_tracker().fork(get_data_parallel_rng_tracker_name()):
        bias = torch.empty(logits.shape[-1], device=logits.device,
                           dtype=logits.dtype).normal_(std=abs(std))
```

负载不均表现为「步变慢」，所以要盯的数字是相对强制均摊对照跑的 TFLOP/s。想看丢的时间去哪了，就看 3.7 节那样剖析一个步：那次剖析跑的是不均衡的路由器，3.7 节演示怎么判断是不是不均衡在拖慢它的 All-to-All。

什么时候用它：每套配置跑一次，当对照用。`--moe-router-force-load-balancing` 给出均衡时的吞吐，拿来当所有均衡器的尺子。永远不要用它训练。

强制路由能告诉你「均匀负载能跑多快」，但它把 logits 换成了随机值，所以只能当对照，不能拿来训练。那么，怎么在训练过程中把一个学出来的路由器引向均匀负载？先从最老的办法说起：加一项损失。

辅助损失

最老的办法是加一项损失，也就是 aux loss：系数 × 专家数 × 对专家求和，每项是「该专家在 micro-batch 里占的 token 比例」乘「它的平均路由概率」（`moe_utils.py#L148-L152`）。它的梯度把路由概率

从「LM 想要的」拉向「均匀负载」。下面用四种设置跑同一个批次：不均衡、小系数的 aux loss、大系数的 aux loss、以及无损失的偏置。

> 图 3.3-c 同一个批次在四种设置下的路由：不均衡 / aux 小系数 / aux 大系数 / 无损失偏置（原站为交互图，此处从略）

图里两个均衡器都用不同的方式把负载摊开了：aux loss 把路由概率从 LM loss 想要的位置拉开，系数越大拉得越远；而无损失偏置只改变「挑哪些专家」，均衡得最好，但它改掉的「选择」也最多。aux loss 这种「拉扯」让系数成了一个取舍：系数太大伤 LM loss，系数太小负载仍然不匀。Yuan3.0 在 10B 模型上对 DeepSeek-V3 那个 sequence-wise aux loss 做了消融（下一张图），两面都摆出来了：测试损失从系数 1e-3 到 0.1 一路升高；而最小的系数 1e-4 比完全不加 aux loss 略好一点，但一层里最忙的专家仍然拿到最闲专家的约 13.6 倍 token。

> 图 3.3-d Yuan3.0 的消融：测试损失与 aux 系数（原站为交互图，此处从略）

图里最小的系数留下了不均衡，而真正把负载压平的那个 1e-2 又把测试损失抬高了。Megatron 用 `--moe-aux-loss-coeff` 配 `--moe-router-load-balancing-type aux_loss` 设系数，每种损失类型一个系数，在所有层都生效。Motif 3 发现它最前面几层 MoE 更容易失衡，于是给它们更强的辅助损失权重。最前几层的问题本章后面还会回来讲。

```
# megatron/core/transformer/moe/moe_utils.py#L148-L152（Σ f·P 乘系数）
aggregated_probs_per_expert = probs.sum(dim=0)
aux_loss = torch.sum(aggregated_probs_per_expert * tokens_per_expert) * (
    num_experts * moe_aux_loss_coeff / (topk * total_num_tokens * total_num_tokens)
)
return aux_loss
```

```
# megatron/core/transformer/moe/router.py#L515-L536（在 micro-batch 内计数）
aux_loss_coeff = self.get_aux_loss_coeff("aux_loss")
if aux_loss_coeff == 0:
    return probs

global_tokens_per_expert, local_num_tokens, total_num_tokens = (
    get_tokens_per_expert_and_token_count(
        routing_map=routing_map,
        reduce_group=self.tp_cp_group,
        topk=self.topk,
        with_padding_mask=with_padding_mask,
    )
)

aux_loss = switch_load_balancing_loss_func(
    probs=scores_for_aux_loss,
    tokens_per_expert=global_tokens_per_expert,
    total_num_tokens=total_num_tokens,
    topk=self.topk,
    num_experts=self.config.num_moe_experts,
    moe_aux_loss_coeff=aux_loss_coeff,
    fused=self.config.moe_router_aux_loss_fusion,
)
```

```
# megatron/core/transformer/moe/router.py#L459-L472（按损失类型取系数）
def get_aux_loss_coeff(self, aux_loss_type: str) -> float:
    if isinstance(self.routing_type, str):
        if self.routing_type == aux_loss_type:
            return self.config.moe_aux_loss_coeff
    if isinstance(self.routing_type, list):
        try:
            idx = self.routing_type.index(aux_loss_type)
            return self.config.moe_aux_loss_coeff[idx]
        except ValueError:
            return 0.0
    return 0.0
```

上面四列图下面那行实测数据来自 Wang 等人——他们提出了无损失偏置，并在一个 1B、64 个路由专家、top-6 的 MoE 上与逐序列 aux loss 作比。在他们的基线系数 1e-3 下，验证集上的 MaxVio 是 0.72、困惑度 9.56；而用偏置是 0.04 与 9.50。完全不加 aux loss 时 MaxVio 升到约 5.1；系数 1e-2 时降到约 0.1，困惑度升到约 9.69。

这个系数 1e-3 的 aux loss，就是「负载不均的代价」那一节里的学出来的路由器，它跑完 120 个迭代爬到了 125 TFLOP/s。

系数设太高模型学得更差，设太低负载又不匀，所以 LM loss 和均衡度要一起看。而均衡度最好从 MaxVio 读，别从 aux loss 本身读：这项损失把每个专家占的 token 比例乘上它的平均路由概率，于是当路由概率是平的，不管 token 比例怎么分，这项损失读起来都和均匀负载一样——一个「完美均衡」的读数可能盖住一个歪掉的负载。Sigma-MoE-Tiny 就踩了这个坑：它下层把概率压平让这项损失降到最

小，而负载一直不匀。`--moe-per-layer-logging` 会把这个值按层记成 `load_balancing_loss`，且已除过系数。

什么时候用它：图省事就用它——`aux_loss`，系数取 1e-3 左右，也就是 2.5 节我们第一次 MoE 实验那样。负载还歪就调高，LM loss 受影响就调低。

到目前为止用的 aux loss 数的是每个专家在一个 micro-batch 内的负载，于是留下一个还没问的问题：负载该在哪些 token 上是均匀的？可以按每条序列、按每个 micro-batch、或按整个全局批次均衡，下面就来比这三种。

范围

想一想一个 micro-batch 里装的是什么。预训练流水线经常把每个 micro-batch 装成单一领域，比如全是代码；那么一个「在这个 micro-batch 内要求负载均匀」的损失就会把代码 token 摊到所有专家上，于是没有专家能专门长成「代码专家」。改成在整个全局批次上均衡——那里混着很多领域——代码 token 就可以挤在少数几个专家上，只要其他领域的 token 把剩下的位置填满。Qiu 等人只在第二种设置下看到了专家按领域分化，而且困惑度也更低。GPU 为此付账：每个 micro-batch 内部负载不再均匀，他们那个 43B 模型的步慢了约 6%，其中约 1% 来自通信开销，其余主要来自不均衡。

一条序列是最紧的范围。逐序列均衡会把每条序列摊到所有专家上，哪怕这个 micro-batch 混着多个领域；DeepSeek-V2-Lite 把它唯一的那项均衡损失（专家级）就是这么算的，DeepSeek-V3 则保留一个小的，用来防止「整个批次看起来均衡、但单条序列全挤在几个专家上」。下面的图让三种范围并排跑，每块 GPU 各装两条来自两个不同领域的序列：范围越紧，专家越难集中在某一个领域；范围越松，每块 GPU 承担的负载越不匀。

> 图 3.3-e 三种范围的对照：逐序列 / 逐 micro-batch / 全局批次（原站为交互图，此处从略）

那个模拟里，sequence-wise 损失把代码序列摊到所有专家上，micro-batch 损失让每块 GPU 的两个领域各分走一半专家，而 global-batch 损失把代码 token 基本留在了它们偏爱的两个专家上，代价是每块 GPU 的负载最不匀。Megatron 通过均衡类型来选范围：`aux_loss` 按 micro-batch，`seq_aux_loss` 按序列，`global_aux_loss` 按全局批次。要组合多种范围，就把几个类型和同样顺序的系数一起传，例如 `--moe-router-load-balancing-type aux_loss seq_aux_loss --moe-aux-loss-coeff 1e-3 1e-4`。

```
# megatron/core/transformer/moe/router.py#L567-L589（按序列计数，再除以 bsz）
scores_for_aux_loss = scores_for_aux_loss.reshape(seq_length, -1)
routing_map = routing_map.reshape(seq_length, -1)

global_tokens_per_expert, local_num_tokens, total_num_tokens = (
    get_tokens_per_expert_and_token_count(
        routing_map=routing_map,
        reduce_group=self.tp_cp_group,
        with_padding_mask=with_padding_mask,
        topk=self.topk * bsz,
    )
)

aux_loss = (
    switch_load_balancing_loss_func(
        probs=scores_for_aux_loss,
        tokens_per_expert=global_tokens_per_expert,
        total_num_tokens=total_num_tokens,
        topk=self.topk,
        num_experts=self.config.num_moe_experts,
        moe_aux_loss_coeff=seq_aux_loss_coeff,
        fused=self.config.moe_router_aux_loss_fusion,
    )
    / bsz
)
```

```
# megatron/core/transformer/moe/router.py#L617-L628（全局计数 + 滑动平均）
global_tokens_per_expert, local_num_tokens, total_num_tokens = (
    get_tokens_per_expert_and_token_count(
        routing_map=routing_map,
        reduce_group=self.tp_dp_cp_group,
        with_padding_mask=with_padding_mask,
        topk=self.topk,
    )
)

self.global_tokens_per_expert += global_tokens_per_expert
self.ga_steps += 1
averaged_tokens_per_expert = self.global_tokens_per_expert / self.ga_steps
```

每种范围都有它自己的失败方式。按序列或按 micro-batch 均衡会妨碍「领域专家」长出来，模型学得更差；而按全局批次均衡会让每块 GPU 的负载不匀，步变慢。不管你选哪个范围，都在那个范围上读 MaxVio：逐 micro-batch 用 `--save-tokens-per-expert-interval` 存下的负载算，逐全局批次用那些负载的和算，逐序列则离线用路由追踪算（A.1 节）。

什么时候用它：默认用 micro-batch（`aux_loss`）。只有当你想要「按领域分化的专家」时才用 `global_aux_loss`，代价是 Qiu 等人那里步时长多了 5.8%。

不管选哪个范围，均衡损失终究是通过梯度去指挥路由器，而这个梯度会和 LM loss 的梯度抢方向。有没有办法完全不碰梯度就把负载均衡掉？

无损失偏置

aux loss 是靠把路由概率从 LM 想要的位置拉开，来救一个过载的专家。无损失偏置不碰概率，用零梯度去搬负载。给每个专家一个偏置，在 top-k 之前加到它的分数上，且只用来挑专家；权重仍然来自分数。每步一次，Megatron 把每个偏置按固定量（更新速率）挪一下：欠载的往上、过载的往下。下一张图跑的就是这个循环。在 DeepSeek-V3 的消融里，它在验证损失上与批次级 aux loss 打平，1B 和 3B 都如此。和 aux loss 一样，它也会改掉一些「路由器本来会做的选择」，在上面 aux loss 那节的四列玩具里，它改得最多。

> 图 3.3-f 16 个玩具专家的偏置更新循环（步长远大于 Megatron 默认值）（原站为交互图，此处从略）

图里跑 16 个玩具专家，步长比 Megatron 默认值大得多，于是几轮就能代替很多轮：过载的专家把偏置降下来、欠载的抬上去，直到负载接近均匀；把步长缩小十倍之后，大步长留下的来回震荡就被压住了。真实训练里的麻烦在「固定步长」这件事上，它必须匹配专家数量。Li 等人扫了小 MoE 的专家数：更新速率 1e-3 时，最忙专家的负载随专家数增长，到 1,024 个专家时约为均值的 50 倍；而单用 1e-2 的 aux loss 时保持在约 7 倍以下。超过 128 个专家后，偏置还伤 LM 质量，作者怀疑是步长对这种规模的专家来说太粗。

上面那张图里过载曲线上的参考线来自同一批 1B 实验，是在每个训练批次上量的、而不是在验证集上：aux loss 系数 1e-3 时 MaxVio 稳在 0.72 附近，用偏置时稳在 0.1 附近。

Megatron 的规则是你能写出的最简单那种：每一步，每个偏置都挪同一个固定量，不管这个专家离均值负载有多远。这叫 sign update（符号更新），但它不是跑这个循环的唯一方式。线性更新按专家离均值的距离成比例地挪，于是严重过载的专家得到大修正、快均衡的得到小修正；AdamW 更新则为每个专家自适应步长。LongCat-Flash 用的是线性更新，目标是他算力预算所对应的专家负载；ZAYA1-8B 把每个专家的负载误差喂给 AdamW。

这些规则还有一个更微妙的差别：当一次更新的步长加起来不等于零时，所有偏置的均值会漂移。这个漂移不改变任何选择（给每个专家的分数加同一个数，top-k 不变），但偏置若用 bf16 存就会损失精度；Megatron 用 fp32 存。Moonlight 和 Trinity Mini 靠减去步长的均值来避免漂移。发布出来的权重也是同样的差别：DeepSeek-V3 的偏置每层均值在 2.2 到 8.0 之间，而 Moonlight 和 ZAYA1-8B 的平均为 0，LongCat-Flash 的则略低于 0。下面把符号更新、居中符号更新、线性更新、AdamW 更新跑在同一个批次上。

> 图 3.3-g 四种偏置更新规则在同一批次上的表现（原站为交互图，此处从略）

图里那个玩具批次有几个很热的专家、一堆恰好略低于均值的专家，所以大多数符号步朝上：符号更新和 AdamW 更新把偏置均值抬起来，而居中符号更新与线性更新把它按在零；但符号更新在每一步的均衡效果都和居中版一样，因为 top-k 不在意所有专家共同的那点平移。不管用哪条规则去搬，偏置只能均衡它数到的那些 token。批次级偏置是在一个批次上拟合、作用到下一个批次的，所以它能让整个批次看起来均匀，却不保证每条序列都均匀：单条序列仍可能挤在几个专家上。显而易见的补丁是再来一个「按整条序列算」的偏置，但那会用到这条序列里后面的 token，把未来泄漏到前面 token 的路由里。

Chang 的因果偏置靠「只用这条序列已经路由过的部分」来避免泄漏。它在每条序列开头重置，每处理一个 token，就把刚被选中的专家的偏置降一点、其余的抬一点，于是每个专家的偏置记录着「它到目前为止被选中的次数，离应得份额（每个 token k/N）高多少或低多少」，一条序列里被用超了的专家，在后面的 token 上就被压下去。和批次偏置一样，它只改变「挑哪些专家」，从不被梯度训练。下一张图用同一条序列，分别只用全局批次偏置、以及再加上这条序列自己的因果偏置。

> 图 3.3-h 全局批次偏置 vs 加上因果偏置（原站为交互图，此处从略）

图里那条序列在中途切到代码。只靠全局批次偏置时，它的代码 token 挤到一个专家上——尽管整个批次仍然均衡；加上因果偏置后，这些代码 token 摊到更多专家上，这条序列的负载也接近均匀得多。另一面，模拟显示因果偏置把更多选择送给了「路由器自己 top-k 之外的专家」，偏离了路由器自己的偏好；而且支持它的证据规模很小，来自 Chang 在 nanochat 和 TinyStories 上的实验。

已发布的模型对「单条序列」有不同处理。DeepSeek-V3 把偏置和前面那个 sequence-wise aux loss 配在一起，系数小到 1e-4：偏置维持批次均衡，那个小损失防单条序列内部的极端不均衡。下一张图在一条普通序列和一条极端序列上比较「不加这个损失 / 小系数 / 大系数」。

> 图 3.3-i 单条序列上的三种设置：不加 / 小系数 / 大系数（原站为交互图，此处从略）

那个玩具里两个系数只是示例、并非 DeepSeek-V3 的取值：小系数能把普通序列拉近均匀，同时只把路由器挪一点点，但极端序列仍然挤在一个专家上；要把那条摊开就得用大系数，而大系数会把路由器拽得离 LM 损失偏好的路由远得多。一条序列是「批次级计数会盖住不均衡」的一种情形，另一种是模态。

当一个批次混着多种模态，在全部 token 上计数的偏置可以把它们的总和均衡掉，而单独看某一模态的 token 仍然挤在几个专家上。DeepSeek-VL2 和 Kimi-VL 对所有 token 只用一个偏置，DeepSeek-V4.1-Flash 每种模态一个。下图比较「一个共享偏置」与「每种模态一个偏置」（各自只数自己模态的 token）。

> 图 3.3-j 共享偏置 vs 每模态一个偏置（原站为交互图，此处从略）

那个玩具让文本偏向 8 个专家里的 4 个、图像偏向另外 4 个，于是共享偏置能把总量均衡掉，因为两个模态的超额互相抵消；而每模态一个偏置则各自均衡。拆开偏置的代价是：它逼每个专家在每种模态里都分到自己那份，从而妨碍了「专门做图像」这件事。

在 Megatron 里跑它：用 `--moe-router-enable-expert-bias` 打开偏置，用 `--moe-router-bias-update-rate` 设固定步长，并且要用 sigmoid 分数（`--moe-router-score-function sigmoid`），因为 Megatron 只允许偏置配 sigmoid 或 sqrt(softplus) 分数（`transformer_config.py#L3139-L3147`）。

```
# megatron/core/transformer/moe/moe_utils.py#L936-L939（偏置只参与选择）
elif expert_bias is not None:
    scores_for_routing = scores + expert_bias.float()
    _, top_indices = compute_topk(scores_for_routing, topk, num_groups, group_topk)
scores = torch.gather(scores, dim=1, index=top_indices)
```

```
# megatron/core/transformer/moe/router.py#L209-L228（int64 计数、fp32 偏置）
if self.enable_expert_bias:
    self.register_buffer(
        'local_tokens_per_expert',
        torch.zeros(
            self.config.num_moe_experts,
            # Keep token counts exact through accumulation and cross-rank reduction.
            # Float16Module leaves integer buffers in their original dtype.
            dtype=torch.int64,
            device=torch.cuda.current_device(),
        ),
        persistent=False,
    )
    self.register_buffer(
        'expert_bias',
        torch.zeros(
            self.config.num_moe_experts,
            dtype=torch.float32,
            device=torch.cuda.current_device(),
        ),
    )
```

```
# megatron/core/transformer/moe/moe_utils.py#L1253-L1263（符号更新，整数精确比较）
torch.distributed.all_reduce(tokens_per_expert, group=tp_dp_cp_group)
num_experts = tokens_per_expert.shape[-1]
total_tokens = tokens_per_expert.sum(dim=-1, keepdim=True)
# Compare each tokens_per_expert value with the row average without converting the integer
# counts to floating point: tokens_per_expert < total / num_experts iff
# tokens_per_expert * num_experts < total.
update_direction = torch.sign(total_tokens - tokens_per_expert * num_experts)
updated_expert_bias = (
    expert_bias + update_direction.to(dtype=expert_bias.dtype) * expert_bias_update_rate
)
```

当更新速率对你的专家数来说太粗，负载就会一直不匀——按「负载不均的代价」那节，这会让步变慢；Li 等人的扫描还看到超过 128 个专家后 LM 质量受损。两个地方能抓到它：每个 checkpoint 都会存 `expert_bias`，你可以跟踪每层偏置在相邻 checkpoint 之间散开多宽；另外 `--save-tokens-per-expert-interval` 写出的 token 计数能给出逐全局批次的 MaxVio（A.1 节）。

什么时候用它：当均衡器必须几乎不跟 LM loss 抢方向时。偏置完全不产生梯度，而 DeepSeek-V3 那个系数 α = 1e-4 的 sequence-wise 损失也只加了一点点。用 sigmoid 分数跑它，并按你的专家数调更新速率——超过 128 个专家时 1e-3 可能太粗。

偏置不管负载长什么样都只挪同样一小步，于是要追上一直在变的路由器可能得走很多步。与其一步步逼近正确的偏置，能不能直接从这个批次的分数把它算出来？

分位数均衡

分位数均衡保留偏置这件事，但是把它解出来而不是挪出来。对每个专家，它把这个 micro-batch 的 token 按「它们对该专家的分数」排序，然后读一个分位数：只有目标份额的 token 能越过的那个分数。偏置就从那个分位数定下来，而不是按固定步长挪。Jin 等人用一个 19B、768 个专家的 MoE 比较各种均衡器：分位数均衡比符号更新更快稳住负载，LM loss 差不多，把平均 MaxVio 砍掉一半——尽管最后 1k 步它的 MaxVio 略高一点。这个精度有代价：每个 micro-batch 里，每个专家都得把所有 token 按它的分数排一遍来求自己的分位数，也就是在「每个 token 一个 top-k」之外再加「每个专家一个 top-k」。下面把符号更新与分位数均衡并排跑在同一个批次上。

> **图 3.3-k 符号更新 vs 分位数均衡（原站为交互图，此处从略）**

那个玩具每一步都路由同一个批次，于是每个解出来的偏置作用的就是它自己解出来的那批 token：走到最后一步，符号更新仍然留着不均衡，而分位数均衡几步之内就接近均匀了。在 Megatron 里，每个 micro-batch 解自己的分位数，这些解在全局批次上取平均，用来路由下一个批次，可选再用指数滑动平均（EMA）平滑（`--moe-router-quantile-balancing-ema`）。EMA 会保留大部分旧偏置，只混进每个新解的一部分，于是偏置逐步缓慢变化。

```
# megatron/core/transformer/moe/moe_utils.py#L233-L247（top-k 与逐专家分位数）
topk_result = (scores - beta).topk(k + 1, dim=1)
indices = topk_result.indices[:, :-1]

if not update_beta:
    return indices, beta

assert (num_tokens * k) % num_experts == 0, (
    "Quantile balancing requires the number of routed assignments "
    f"({num_tokens} tokens * top-{k}) to be divisible by "
    f"{num_experts} experts."
)
col_target = num_tokens * k // num_experts
alpha = topk_result.values[:, -1:]
beta_local = (scores - alpha).topk(col_target + 1, dim=0).values[-1].contiguous()
return indices, beta_local
```

```
# megatron/core/distributed/finalize_model_grads.py#L441-L457（平均、EMA、再居中）
stacked_beta = torch.stack(qb_beta_list, dim=0)
local_avg_list = [
    accum / count.clamp(min=1).to(accum.dtype)
    for accum, count in zip(qb_beta_accum_list, qb_beta_count_list)
]
stacked_local_avg = torch.stack(local_avg_list, dim=0)

torch.distributed.all_reduce(
    stacked_local_avg, op=torch.distributed.ReduceOp.AVG, group=dp_cp_group
)

ema = config.moe_router_quantile_balancing_ema
stacked_new_beta = ema * stacked_beta + (1.0 - ema) * stacked_local_avg
stacked_new_beta = stacked_new_beta - stacked_new_beta.mean(dim=-1, keepdim=True)
```

Kimi K3 在整个全局批次上跑分位数均衡，每个专家的分位数从「跨 GPU 求和后的直方图」里读。它之所以放弃固定步长规则，是因为每层的路由池到了 896 个专家之后均衡越来越难；而提出分位数均衡的 Su Jianlin 写道，Kimi K2 当时用的还是符号更新。

取平均有代价：一个由整个全局批次共享的偏置，能把那个批次均衡掉，而每块 GPU 的 micro-batch 仍然不匀。Neitemeier 等人在一个 7.5B 实验（一个 seed）里量了这个差距有多大。Megatron 用的「按 rank 平均的分位数」把全局批次均衡到 MaxVio 0.92，而在最差的那层里，某个专家并行 micro-batch 上最忙的专家仍然拿到了公平份额的约 7 倍，也就是局部 MaxVio 5.96。直接解出精确的全局批次分位数的做法把两个都降下来了。他们把局部 MaxVio 当作调度成本的代理指标，而不是实测吞吐。

这些均衡器在吞吐上怎么比？我们从初始化开始、同一模型、一个 seed，跑五种路由配置，比它们最后 20 个（共 120 个）迭代的中位吞吐。分位数均衡到 139 TFLOP/s，aux loss 是 125，被测的那套「只有偏置」的配置是 99。只有偏置那套用 sigmoid 分数、更新速率 1e-3，其余四种用 softmax。（这些运行用的是较老的 Megatron 提交，它把偏置的 token 计数累加在 bf16 里，计数可能被圆整，从而翻转偏置更新的方向；在 `d6316eb45` 里计数是精确的。）把 EMA 设成 0.9，分位数均衡到 121 TFLOP/s。那个设置让每个新解只占偏置更新权重的 10%（`transformer_config.py#L910-L913`）。

Jin 等人给分位数均衡加 EMA：平滑越强，偏置在 checkpoint 之间越稳，但训练早期纠正负载也越慢。五次运行都画在下面。

> 图 3.3-l 五种路由配置的中位吞吐（120 个迭代，取最后 20 个）（原站为交互图，此处从略）

图里五次运行中，只有偏置那套是平的，其余四条都在往上爬，最后 20 个迭代里分位数均衡最快。结束时吞吐还在涨，所以这个排序只描述了最初这 120 个迭代。分位数均衡同时还以五次里最高的训练 LM loss 收尾，所以光凭吞吐领先，并不能说明它能更快地训到同样质量。

要试分位数均衡，设 `--moe-router-load-balancing-type quantile_balancing`。同一个开关还接受 `sinkhorn`：Sinkhorn 在训练时用一种均衡指派的过程为每个 token 选专家。

风险是「全局批次看起来均衡、而每块 GPU 的 micro-batch 不均衡」，Neitemeier 等人把它读作更高的调度成本。要看这个差距，就用 `--save-tokens-per-expert-interval` 写出的 token 计数把 MaxVio 算两遍：一遍逐 micro-batch、一遍逐全局批次，然后对比（A.1 节）。

什么时候用它：当符号更新式的偏置均衡得太慢时。在 768 个专家下，解出偏置比一步步逼近得到更低的平均 MaxVio，LM loss 差不多，而且在五次配置对比里拿到最高中位吞吐（一个 seed，最后 20/120 个迭代）。同时要把它的限制算进去：Megatron 目前不接受它和任何 aux loss、`--moe-router-fusion` 或分组受限路由同时使用。

该盯什么

部位	症状	该盯的信号
负载不均的代价	「慢」：装着最忙专家的那块 GPU 决定了步长	相对强制均摊对照跑的 TFLOP/s
辅助损失	「学得差」：系数太高模型学得更差，太低负载又不匀	LM loss 与 MaxVio 一起看：路由概率被压平时，aux loss 看起来均衡而负载其实还歪着
范围	「学得差」：按序列或 micro-batch 均衡妨碍领域专家形成。「慢」：按全局批次均衡让每块 GPU 负载不匀，步长了 5.8%，主要来自不均衡、其中约 1% 来自通信	在你均衡的那个范围上读 MaxVio：逐 micro-batch 与逐全局批次用 token 计数，逐序列用路由追踪（A.1 节）
无损失偏置	「慢」：固定步长 1e-3 在 128 个专家时最忙专家仍拿到约 20 倍平均负载。「学得差」：超过 128 个专家该步长太粗	`expert_bias` 每层在相邻 checkpoint 之间散开多宽，以及逐全局批次的 MaxVio（token 计数，A.1 节）
分位数	「慢」：用滑动平均平滑偏置会让它滞后于路由器分数。偏置由全局批次共享，每块 GPU 的负载仍可能不匀（见「范围」）	同时看逐 micro-batch 与逐全局批次的 MaxVio（用保存的 token 计数，A.1 节），以发现「全局均衡掩住局部不匀」

导航：[[2026-11-moe-handbook|全书目录]] · ← 第 3 章（上）监控与路由 · 第 3 章（中）专家 →

第 3 章 解剖 MoE 训练（下）：专家

> 说明：原站里的图都是交互式组件，这里无法复现，一律保留图注并标注「（原站为交互图，此处从略）」。原站大部分图不给编号，本文按节内顺序编号（如 图 3.4-a）。

上一节里每个均衡器都在「训练开始前就定好的一组专家」上摊 token，而我们还没问过这组专家该长什么样：一层该有多少个专家、每个多宽、有没有哪个专家该看见每一个 token？

3.4 专家：多少、多大、共不共享

一旦路由器为某个 token 挑好了专家，这些专家长什么样，同时决定了这一层能学到什么、以及跑起来花多少。你要选：一层有多少专家、每个多宽；怎么把「很多小专家」带来的通信量压住；要不要有一个专家看见每个 token；以及每个专家内部用哪个激活函数。

前沿开源 MoE 的专家选择，外加 Nemotron（用 LatentMoE 与 ReLU^2）和 LongCat-Flash（可变算力）。「数量与宽度」给出每 token 用掉全部专家中的几个、以及每个专家的 FFN 宽度；共享专家单列；「潜空间维度」给出投影前与投影后的 token 宽度。「未说明上限」指的是公开的配置。

模型	数量与宽度	潜空间 MoE	共享专家	可变算力	激活函数
DeepSeek-V4.1-Flash	384 里选 6，宽 2,304	全宽专家	1 个，宽 2,304	固定 top-k，等宽	SwiGLU，限 10
GLM-5.3	256 里选 8，宽 2,048	全宽专家	1 个，宽 2,048	固定 top-k，等宽	SiLU 门；未说明上限
Kimi K3	896 里选 16，宽 3,072	7,168 → 3,584；上投影前做归一化	2 个，各宽 3,072	固定 top-k，等宽	SiTU-GLU，上限 4 与 25
LongCat-Flash	768 里选 12，宽 2,048	全宽专家	无	768 个里有 256 个是恒等专家；每 token 激活 18.6B 到 31.3B 参数，平均 27B	SiLU 门；未说明上限
MiMo-V2.6-Pro-RL	384 里选 8，宽 2,048	全宽专家	无	固定 top-k，等宽	SiLU 门；未说明上限
Nemotron 3 Ultra	512 里选 22，宽 5,120	8,192 → 2,048	1 个，宽 10,240	固定 top-k，等宽	ReLU^2
Qwen3.8 2.4T-A95B	512 里选 10，宽 2,048	全宽专家	1 个，宽 2,048	固定 top-k，等宽	SiLU 门；未说明上限
Step 5 Preview	352 里选 8，宽 1,536	全宽专家	1 个，宽 1,536	固定 top-k，等宽	SwiGLU，限 7

在参数预算固定的情况下，加专家就意味着把每个专家做窄。同样算力下，哪种切法能给你更好的模型？

数量与宽度

设想一层 MoE，参数与算力预算固定，然后把它切细：专家数翻倍、每个宽度减半、top-k 也翻倍，两个预算都不变。图 3.4-a 画的就是这条阶梯：粗粒度，然后是 16 个 d/2 宽、top-4，再是 32 个 d/4 宽、top-8。Liew 等人在一个 18 层、总参数与激活参数都固定的模型上做了这个消融，损失随着每一级变细而下降，一直到 128 个 d/4 宽、top-8，从 2.334 降到 2.297；再细下去就没有进一步提升了。所以在这个实验里，「切细」到 128 个专家之后就不再划算，而每一级变细仍然意味着更小的 GEMM、以及每个 token 更多的副本要调度。

> 图 3.4-a 切细的阶梯：粗粒度 → 16 × d/2（top-4）→ 32 × d/4（top-8）；总参数与每 token 算力不变（原站为交互图，此处从略）

已发布的模型停在这条阶梯的不同位置：gpt-oss-120b 与 Trinity Large 让每个专家和模型一样宽，分别 128 选 4、256 选 4；Mistral Small 4 把这个宽度减半，128 选 4；Qwen3.5-35B-A3B 再减到

四分之一，256 选 8。

图里可以看到，切专家改变的是「个数与宽度」，而总参数和每 token 算力保持不变。所有方块的总面积代表参数，深色面积代表一个 token 的专家计算。你改的是粒度（granularity：切得多细。越细＝更多更窄的专家；为了保持每 token 算力不变，每个 token 就用更多个窄专家）。你也可以提高稀疏度：加更多参数，但每个 token 用到的数量不变。多出来的参数有两种花法，下面比较：加更多路由专家，或者加「按 token id 查表」的 n-gram embedding 表。

> 图 3.4-b LongCat 在 280M 激活参数下：多花在专家上 vs 花在 n-gram 表上（原站为交互图，此处从略）

那张图画的是 LongCat 在 280M 激活参数下的比较。在那里，当专家已经持有约 12 倍激活参数时，n-gram 表比把同样参数花在更多专家上更能降 loss（在 17 倍激活参数时约好 0.01）；而如果在专家只有约 4 倍激活参数时就加表，那是从一开始就亏。这个收益有上限：在英文验证损失上，一旦表占到参数的一小半以上（略超 40%），「更多专家」又赢回来，LongCat 从那个交点读出的规则是：表里的参数不超过 50%。在那个比较里，先扩容专家是划算的，等专家足够大了，接下来的一部分参数放到查表里更有效。所以「多余参数往哪花」取决于你已经有都少专家容量。

两种花法在已发布模型里都有：Kimi K2 把池子扩到 384 个专家、top-8；LongCat-Flash-Lite 则把 68.5B 参数里的 31.4B 放进了 n-gram embedding 表。

不管是加更多专家，还是把现有容量切成更多窄专家，最终形状也会影响训练速度：更窄的专家跑更小的 GEMM，GPU 利用率更低。

对稠密模型来说，算力最优的缩放律能告诉你怎么在模型大小和训练 token 数之间分配固定的 FLOP 预算：模型越大，能喂的 token 越少；模型越小，能训的 token 越多。MoE 多了一个选择：你可以增加它的总参数而不增加每个 token 用到的参数，于是你还得选稀疏度。而无论哪种模型，在时间限制下做选择，都还要算上「每种配置对硬件的利用效率」。

Sarkar 等人把一个拟合出来的损失缩放律和一个训练吞吐模型结合起来，比较了「固定模型 FLOP 下损失最小」的稀疏度与「固定集群时间下损失最小」的稀疏度。他们对某个集群与训练窗口的预测画在下面。（这里的稀疏度指「token 用不到的参数占全部参数的比例」。）

> 图 3.4-c 按模型 FLOP 选 vs 按集群时间选：预测损失与稀疏度（原站为交互图，此处从略）

在他们的预测里，只按模型 FLOP 选会把你推向网格里最稀疏的那个模型。一旦把集群上的训练速度算进去，预测损失最低的点就移到一个不那么稀疏的配置，比最稀疏那个低 0.031 nats。和稠密缩放一样，你是在模型容量与「能训多少 token」之间权衡。对 MoE 而言，提高稀疏度可以在不增加每 token 模型 FLOP 的同时降低硬件利用率，于是多出来的容量仍然要花训练时间。

Meagatron 用三个开关定形状：`--num-experts` 定数量，`--moe-ffn-hidden-size` 定每个专家的宽度，`--moe-router-topk` 定每个 token 挑几个专家。

```
# megatron/core/transformer/moe/experts.py#L262-L293（fc1/fc2 的分组形状）
# Double the output width with gated linear unit, see https://arxiv.org/pdf/2002.05202.pdf
ffn_hidden_size = not_none(self.config.moe_ffn_hidden_size)
if self.config.gated_linear_unit:
    ffn_hidden_size *= 2

self.linear_fc1 = submodules.linear_fc1(
    self.num_local_experts,
    self.input_size if self.config.moe_latent_size is None else self.config.moe_latent_size,
    ffn_hidden_size,
    ...
)
```

```
# megatron/core/transformer/moe/router.py#L77-L81（路由器权重：专家数 × 隐藏维）
self.weight = torch.nn.Parameter(
    torch.empty((self.config.num_moe_experts, self.config.hidden_size), dtype=torch.float32)
)
```

切得太细，每个专家的 GEMM 小到会拉低每块 GPU 的 TFLOP/s；粒度选错，模型还学得更差。想知道某个形状要付多少代价，就从 profiler 里读每层 MoE 的分组 GEMM 时间，配 `--log-throughput` 给出的每 GPU TFLOP/s 一起看。`--save-tokens-per-expert-interval` 写出的 token 计数还能暴露「收不到任何 token」的死专家（A.1 节），但一个专家也可以一边收着 token 一边死掉——那就要看它的输出范数（3.6 节）。

什么时候用它：在总参数与激活参数不变的前提下用更细的专家，但有个头。在那个 18 层模型里，损失一直降到 128 个 d/4、top-8，再细就没有收益了。等小 GEMM 和 token 副本开始主导你的步时长，就别再往细里走了。

更细的专家只在某一点之前对损失有帮助，而每一级变细都会让每个 token 的更多副本上网络。能不能保留很多小专家，同时让每份副本少送点数据？

潜空间 MoE（LatentMoE）

每份 token 副本都带着它的完整隐状态——d 个数，每挑一个专家就送一次。LatentMoE 把这份副本缩小：在调度前把每个 token 投影到一个更小的潜空间 ℓ，让专家在 ℓ 上跑，合并之后再投影回去。每份副本从 d 缩到 ℓ，省下来的通信就够养更多专家：在专家参数、专家计算和通信量都不变的前提下，这一层可以负担 $\alpha = d/\ell$ 倍多的专家、以及 α 倍大的 top-k。下面把一个标准层和它的潜空间版本并排画，专家内部的参数与计算保持不变。

> **图 3.4-d 标准 MoE 层 vs LatentMoE 层（专家参数与计算固定）（原站为交互图，此处从略）**

画成方块的话：标准层从 16 个里选 4 个，每个收到全宽的 token 向量；潜空间层从 32 个里选 8 个，但每个只收到一半宽度的数值。两者每 token 送出的数据量相同；它们的总面积与深色面积也相等，说明专家参数与计算没变——两次投影加的是这些面积之外的工作。

这个交换在质量上划算吗？在英伟达那个 73B-A8B 的消融里，总参数与激活参数相同的条件下，LatentMoE 在 MMLU-Pro 上比标准 MoE 高 4.6 分，一个小规模的复现也降低了困惑度。Nemotron 3 Super、Nemotron 3 Ultra 和 Kimi K3 都上了它。代价是每个 token 要多过两次 $d \times \ell$ 投影，服务端吞吐从并发 1 时的低 12% 到某些并发下略高。在 Megatron 里，潜空间宽度是 `--moe-latent-size`。

```
# megatron/core/transformer/moe/moe_layer.py#L304-L335（两个投影 + 可选归一化）
self.fc1_latent_proj = linear_cls(
    self.config.hidden_size,
    self.config.moe_latent_size,
    parallel_mode="duplicated",
    ...
)
if self.config.moe_use_norm_before_up_proj:
    self.fc2_norm = TENorm(
        config=self.config,
        hidden_size=self.config.moe_latent_size,
        eps=self.config.layernorm_epsilon,
    )
self.fc2_latent_proj = linear_cls(
    self.config.moe_latent_size,
    self.config.hidden_size,
    parallel_mode="duplicated",
    ...
)
```

```
# megatron/core/transformer/moe/moe_layer.py#L554-L556（调度前降维）
# Project the hidden_states from hidden dimension down to latent dimension.
if self.config.moe_latent_size:
    hidden_states, _ = self.fc1_latent_proj(hidden_states)
```

```
# megatron/core/transformer/moe/moe_layer.py#L662-L666（合并后归一化并升维）
output = self.token_dispatcher.combine_postprocess(output)
if self.config.moe_latent_size:
    if self.config.moe_use_norm_before_up_proj:
        output = apply_module(self.fc2_norm)(output)
    output, _ = self.fc2_latent_proj(output)
```

潜空间 MoE 会从两头出问题：跑得慢，或者激活值炸掉。跑得慢的时候，别只盯通信——All-to-All 现在每副本只传 ℓ 个数而不是 d——要去看那两次额外投影和更小的专家 GEMM。LatentMoE 的英伟达作者提的补法是「小 GEMM 的专用 kernel」以及「给共享专家单开一条流」。炸掉的时候（Kimi K3 报告它那条路由分支的激活在 2.8 万亿参数规模上炸），一个该看的地方是上投影 `fc2_latent_proj` 的输出范数，可以从 `--save-activations-interval` 的 dump 里离线逐层算（A.1 节）。K3 的修法是在上投影前加一个 RMSNorm（`--moe-use-norm-before-up-proj`），它发布时把路由缩放系数设成 1.0。

什么时候用它：当你想要更多专家、更大的 top-k，但不想让每 token 的通信量变大时。把每份副本缩到 $\ell = d/\alpha$ 就能负担 α 倍的专家与 top-k，等规模下分数更高，代价是两次投影。

到此为止每个专家都躲在路由器后面，只看见发给它的 token。如果把一个专家挪到路由器外面、让它看见每个 token，能换来什么？值它那份算力吗？

共享专家

共享专家待在路由器外面：它处理每个 token，不管这个 token 路由到了哪些专家。DeepSeekMoE 提出共享专家，让它学共同模式，从而减少路由专家之间重复的知识、给它们专业化腾出空间。在等算力的消融里，加一个共享专家在多数基准上有提升。Qwen 家的模型说明了这个问题有多开放：Qwen3 报告里的 MoE 模型把它去掉了，Qwen3-Next 又在 sigmoid 门后面把它请回来，Qwen3.5 则保留。

用「等算力、从头训」的模型损失来衡量，收益小且不一致。Pangu Ultra MoE 和 Sun 等人的专家阈值模型里，共享专家把损失降了约 0.01 到 0.02；而在 OLMoE 和 Li 等人的小 MoE 扫描里，它要么没作用、要么略差。

共享也会减少路由器的自由度：32 个专家、激活 4 个时，如果给共享专家占掉一个位置，剩下选 3 个只有 4,495 种组合，而从 32 个里选 4 个有 35,960 种。如果你加共享专家却不减少路由 top-k，这个常开的 FFN 就增加了每 token 算力：Mellum2 那样做了，结果质量没提升、推理还更慢。

真要加，还得决定它的输出怎么和路由专家混。下图在等算力下并排画了五种设计：只有路由专家、固定权重的共享专家（DeepSeek-V3）、带门的共享专家（Qwen3-Next）、与路由专家一起归一化的共享专家（Inkling）、以及一条并行的稠密 FFN（Grok 2）。

> 图 3.4-e 五种共享专家设计（等算力）（原站为交互图，此处从略）

这样并排看：固定权重的共享专家在每个 token 里占的份额都一样；Qwen 的门让这个份额在 0 到 50% 之间移动；Inkling 的路由器把共享专家和路由专家放在一起打分，于是逐 token 学出份额；而一条不做重归一化的并行稠密 FFN，不管路由器打多少分都握着超过一半的份额。

Megatron 的开关是 `--moe-shared-expert-intermediate-size`、`--moe-shared-expert-gate`（给它输出加一个 sigmoid 门）和 `--use-grouped-gemm-for-shared-expert`。不加门时，Megatron 以权重 1 加上共享输出（`moe_layer.py#L668-L669`），而路由权重在选中的专家上归一化后加起来等于路由缩放系数（`moe_utils.py#L942-L947`，见 3.1 节）。（Megatron 用 sigmoid 路由，或者在选出 top-k 之后再做 softmax 来实现这件事；后者要把 `--moe-router-pre-softmax` 关掉。）于是共享专家占每个 token 输出权重的 1/(1 + factor)，系数取 2.5 时就是 29%；系数越大它占比越小。GLM-4.5-Air 和 Mistral Large 3 把系数设成 1，于是它拿一半。

```
# megatron/core/transformer/moe/shared_experts.py#L201-L208（给输出加 sigmoid 门）
def forward(self, hidden_states: torch.Tensor) -> torch.Tensor:
    output, _ = super().forward(hidden_states)
    if self.use_shared_expert_gate:
        logits = torch.nn.functional.linear(hidden_states, self.gate_weight)
        gate_score = torch.nn.functional.sigmoid(logits)
        output = output * gate_score
    return output
```

```
# megatron/core/transformer/moe/moe_layer.py#L668-L669（以权重 1 相加）
if shared_expert_output is not None:
    output = output + shared_expert_output
```

想跟踪「训练中共享专家对每个 token 输出的贡献有多大」，就把它过门之后的输出，和路由分支过合并（以及可能的潜空间升维）之后的输出作比，用 A.1 里那个稀疏前向 hook。

共享专家是前沿 MoE 常见的起步选择，GLM-5.3、Kimi K3、DeepSeek V4.1 Flash 都有；MiMo-V2.6 没有。想在等算力下比「有」和「没有」，就给共享专家和路由专家一样的宽度，并用它替换掉一个路由位置。

不管共不共享，到目前为止每个专家都一样宽、每个 token 都拿到一样的 top-k，于是每个 token 的算力都一样。能不能让路由器给某些 token 少一点算力、给另一些多一点，而平均值保持不变？

可变算力

有两种设计能让每 token 算力变化而均值固定。LongCat-Flash 加了零计算专家：把输入原样传过去。它的路由器从「512 个 FFN 专家 + 256 条恒等路径」里为每个 token 选 12 个。一个偏置调节 FFN 专家被选中的频率，目标是每 token 平均 8 个，同时允许个别 token 用得多或少；预训练期间这个数量的标准差升到约 2.5。小米 AI Lab 在 MoDSE（Mixture of Diverse Size Experts）里换了个思路：把专家宽度混起来，让路由器为每个 token 挑大的或小的专家。下图模拟了同宽专家（如 Mixtral）以及这两种设计下每 token 的激活宽度。

> 图 3.4-f 每 token 的激活宽度：同宽专家 / 零计算专家（LongCat）/ 混合宽度（MoDSE）（原站为交互图，此处从略）

模拟里，每种设计用一个偏置把平均激活宽度按在同宽专家的水平上，于是有的 token 拿到更多、有的更少；而每个 token 仍然填满它所有的 top-k 位置，变的只是这些位置里落进什么。在 LongCat-Flash 的消融里，零计算专家在等算力下拿到更低的损失。混合专家宽度可能提升质量，但可能伤吞吐：小米 AI Lab 和 Wang 等人各自在小模型上用一个路由器管不同尺寸的专家并报告了收益；Li 等人没看到收益，不过他们的设置里专家分成两个池、每个池固定 top-k，于是每 token 算力还是相同。GroveMoE 的代价出现在推理端：它的混合尺寸比 Qwen3-30B-A3B 慢约 30%，作者归因于每层要用两套 MoE kernel 分别处理两种尺寸。

Megatron source：没有。在 Megatron-LM main 的 `megatron/` 目录里 grep 不到零计算专家，也没有混合专家宽度；每个分组线性层只有一个 `moe_ffn_hidden_size`。

要跟踪每层每 token 算力的均值与离散度：零计算路由看「选中的 FFN 专家个数」，混合宽度路由看「选中专家宽度之和」。把均值与目标预算比，同时记录逐专家 token 数、每个专家并行 rank 的算力和步时长。

什么时候用它：从「一种宽度 + 固定 top-k」起步，就像 MiMo-V2.6-Pro-RL、GLM-5.3、Kimi K3 那样。这也正好匹配 Megatron 的分组线性层——它们只有一个 `moe_ffn_hidden_size`。

专家的数量、宽度、算力定了之后，每个专家内部还剩一个选择：两次投影之间那个函数。这个选择对 MoE 影响多大？

激活函数

和稠密 Transformer 一样，每个专家在输入与输出投影之间用一个激活函数，比如 DeepSeek-V3、Qwen3、Kimi K2 用的 SwiGLU，或者 ReLU²。在 MoE 里你还要盯「激活在不同专家之间怎么表现」：Step 3.5 Flash 发现有一两个专家的输出范数会飞快长大，而其他专家保持稳定、训练损失几乎没变。激活函数、以及加在它上面的截断，都有助于控制这种增长。下图画出 SwiGLU、ReLU² 和 Expert-Specific PolyNorm 的输出随输入尺度变大怎么长。

> 图 3.4-g SwiGLU / ReLU² / Expert-Specific PolyNorm 的输出随输入尺度增长（原站为交互图，此处从略）

放大之后，SwiGLU 和 ReLU² 的输出随输入尺度的平方增长，而 Expert-Specific PolyNorm 把它的门归一化了，于是只线性增长。一个截断就能把这种增长限住。两项研究在他们测试的设置里发现「截」与「不截」质量相近：蚂蚁 Ling 团队一个 7.9B MoE 的消融里，带截断的 SwiGLU 在 17 个基准上的均值与朴素 SwiGLU 打平；Step 3.5 Flash 的激活截断把离群专家的范数压住了，损失看不出差别（尽管那条分支是从未截断的那次运行中途接过来的）。截断并没有消掉 Ling 团队那次 FP8 运行里的损失尖峰：带截断的 SwiGLU 把尖峰推迟了，但到第 77,000 步还是来了一个。下一张图把输入尺度扫过「朴素 SwiGLU / 硬截断 / 同一边界的 SiTU-GLU 平滑上限」，看它们各自怎么弯。

> 图 3.4-h 朴素 SwiGLU / 硬截断 / SiTU-GLU 平滑上限在扫输入尺度时的表现（原站为交互图，此处从略）

这次扫描里，朴素 SwiGLU 最大的隐藏单元随门输入的平方一路长到 6,862——那是 K-EXAONE 2.0 在最后一层、推理时关掉截断量到的峰值；硬截断跟着它长到边界就停住；而 SiTU-GLU 在远没到边界之前就弯开了。扫描把两者的「单个隐藏单元停在约 100」设为共同目标：硬截断在 10，SiTU-GLU 的门上限 4、上投影上限 25，从差不多的输入开始，两者都几乎不再给这些单元的门权重传梯度。整个换掉激活函数的证据更薄：Zhuo 等人的 PolyNorm 在 OLMoE-1B-7B 上把平均验证损失比 SwiGLU 降了 0.02，而 Motif 3 只把它的 Expert-Specific PolyNorm 与 SwiGLU 在有效秩上做了比较。Kimi K3 说 SiTU-GLU 的平滑上限在训练行为上比硬截断更好。

在 Megatron 里，`--activation-func-clamp-value` 给 MoE 层的 SwiGLU 设硬截断上限；取 10 就对应 DeepSeek V4.1 Flash 的上限，GigaChat 3.5 与 Hy4-preview 也用这个值。Kimi K3 的平滑上限 SiTU-GLU 是 `--activation-func-tanh-clamp-scale 4 --activation-func-tanh-clamp-scale-linear 25`，从 Megatron Core v0.20 起可用；Nemotron 3 的 ReLU² 是 `--squared-relu`。

```
# megatron/core/transformer/moe/experts.py#L1100-L1118（SiTU-GLU 或带截断的 SwiGLU）
if tanh_clamp_scale is not None:
    intermediate_parallel = situ_glu(
        intermediate_parallel,
        tanh_clamp_scale,
        self.config.activation_func_tanh_clamp_scale_linear,
        self.config.glu_linear_offset,
    )
else:
    def glu(x):
        x_glu, x_linear = torch.chunk(x, 2, dim=-1)
        if (val := self.config.activation_func_clamp_value) is not None:
            x_glu = x_glu.clamp(min=None, max=val)
            x_linear = x_linear.clamp(min=-val, max=val)
        return self.config.activation_func(x_glu) * (x_linear + self.config.glu_linear_offset)

    intermediate_parallel = glu(intermediate_parallel)
```

```
# megatron/core/activations.py#L17-L39（软截断与 SiTU-GLU）
def tanh_soft_clamp(x: torch.Tensor, scale: float) -> torch.Tensor:
    """Tanh Soft Clamp to precondition activation inputs."""
    return (scale * torch.tanh(x.float() / scale)).to(x.dtype)

@jit_fuser
def situ(x: torch.Tensor, scale: float) -> torch.Tensor:
    """Sigmoid-Tanh Unit from Kimi K3: ``s * tanh(x / s) * sigmoid(x)``"""
    return tanh_soft_clamp(x, scale) * torch.sigmoid(x)

@jit_fuser
def situ_glu(x, gate_scale, linear_scale=None, linear_offset=0.0):
    """SiTU-GLU: ``situ(x_gate, gate_scale) * (x_linear + linear_offset)``."""
    x_gate, x_linear = torch.chunk(x, 2, dim=-1)
    if linear_scale is not None:
        x_linear = tanh_soft_clamp(x_linear, linear_scale)
    return situ(x_gate, gate_scale) * (x_linear + linear_offset)
```

专家里的离群激活能把一次训练炸掉。要抓它，就离线从 `--save-activations-interval` 的 dump 里算 Step 3.5 Flash 那个信号：每层专家输出范数的「最大值 / 中位数」之比（A.1 节）。一个专家把别人甩开就表现为这个比值上升。每份 dump 只保留最后一个 micro-batch 并拷到 CPU，所以采样要稀。

什么时候用它：从你参考模型的激活方案起步。前沿 MoE 用不同方式限住激活——Step 5 Preview 的 SwiGLU 截在 7、DeepSeek V4.1 Flash 截在 10、Kimi K3 用 SiTU-GLU 的平滑上限。做新架构时，SwiGLU 是合理的基线；如果专家范数发散了，就试试截断，并且既要看范数也要看训练损失再决定上限。

该盯什么

部位	症状	该盯的信号
数量与宽度	「慢」：专家 GEMM 太小。某些专家可能拿不到 token	profiler 里每层 MoE 的分组 GEMM 时间、每 GPU TFLOP/s（`--log-throughput`）、逐专家 token 数（`--save-tokens-per-expert-interval`，A.1 节）
潜空间 MoE	「慢」：两次额外投影与更小的专家 GEMM 增加时间。「炸了」：Kimi K3 报告路由分支激活爆炸	速度上，剖析两次潜空间投影与更小的专家 GEMM，配合被缩小的 All-to-All。稳定性上，一处该看的是上投影输出范数（`fc2_latent_proj`），从激活 dump 里算（A.1 节）
共享专家	推理时「慢」：它增加算力，Mellum2 就发现推理更慢	共享分支过门后的输出 vs 路由分支合并（含潜空间升维）后的输出，用 A.1 节的稀疏前向 hook
可变算力	「慢」：每种专家尺寸跑各自的 kernel，GroveMoE 报告推理比 Qwen3-30B-A3B 慢约 30%	每层「选中 FFN 个数」或「专家宽度之和」的均值与离散度 vs 目标算力预算；同时记录逐专家 token 数、每 EP rank 算力、步时长
激活函数	「炸了」：离群激活	每层专家输出范数的最大值/中位数之比（Step 3.5 Flash 的信号），从激活 dump 里算（A.1 节）

本节所有东西都发生在一层 MoE 内部，而模型要叠很多层，这就带出两个还没问的问题：是不是每一层都该是 MoE 层？每层外面那个 block 该怎么接？

导航：[[2026-11-moe-handbook|全书目录]] · ← 第 3 章（中）负载均衡 · 第 3 章（下）块布局与训练 →

第 3 章 解剖 MoE 训练：块布局与训练

> 说明：原站里的图都是交互式组件，这里无法复现，一律保留图注并标注「（原站为交互图，此处从略）」。原站大部分图不给编号，本文按节内顺序编号（如 图 3.5-a）。

3.5 块布局

把模型想象成一摞 block，每个 block 是注意力后面接一个稠密 MLP 或一个 MoE 层。每个 MoE 层都要用一次 All-to-All 把 token 发出去、再收回来，所以 MoE 层放在哪，决定了每一步要做多少次 All-to-All，也决定了哪些层的均衡最难做。而每个 block 内部怎么接线，又决定了每次 All-to-All 能有多少藏在注意力后面。

前沿模型把 MoE 放在哪、以及怎么把它接到注意力上。稠密层数指主干，不含预测模块。

模型	稠密层	块接线
DeepSeek-V3	前 3 层稠密，其余 58 层 MoE	注意力后接 FFN
DeepSeek-V4.1-Flash	全部 40 个 block 都是 MoE	单遍 mHC 残差连接
GLM-5.3	前 3 层稠密，其余 75 层 MoE	注意力后接 FFN
Kimi K3	第 1 层稠密，其余 92 层 MoE	注意力后接 Stable LatentMoE；注意力残差
Llama 4 Maverick	稠密与 MoE 交替，从稠密开始	每两层一个 MoE
LongCat-Flash	稠密 FFN 与 MoE 在每个 macro-block 内共存	捷径连接的 MoE 与后续的注意力、FFN 工作重叠
MiMo-V2.6-Pro	第 1 层稠密，其余 69 层 MoE	混合注意力块后接 FFN

第一个布局决定是：哪些层真的有专家。你可以每层都放 MoE，可以留几层稠密，也可以让两种交替。该选哪个？

稠密层

稠密层没有路由器也没有专家，所以不产生 All-to-All。图 3.5-a 把三种布局的 All-to-All 次数并排数出来：每层都是 MoE（如 Qwen3、gpt-oss、MiniMax M2、Llama 4 Scout）、开头放几层稠密、每两层一个 MoE。开头稠密只是把最早的几个 MoE 层换掉，DeepSeek-V3 就保留前 3 层稠密；而每两层放一个 MoE 能把 All-to-All 减半，Llama 4 Maverick 就是稠密与 MoE 交替。两种做法都拿一部分 MoE 容量，换更少的 All-to-All 和更少的参数。

> 图 3.5-a 三种布局的 All-to-All 次数：全 MoE / 开头稠密 / 隔层 MoE（原站为交互图，此处从略）

图里那个玩具模型的「路由随深度变化越来越不跟 token id」，所以第一层最慢：开头放稠密能去掉它最慢的两层，却只砍掉 16 次 All-to-All 里的 4 次；而隔层 MoE 把 All-to-All 减半，却保留了第二慢的那层。在 Ling 的 60 层模型里，开头稠密的代价很小：把前 1 到 3 层改成稠密，在等 FLOP 下效率几乎没有变化，效率杠杆接近 1。

另一个理由是均衡，但证据更薄。DeepSeekMoE 观察到第一层的均衡收敛更慢，Ling 也说开头稠密能缓解浅层不均衡，不过它那组消融只报了 loss。后来的报告彼此矛盾。有些干脆让浅层不碰学出来的路由器：图像生成的 MoE Nucleus-Image 在没有 3 层开头稠密时会发散，而 DeepSeek-V4 对前 3 个 MoE 层做了哈希路由。

另一些仍然在浅层路由，转而治不均衡。Solar Open 每层都是 MoE，用序列级辅助损失修掉浅层不均衡。Sigma-MoE-Tiny 让每个 token 只从 96 个专家里选 1 个，只靠辅助损失时它的下层塌了：损失走了捷径，把路由器概率压成均匀分布，而有些专家仍然几乎拿不到 token。它的修法是训练早期在前 8 层激活更多专家，之后再退回 1 个。稠密层也不总能解决问题：Motif 3 前两层稠密，可它最早的几个 MoE 层在训练早期仍然最不均衡；而 ERNIE 5.0 在文本任务上没发现第一层有严重不均衡。

Megatron 用列表形式在 `--moe-layer-freq` 里指定布局，每层一个值，0 表示稠密、1 表示 MoE：`"([0]+[1]15)"` *是开头一层稠密*，`"([0,1]8)"` 是 Maverick 那样稠密在前的交替。这个开关也接受整数，而整数形式会把 MoE 放在第 0 层，所以 `--moe-layer-freq 2` 是从 MoE 开始，而不是像 Maverick 那样从稠密开始。稠密层的宽度来自 `--ffn-hidden-size`，与专家宽度分开。

```
# megatron/core/models/gpt/gpt_layer_specs.py#L635-L645（整数或列表模式）
if isinstance(config.moe_layer_freq, int):
    moe_layer_pattern = [
        1 if (i % config.moe_layer_freq == 0) else 0 for i in range(config.num_layers)
    ]
elif isinstance(config.moe_layer_freq, list):
    moe_layer_pattern = config.moe_layer_freq
    assert len(moe_layer_pattern) == config.num_layers, (
        f"Invalid length of moe_layer_pattern: {len(moe_layer_pattern)}, "
        f"expected {config.num_layers}, "
        f"current moe layer pattern: {config.moe_layer_freq}"
    )
```

```
# megatron/core/models/gpt/gpt_layer_specs.py#L652-L659（逐层选稠密或 MoE）
layer_specs = []
for layer_number in range(config.num_layers):
    if moe_layer_pattern[layer_number] == 1:
        layer_specs.append(moe_layer_spec)
    elif moe_layer_pattern[layer_number] == 0:
        layer_specs.append(dense_layer_spec)
    else:
        raise ValueError(f"Invalid layer pattern: {moe_layer_pattern}")
```

上面那些报告都在浅层看到了不均衡；即便那里放的是稠密层，紧跟着的第一个 MoE 层仍然可能最不均衡，Motif 3 就是这样。这些「第一个 MoE 层」才是要盯的对象。用 `--moe-per-layer-logging` 追它们的辅助损失（2.4 的启动器会把逐层数值发到 Trackio），用路由 trace 追它们的 MaxVio（A.1）。稠密层没有路由器可看。z-loss 量的是路由器 logits 长得有多大，不是 token 摊得多均匀。

想在几乎不付代价的前提下减少 All-to-All，就用开头稠密：在 Ling 的 60 层模型里，1 到 3 层开头稠密在等 FLOP 下几乎没改变效率。但别指望它治浅层不均衡：DeepSeekMoE 和 Ling 都没给出这方面的数据，而 Motif 3 在前面放了两层稠密之后，第一个 MoE 层照样不均衡。

你保留的每个 MoE 层仍然要把 token 发出去再收回来；而在一个 MoE block 里（MoE 跑在注意力的输出上），GPU 有相当一部分时间在等。能不能把 block 重新接线，让一部分通信在注意力计算时同时进行？

块接线

在一个 MoE block 里，每次 All-to-All 两侧都绑着计算。路由器要读注意力的输出，所以注意力没算完就发不出去；下一层要读 MoE 的输出，所以 combine 没做完后面什么都开不了。Ultra-Scale Playbook 讲了怎么把通信与计算重叠的一般做法。

重接 block 能去掉其中一条依赖、或者两条都去掉，省出来的通信就能和计算重叠。并行 block 把同一个输入同时喂给注意力和 MoE，Cohere 的 North Mini Code 与 Moondream 3 就是这么做的，于是 dispatch 在注意力算的时候就走。combine 更难藏，因为它只能和同层剩下的工作重叠——在 Megatron 的捷径 block 里就是注意力的输出投影和共享专家；两样都没有时，它就是裸露的。捷径连接的 MoE 把 combine 也藏起来了，藏在它旁边跑的稠密计算后面，LongCat-Flash 和 Pos-2（ScMoE 的首选变体、Cai 等人提出的捷径连接 MoE）都是这样。图 3.5-b 在一个 micro-batch 里比较了三种接线，按它示意的时长，这一层里裸露的 All-to-All 占比从 MoE block 的 33% 降到捷径的 7%。

> 图 3.5-b 一个 micro-batch 内的三种块接线：MoE block / 并行 block / 捷径连接 MoE（原站为交互图，此处从略）

```
# megatron/core/models/hybrid/shortcut_block.py#L102-L127（注意力与下一个 MoE 配对）
compute_layer = layers[physical_index]
paired_type = layer_type_list[physical_index]
if paired_type not in SUPPORTED_SHORTCUT_PREDECESSORS:
    raise ValueError(
        "Shortcut MoE must be preceded by a Mamba, GDN, or supported attention layer"
    )

supports_two_stage = (
    isinstance(compute_layer, TwoStageAttentionLayer)
    and compute_layer.supports_two_stage_attention()
)
if not supports_two_stage:
    raise ValueError(
        f"Shortcut attention layer {paired_type!r} does not support two-stage attention"
    )
moe_layer = layers[physical_index + 1]
grouped_layers.append(
    ShortcutMoEBlock(
        compute_layer,
        moe_layer,
        overlap_a2a=config.moe_shortcut_parallel,
        attn_local_idx=physical_index,
        moe_local_idx=physical_index + 1,
    )
)
physical_index += 2
```

```
# megatron/core/models/hybrid/shortcut_block.py#L490-L502（路由器读这一对的输入）
# Launch the moe_router
with quant_context_factory(moe_config, self.moe_layer_idx):
    moe_hidden_states = self._read_shortcut_hidden(
        moe_hidden_states, recompute_context=attn_recompute_context
    )
    route_input, route_probs = self._moe_router_preprocess(
        shortcut_hidden=moe_hidden_states,
        padding_mask=padding_mask,
        packed_seq_params=moe_packed_seq_params,
        recompute_context=attn_recompute_context,
    )
    if overlap:
        self.route_ready_event.record(torch.cuda.current_stream())
```

图里那个并行 block 让 dispatch 与注意力重叠，而 combine 仍然裸露；捷径连接的 MoE 则用左右的层盖住了大部分 dispatch 和 combine。代价落在路由器上：它现在读的是「被重叠的那段计算之前」的流，永远看不到那段计算的输出。

模型因此学得更差吗？Cai 等人和 LongCat-Flash 团队看到的是差不多。Cai 等人在 LLaMA2 风格模型的每个 block 里都放 MoE，接成 Pos-1（Megatron 实现的那个 ScMoE 变体），在零样本基准上平均略高于标准 top-2 MoE。在 LongCat-Flash 里，带不带捷径的损失曲线几乎分不出来。AMD 的 FarSkip-Collective 把 block 接成「计算用的残差里还不含最新通信回来的输出」；装进 Instella-

MoE 后，在 200B 训练 token 之后与 MoE block 的基准均值持平，尽管 AMD 那两份报告的表里它在 MMLU 和 HumanEval+ 上是输的。

换回来的是速度。ScMoE 在一个 8 卡 NVLink 节点上训练比标准 top-2 MoE 快 1.12 到 1.14 倍，Instella-MoE 的预训练吞吐提升 12.7%。LongCat-Flash 直接量了裸露通信本身：用捷径连接的 MoE 之后，dispatch 与 combine 里没被重叠的时间占比从 25.3% 降到 8.4%（在他们的 32 卡专家并行组里，用管线化的 all-gather 与 reduce-scatter，而不是 All-to-All）。

在 Megatron 里，图里那行「并行」（ScMoE 的 Pos-1）对应的是 `--moe-shortcut-connection`：让每个 MoE 读它前面那个注意力层的输入，于是路由器不用等那次注意力。共享专家仍然读注意力的输出（这是图里没画的一个细节），路由专家则在主 stream 上跑在注意力核心之后、输出投影之前，就像图里那行的专家排在注意力之后。光重接不会重叠任何东西：只有 `--moe-shortcut-parallel` 把 All-to-All 放到侧 CUDA stream 上，它们才重叠。可选的、给合并后输出加一层归一化是 `--moe-shortcut-post-norm`。三个开关都只在 upstream main 上，而且只能在 hybrid stack 里用，不能和 FSDP、CUDA graphs、全量重算或 `--moe-shared-expert-overlap` 一起用。不改 block 的话，Megatron 也能把 All-to-All 藏到另一个 micro-batch 后面（3.7 重叠）。

裸露的 All-to-All 让每一步变慢，而 trace 能告诉你漏了多少。抓一条 nsys 时间线（`--profile` 配合 `--profile-step-start`、`--profile-step-end`，见 3.7 剖析），把侧 stream 上的 All-to-All kernel 和主 stream 上的计算对比：All-to-All 单独在跑的那几段，就是裸露的部分。

什么时候用它：dispatch 裸露、且你能切到 Megatron 的 hybrid stack 时，就用捷径 block（图里「并行」那行）：ScMoE 的 Pos-1 设计在单个 8 卡 NVLink 节点上比标准 top-2 MoE 快 1.14 倍。`--moe-shortcut-connection` 和 `--moe-shortcut-parallel` 两个都要设，只有后者才让 All-to-All 重叠；同时别开 FSDP、CUDA graphs、全量重算和 `--moe-shared-expert-overlap`，它们和它不兼容。

该盯什么

部位	症状	该盯的信号
稠密层	「学不好」：最早的 MoE 层不均衡，或者路由塌掉。「炸了」：一个图像生成 MoE 没有开头稠密层就发散了。「慢」：每层都 MoE 意味着更多 All-to-All	稠密块之后头几个 MoE 层的辅助损失（`--moe-per-layer-logging`）与 MaxVio（路由 trace，A.1 节）
块接线	「慢」：All-to-All 没藏在计算后面	All-to-All 裸露了多少：在 nsys 时间线里（3.7 剖析），把侧 stream 的 All-to-All kernel 与主 stream 的计算对比

路由器、均衡器、专家、块布局都选完之后，架构就完整了，剩下的就是把它训出来。训练该从哪儿开始？用哪个优化器？又怎么知道什么时候出事了？

3.6 训练：起点、优化器、稳定性

训练 MoE 会多出一些稠密模型没有的失败模式，做大 MoE 的团队都撞过其中好几种。一个 Switch Transformer 在 bfloat16 下发散了；六个 ST-MoE 基线里有两次跑崩。路由也可能塌到少数几个专家上，而不均衡会自我强化。这些运行到底为什么失败？又怎么能提前发现自己的即将失败？

各模型作者报告的训练选择。初始化与优化器几列标出与本节相关的选择；MTP 一列描述训练方式，可能与发布的推理配置不同。

模型	初始化	优化器	MTP	稳定性选择
DeepSeek-V3	随机初始化	AdamW	1 层 MTP	梯度范数裁到 1.0；FP8 训练期间 embedding、输出头、MoE 门控、归一化与注意力保持 BF16 或 FP32
DeepSeek-V4.1-Flash	从零开始	线性权重用 Muon，归一化与非矩阵参数用 AdamW	预训练期间主干不用 MTP；DSpark 之后单独训练	SwiGLU 截断到 10；按模态分别均衡
Kimi K3	从零开始，语言与视觉一开始就联合训练	矩阵参数用 Muon，注意力投影按 head 更新	1 层 MTP	权重截断、注意力输出用 FP32、LatentMoE 归一化、SiTU-GLU、分位数均衡
LongCat-Flash	先训 14 层，再叠成 28 层	Adam 家族，超参按 Adam 的缩放规则迁移	MTP 当作辅助目标	隐状态 z-loss、路由器梯度监控、Adam epsilon 取 1e-16、确定性计算
Step 3.5 Flash	未报告	Muon，Polar Express 迭代在 FP16 下做	3 个 MTP head	梯度裁到 1.0；专家激活截断；把专家范数与损失一起盯

第一个决定是训练从哪开始：随机权重，还是从 2.4 已经训好的稠密模型开始。从稠密 checkpoint 起步能换来什么，又要付什么代价？

向上循环（Upcycling）

Upcycling 把稠密 checkpoint 的 MLP 拷进专家，作为 MoE 的起点。和稠密 MLP 一样宽的专家拿到完整一份，窄的拿到一片：在我们那次运行里，稠密 MLP 宽 8192、专家宽 1024，Megatron 把稠密 MLP 切成 8 片，每片拷进 64 个专家中的 8 个。不管哪种，MoE 都从稠密那次停下地方接着开始。

拷完权重还有第二个选择：优化器状态与学习率调度要不要一起带过来。Komatsuzaki 等人接着稠密那次调度的位置继续，但只在视觉模型上复用优化器状态，因为语言模型上没有收益。Megatron 的 upcycling 步骤会在第 1 个迭代就把 MoE 存下来、且不带优化器状态（`training.py#L3069-L3083`），所以 MoE 这次是用全新优化器和全新调度开始的。

这个抢跑不是免费的：它花的是稠密那次已经花掉的算力，而从零开始的运行最终会追上来。快慢差别很大：在 OLMoE 里，从零开始的那次在 500B token 之后追上，相当于稠密模型训练量的 25%；而在 Komatsuzaki 等人的稀疏 upcycling 运行里，要花稠密 checkpoint 算力的 120%。下图按 OLMoE

公开的训练日志重画了那个对比：一个从稠密 OLMo-1B（已训 2T token）upcycle 来的 MoE，对一个从零开始训的同样的 MoE，各训 610B token。

> 图 3.6-a upcycle 自 OLMo-1B 的 MoE vs 从零训练的 MoE（原站为交互图，此处从略）

图里 upcycle 那次在四项指标上都是一开始就领先，从零的那次只是慢慢把差距缩小。它的 HellaSwag 与 MMLU 准确率在约 500B token 处追平（作者把「追上」定在这里），而训练与验证损失要到约 600B token 才追平。

训好的 MoE 也能用同样的方式长大——只是拷的是它自己的专家，而不是稠密 MLP。下图把一个 8 专家 MoE 变成 16 专家，并展示拷贝对路由的影响。

> 图 3.6-b 把 8 专家按三种方式扩到 16 专家后的路由行为（原站为交互图，此处从略）

在 top-2 路由、每个副本继承原专家路由器那一行的情况下，图里的精确拷贝会打平：每个 token 都挑一个专家和它的副本、权重相同，于是实际只跑了一个不同的专家。给副本加扰动，路由器就能在这一对之间拿到梯度；随机新专家能避免打平，但把路由权重压在未训练的专家上。三种做法都被用来长大模型：Expert Upcycling 每个专家连同路由器那一行一起拷，从 32 到 64；K-EXAONE 2.0 给每个副本加一个随机旋转，从 128 到 256；Expert Upcycling 还试了 Drop-Upcycling，把每个副本的部分权重重新初始化（作者发现最好的是重初始化一半）；EMO 则从高斯分布里抽新专家和新的路由器行。在 Megatron 里，从稠密 checkpoint upcycle 是 `--moe-use-upcycling` 加 `--moe-upcycling-granularity`，而在那个 pin 上还需要 2.5 里说的两处本地修补。

```
# megatron/training/training.py#L3043-L3083（先建稠密、再 upcycle、再存）
if args.moe_use_upcycling:
    torch.distributed.barrier()
    assert not checkpoint_exists(args.save), (
        "The upcycling destination directory already exists. "
        "Please check if --moe-use-upcycling is mistakenly enabled. "
        "Upcycling should only be set for the first run when converting the dense model. "
        "All subsequent runs should remove this flag. "
    )
    # before changing moe related global args, save them in local variables
    num_experts = args.num_experts
    expert_model_parallel_size = args.expert_model_parallel_size
    moe_ffn_hidden_size = args.ffn_hidden_size

    # set dense model related args in to global args before getting dense model
    args.num_experts = None
    args.expert_model_parallel_size = 1
    args.ffn_hidden_size = moe_ffn_hidden_size * args.moe_upcycling_granularity

    # get dense model
    dense_model_for_upcycling = _build_model_wrapper(wrap_with_ddp=True)

    # recover moe upcycling related args in global args before executing upcycling
    args.num_experts = num_experts
    args.expert_model_parallel_size = expert_model_parallel_size
    args.ffn_hidden_size = moe_ffn_hidden_size

    # execute upcycling
    _, args.num_floating_point_operations_so_far = upcycling_utils.load_and_upcycle_model(
        load_checkpoint,
        unwrapped_model,
        dense_model_for_upcycling,
        load_kwargs={
            'model': dense_model_for_upcycling,
            'optimizer': None,
            'opt_param_scheduler': None,
        },
    )
    args.iteration = 1
    save_checkpoint(
        args.iteration, model, None, None, args.num_floating_point_operations_so_far
    )
```

那次加载时，`load_checkpoint` 拿到的 `ddp_model` 是稠密模型（`checkpointing.py#L2632-L2635`），并且稠密模型是用「`num_experts` 未设」的参数现建的，而不是用 `pretrain_gpt.py` 启动时建好的那份 config 容器——这样稠密 checkpoint 才能加载进一个稠密模型（`training.py#L2917-L2925`）。

搞砸的 upcycle 会丢掉抢跑优势，所以要拿它的 loss 和「稠密那次继续训同样多 token」比。又因为专家一开始是完全相同的拷贝，还要在它们分化期间逐层看 MaxVio，从路由 trace 离线算（A.1）。

什么时候用它：已有稠密 checkpoint、且这次 MoE 训练较短时用。因为从零开始的那次在 OLMoE 里要 25% 的稠密训练 token 才追上，在 Komatsuzaki 等人的运行里要 120% 的算力。upcycling 是

一次性的初始化，只在第一次运行设，且在那个 pin 上要先用 2.5 的两处本地修补。

不管你是 upcycle 还是用随机权重，只决定权重从哪儿出发；从第一步起，就是优化器决定它们往哪走。可是每个专家只看见路由给它的 token，那么一个只拿到几把 token 的专家，该被推得和忙碌专家一样远吗？

优化器

DeepSeek-V3 用的 AdamW 把每个权重的更新按它自己的梯度历史缩放，所以一个专家走多远，取决于它那些 token 产生的梯度。而近期好几个 MoE 用的、针对权重矩阵的 Muon 换了个尺度的算法：它把每次更新正交化，把非零奇异值往 1 上推，再乘一个只由矩阵形状决定的系数，于是步长不再取决于背后梯度有多大。

Megatron 对每个专家的矩阵各做一次正交化（`transformer_engine.py#L2665-L2668`、`transformer_config.py#L956`、`emerging_optimizers.py#L147-L149`），跟 DeepSeek-V4 一样，于是路由器很少挑中的专家也能走得和最忙的差不多远——只是方向是自己的——而且在它一个 token 都没拿到的 step 里，它还会沿动量继续动。下图用 49 个玩具专家画出了这件事：AdamW、按专家各自做 Muon、把全部专家当一个矩阵做 Muon，三种情况下更新大小与每步 token 数的关系。

> 图 3.6-c 更新大小 vs 每步 token 数：AdamW / 逐专家 Muon / 全体合一 Muon（原站为交互图，此处从略）

玩具里某个专家的 token 变少时，AdamW 下它的更新随梯度变成噪声而缩小，全体合一的 Muon 下随梯度大小一起缩小，而逐专家 Muon 能把最忙专家的步长压到每步几个 token。再往下，逐专家的步长也开始缩：每个 token 只给梯度加一个方向，token 这么少的时候，连最近几步的动量都填不满这个专家的矩阵。

图说明的是 Muon 怎么挪动每个专家；至于它能不能训出更好的 MoE，是在整模型上量过的。Moonlight（3B 激活、16B 总参）用 Muon 明显好于同样设置下用 AdamW 训练的那份，比较点在约 1.2T token。

上了 Muon 之后还剩一个选择，各家做法不同：用哪个优化器更新路由器。DeepSeek-V4 让它留在 Muon 上，而 Qwen3.8-Next 在 Muon 早期把路由器弄不稳之后，把它挪到了 AdamW。Lau 和 Su 在两个小 MoE（一个按 OLMoE-1B-7B 造型，一个缩小版 gpt-oss）上比较了路由器的优化器：隐藏矩阵用 Muon 时，路由器上用 AdamW 最终略差于「行归一化」或「左谱」的路由更新。

在 Megatron 里，Muon 是 `--optimizer muon`；要把路由器放到 Adam 上，就加 `--moe-router-skip-muon`，从 Megatron Core v0.20 起可用。

```
# megatron/core/optimizer/emerging_optimizers.py#L147-L154（哪些参数进 Adam）
def _is_nonlinear_or_embedding(param):
    """True for parameters that should NOT use the emerging optimizer."""
    return getattr(param, 'is_embedding_or_output_parameter', False) or len(param.shape) != 2

def _is_muon_excluded(param):
    """True for parameters that should use the scalar optimizer instead of Muon."""
    return not getattr(param, 'use_muon', True) or _is_nonlinear_or_embedding(param)
```

```
# megatron/core/transformer/moe/router.py#L88-L91（把路由器挪出 Muon）
if self.config.moe_router_skip_muon:
    setattr(self.weight, 'use_muon', False)
    if self.bias is not None:
        setattr(self.bias, 'use_muon', False)
```

```
# megatron/core/optimizer/emerging_optimizers.py#L303-L324（先正交化，再按形状缩放）
if scale_shape is None:
    size = [grad.size(-2), grad.size(-1)]
    if partition_dim is not None:
        size[partition_dim] *= get_pg_size(tp_group)
else:
    # This overrides only the final Muon scalar; NS still uses grad's physical
    # shape and partition metadata for its collectives.
    size = scale_shape
# Only forward the kwarg when enabled; older emerging_optimizers do not
# accept it at all, and __init__ has already rejected use_syrk on those.
ns_kwargs = {"use_syrk": True} if use_syrk else {}
orth_grad = newton_schulz_tp(
    grad,
    steps=num_ns_steps,
    coefficient_type=coefficient_type,
    tp_group=tp_group,
    partition_dim=partition_dim,
    tp_mode="duplicated" if tp_mode_this_group == "blockwise" else tp_mode_this_group,
    **ns_kwargs,
)
scale_factor = get_muon_scale_factor(size[0], size[1], mode=scale_mode)
```

Muon 下专家可能塌，而更新尺度太大会让训练发散。要盯每个专家收到什么、产出什么：每步每专家 token 数，从路由 trace 离线算（A.1）；以及每层「最大 / 最小专家输出范数各自除以该层中位数」，用我们那个前向 hook（A.1）。

什么时候用它：你有余力调它的时候用 Muon。从 `--muon-extra-scale-factor 0.2` 起步，因为 Megatron 默认的 1.0 在 Mellum2 的测试里把一个 7B 稠密模型跑崩了；路由器的优化器用 `--moe-router-skip-muon` 选，两种做法都有人用。它不能和 Megatron-FSDP 或 Torch-FSDP2 一起跑。

不管优化器怎么挪权重，每一步都只从「预测下一个 token」里学。能不能靠预测得更远，从每一步里榨出更多信号？那要付多少步时？

MTP（多 token 预测）

多 token 预测为每多预测一个 token 加一个 transformer 层：这一层拿主模型的隐状态和下一个 token 的 embedding，通过共享输出头预测再下一个 token。DeepSeek-V3 发现它在两个测试规模的大多数基准上都更好，而且推理时可以把这个额外的层丢掉。下图跟着序列的一个位置走完这套接线。

> 图 3.6-d MTP 层：隐状态 + 下一 token embedding → 拼接投影 → transformer 块 → 共享输出头（原站为交互图，此处从略）

进来的时候两个输入各自过自己的 RMSNorm，MTP 层把它们拼起来、从两倍隐藏维投影回隐藏维，再进自己的 transformer 块。两个预测都从同一个输出头出来，MTP 损失加权后加到主损失上。

这个额外的层要花步时，而在流水线并行下这笔开销落在一个地方：除非布局把它挪开，Megatron 把 MTP 层放在最后一个 stage（`multi_token_prediction.py#L886-L905`），而那个 stage 本来就拿着 MTP 要复用的输出头。在我们 256 节点的运行里，带一层 MTP 的早期 step 比不带的慢约 37%，把模型摊到 8 个流水 stage 也没把这个差距抹平。Mellum2 报的是一个 14B MoE 多花 7% 训练时间。

```
# megatron/core/transformer/multi_token_prediction.py#L1709-L1714（隐状态加下一 token embedding）
hidden_states = apply_module(self.hnorm)(hidden_states)
hidden_states = make_viewless_tensor(
    inp=hidden_states, requires_grad=True, keep_graph=True
)
hidden_states = torch.cat((decoder_input, hidden_states), -1)
hidden_states, _ = self.eh_proj(hidden_states)
```

```
# megatron/core/transformer/multi_token_prediction.py#L1276（损失按层数摊）
mtp_loss_scale = config.mtp_loss_scaling_factor / config.mtp_num_layers
```

```
# megatron/core/transformer/multi_token_prediction.py#L932-L937（无自定义布局时全放最后一个 stage）
# without custom PP layout, we only support put all of MTP layers on the last pipeline stage
if mtp_num_layers is not None:
    if pp_size is None:
        # Compatibility fallback for callers without explicit pipeline metadata.
        pp_size = parallel_state.get_pipeline_model_parallel_world_size()
    mtp_on_this_rank = pp_rank == pp_size - 1
```

那个差距很大一部分可能来自层放的位置。一个流水 step 要等最慢的 stage，所以加在某个 stage 上的一层，会让这一步慢「它在该 stage 工作量中的占比」那么多，而不是它在整个模型里那个小得多的占比。Ling 2.0 团队撞上了同样的不平衡：他们量到一个 MTP 层的算力约是标准 MoE 层的 1.7 倍，围绕它重新切分流水线后端到端赚了 40%。

Megatron 用 `--mtp-num-layers` 打开 MTP（多加几层深度），`--mtp-loss-scaling-factor` 给 MTP 损失加权：DeepSeek-V3 在前 10T token 用 0.3、之后用 0.1，Mellum2 用 0.1。要指定哪个 stage 放 MTP 层（记为 `m`），用 `--pipeline-model-parallel-layout`。如果只想要 MTP 层当投机解码的草稿模型，`--mtp-detach-heads` 会把它的输入 detach，于是 MTP 损失只训 MTP 层、不动主模型。

MTP 打开时 Megatron 会在 lm loss 旁边记一个 `mtp_1 loss`，而 MTP 层的代价要别处看：步时，以及持有它的那个 stage 的显存。要找出最慢的 stage，用 `--timing-log-level 2 --timing-log-option all` 逐流水 rank 记前向/反向计算时间（A.1）；持有 MTP 层的那个 stage 应该会表现为最慢的 rank。

什么时候用它：当 DeepSeek-V3 报告的那种更密的训练信号值这点步时时用。流水线并行下，用 `--pipeline-model-parallel-layout` 安排 MTP 层的位置，否则它落在最后一个 stage。

到此为止，每一部分都告诉了你该怎么盯它自己的选择。可有些故障不属于任何单个选择，且可能从模型任何地方开始——那每一次运行都该常备什么来抓它们？

稳定性

MoE 训练可能大声失败，也可能悄悄失败。大声的会在 loss 上现形（尖峰或变 NaN），而悄悄的（比如路由塌缩、离群专家）可以让 loss 一直平着。

最大声的失败也最好抓，因为 Megatron 默认每步都检查 loss 与梯度里有没有 NaN（3.0）。接下来你想知道的是该不该怪硬件，所以检查触发时 Megatron 会重跑这一步（`--rerun-mode validate_results`，也是默认开）。第一次重跑在同一张卡上，用来区分「偶发错误」和「会重复的错误」。如果 NaN 复现，Megatron 以退出码 16 退出，让任务从另一张卡的 checkpoint 重启，那里的第二次重跑就能区分「坏卡」和「数据或模型真的产生了 NaN」。

坏硬件和坏 batch 不一定会变成 NaN，它们也可能返回一个指数大得离谱的有限 loss 或梯度。Megatron 用 `--check-for-spiky-loss` 和 `--check-for-large-grads` 抓这类：把「比之前见过的 100 个里最大的还大 10 倍」标出来，在同一张卡上重跑一次，记录这个值是否复现，然后继续训练。

这些检查都是对「已经出错的 step」做出反应。要从源头不让 loss 崩，3.1 已经保护了路由器（fp32 计算加一个压住 logits 的 z-loss），近期的大 MoE 还各自加了护栏。有的在数值长大之前先截：DeepSeek-V3 与 Step 3.5 Flash 裁梯度范数，DeepSeek-V4.1-Flash 截专家的 SwiGLU 激活，Step 3.5 Flash 也截专家激活。有的把最敏感的操作留在更高精度：DeepSeek-V3 在 FP8 训练时把 embedding、输出头、MoE 门控、归一化与注意力留在 BF16 或 FP32，Kimi K3 把注意力输出留在 FP32。LongCat-Flash 把 z-loss 的思路搬到了隐状态上。

悄悄的失败可能根本不在 loss 上现形，所以得去别处找。下图在一个健康运行旁边模拟了三种：路由塌缩、死专家、离群专家。

> 图 3.6-e 健康运行 vs 路由塌缩 / 死专家 / 离群专家（原站为交互图，此处从略）

模拟让 loss 在三种故障下都保持平直，所以只有别的信号能把它们和健康运行区分开。路由塌缩把 MaxVio 顶上去。死专家把最小的专家输出范数拉到该层中位数之下很远；离群专家把最大的推到很远之上——从约 14 倍到约 5,700 倍中位数（Step 3.5 Flash 第 45 层的那个数）。loss 报警（比至今最低 loss 高 0.1）借的是 Qwen3.8-Next 的尖峰判据那个余量。

```
# pretrain_gpt.py#L263-L292（NaN、Inf、尖峰损失）
# Check individual rank losses are not NaN prior to DP all-reduce.
rerun_state_machine = get_rerun_state_machine()
if args.check_for_nan_in_loss_and_grad:
    rerun_state_machine.validate_result(
        result=loss,
        rejection_func=torch.isnan,
        message="found NaN in local forward loss calculation",
        tolerance=0.0,  # forward pass calculations are deterministic
        fatal=True,
    )
    rerun_state_machine.validate_result(
        result=loss,
        rejection_func=torch.isinf,
        message="found Inf in local forward loss calculation",
        tolerance=0.0,  # forward pass calculations are deterministic
        fatal=True,
    )
# Check for spiky loss
if args.check_for_spiky_loss:
    rerun_state_machine.validate_result(
        result=loss,
        rejection_func=partial(
            rerun_state_machine.is_unexpectedly_large,
            threshold=SPIKY_LOSS_FACTOR,
            context="loss",
        ),
        message="Spiky loss",
        tolerance=0.0,  # forward pass calculations are deterministic
        fatal=False,
    )
```

```
# megatron/core/distributed/param_and_grad_buffer.py#L389-L425（逐 bucket 的梯度范数检查）
def check_grads(self, check_for_nan_or_inf, check_for_large):
    """
    Make sure norm of grads in bucket are not NaN prior to data-parallel
    all-reduce / reduce-scatter.
    """
    rerun_state_machine = get_rerun_state_machine()
    for i in range(len(self.buckets)):
        grad_norm = self.buckets[i].grad_data.norm(p=2)
        # check for NaN, Inf and unexpectedly large grads
        if check_for_nan_or_inf:
            rerun_state_machine.validate_result(
                result=grad_norm,
                rejection_func=torch.isnan,
                message=f"found NaN in local grad norm for bucket #{i} "
                f"in backward pass before data-parallel communication collective",
                tolerance=0.001,  # 0.1% tolerance to account for non-determinism
                fatal=True,
            )
```

尖峰方面，用 z-loss 盯路由器的 logits（3.1）。那些让 loss 保持平直的故障需要各自的信号：路由塌缩看逐层 MaxVio（从路由 trace 离线算，A.1），与负载均衡损失并排看；死专家与离群专家看每层「最大、最小专家输出范数分别除以该层中位数」，用我们那个 hook（A.1）。

什么时候用它：一直用——保留默认的 NaN 检查与 `--rerun-mode validate_results`。

该盯什么

部位	症状	该盯的信号
Upcycling	「学不好」：开始阶段 loss 下降慢	loss 与「稠密那次继续训同样 token」对比；在拷贝的专家分化期间，还要逐层看 MaxVio（路由 trace，A.1 节）
优化器	「学不好」：专家塌缩。「炸了」：正交化在 bf16 下跑，或更新尺度太大	每步每专家 token 数（路由 trace，A.1 节）；逐层最大与最小专家输出范数各自除以中位数（前向 hook，A.1 节）
MTP	「慢」：某个流水 stage 比别人慢。「OOM」：53 层模型放在 4 个流水 stage 上带 MTP 时爆显存	`mtp_1 loss` 与 LM loss 并列看；步时与峰值显存；逐流水 rank 的前向/反向计算时间（`--timing-log-level 2 --timing-log-option all`，A.1 节），尤其是持有 MTP 的那个 stage
稳定性	「炸了」：NaN 与 loss 尖峰。「学不好」：路由塌缩、死专家、离群专家	路由器 z-loss（3.1 节）——ST-MoE 把自己那次不稳定归因到的就是路由器 logits；逐层 MaxVio（路由 trace）与最大、最小专家输出范数除以中位数（A.1 节）

一次训练得干干净净的运行，仍然可能有很大一部分时间在等通信而不是在算。MoE 的一步里时间都去哪了，能拿回多少？

导航：[[2026-11-moe-handbook|全书目录]] · ← 第 3 章（中）专家 · 第 3 章（下）系统工程 →

第 3 章 解剖 MoE 训练：系统工程

> 说明：原站里的图都是交互式组件，这里无法复现，一律保留图注并标注「（原站为交互图，此处从略）」。原站大部分图不给编号，本文按节内顺序编号（如 图 3.7-a）。

3.7 系统工程：时间去哪了，怎么拿回来

每个 MoE 层都要把 token 发给专家再收回来，而在真实运行里这部分通信可能吃掉大半步。我们在一个节点上追的那一步里，有一半时间花在裸露的 All-to-All 上，而那时路由器还不均衡。下图放大这两次传输——把 token 发出去的 dispatch 和收回来的 combine——它们在节点内走 NVLink、跨节点走网络。

> 图 3.7-a 一个 MoE 步的时间构成：dispatch / 专家计算 / combine / 其余（原站为交互图，此处从略）

你可以把专家摆得让更少流量穿过节点间那张更慢的网，把剩下的流量藏到别的计算后面；可以把这份内存代价控制住，给每个 token 能到达的节点数加上限；也可以用丢 token 或降精度换速度。每一部分都来自各自的设置，收益不能相加，要当作备选方案逐个测。

各模型报告里记录的系统选择。并行与重叠描述训练；后训练量化单独标注。这些是各模型的具体实现，不是可以互换的 Megatron 开关。

模型	互联与重叠	内存与精度	路由约束
DeepSeek-V3	跨 8 节点 EP64；DualPipe 重叠计算与通信	FP8 混合精度训练；重算 RMSNorm、MLA 上投影与 SwiGLU 输出	每 token 最多 4 个目标节点；不丢 token
DeepSeek-V4	融合 MoE 流水调度把专家组按波次排布，重叠 dispatch、GEMM 与 combine	张量级 checkpoint 与选择性重算	取消了 V3 的目标节点上限
Kimi K3	优化器只取本地持有的参数分片；通信与计算管线化	省掉全参数 gather buffer；SFT/RL 用 MXFP4 专家权重、MXFP8 激活	发布配置里只有一个专家组
LongCat-Flash	32 卡一组的 EP；管线化 all-gather/reduce-scatter + 捷径连接 MoE	BF16 训练 + 确定性 kernel	恒等专家不需要专家通信

在动上面任何一个杠杆之前，你得先看清自己那一步的时间花在哪，因为每种浪费对应不同的解法。单节点上剖析出来的那一步长什么样？

剖析（Profiling）

Megatron 能帮你把任意一步录成 trace。只给 `--profile` 会录 nsys trace，加 `--use-pytorch-profiler` 则改录 torch profiler trace（`argument_utils.py#L706-L709`）。两种情况下都用 `--profile-step-start`、`--profile-step-end`、`--profile-ranks` 选要录哪几步、哪些 rank。

我们追的那一步来自一个 7B-A1B 的 MoE：总参 7B、每 token 激活 1B，16 层、每层 64 专家、top-8 路由，在一个 8×H100 的节点上训练。追它只花了一个短任务：录第 4 个迭代，启动后约 6 分钟内就跑过了。

那次 All-to-All 占了一半步时，可这一半里有多少是网络真的在搬数据？当某张 GPU 的专家收到比别的卡更多的 token 时，它的专家计算结束得晚，而 NVIDIA Megatron Core 团队报告说，其他 rank 随后会在同步点等它。在 MoE 层里，All-to-All 可能就是这样的同步点之一，所以它在 trace 里的时间有一部分是「在等」而不是「在传」。我们那一步的负载均衡损失是 3.2（1 表示均衡），所以路由器还是不均衡的。同一配置另跑一次、把路由强制成均匀之后，同一个迭代快了 40%。

```
# megatron/training/training.py#L4955-L4982（逐 rank 的 torch profiler 窗口）
if (
    cfg.profiling.use_pytorch_profiler
    and (len(cfg.profiling.profile_ranks) == 0 or
         torch.distributed.get_rank() in cfg.profiling.profile_ranks)
):
    if cfg.profiling.pytorch_profiler_collect_chakra:
        et_dir = Path(f"{cfg.logger.tensorboard_dir}/../chakra")
        et_dir.mkdir(parents=True, exist_ok=True)
        et = torch.profiler.ExecutionTraceObserver().register_callback(
            f"{et_dir}/rank-{torch.distributed.get_rank()}.json.gz"
        )
    else:
        et = None

    def trace_handler(p):
        profile_dir = Path(f"{cfg.logger.tensorboard_dir}/../torch_profile")
        profile_dir.mkdir(parents=True, exist_ok=True)
        p.export_chrome_trace(f"{profile_dir}/rank-{torch.distributed.get_rank()}.json.gz")

    prof = torch.profiler.profile(
        schedule=torch.profiler.schedule(
            wait=max(cfg.profiling.profile_step_start - 1, 0),
            warmup=1 if cfg.profiling.profile_step_start > 0 else 0,
            active=cfg.profiling.profile_step_end - cfg.profiling.profile_step_start,
            repeat=1,
        ),
        on_trace_ready=trace_handler,
        record_shapes=cfg.profiling.pytorch_profiler_collect_shapes,
        with_stack=cfg.profiling.pytorch_profiler_collect_callstack,
        execution_trace_observer=et,
    )
    prof.start()
```

在你自己的运行上，先看 3.2 那些均衡信号。用 `--save-tokens-per-expert-interval` 写出的 token 计数算 MaxVio，能看出负载在专家之间有多不均。把每张 GPU 上各专家的计数加起来，就知道哪个 rank 收到的 token 最多、也因此最可能被别人等。想知道这份不均衡让 All-to-All 多花了多少，就带 `--moe-router-force-load-balancing` 重追一次——它把路由器的 logits 换成随机的，让每个专家负载大致相同（`router.py#L1185-L1187`）。如果强制均衡之下 All-to-All 变小了，说明不均衡正在白花你的时间，更好的均衡（3.3）能把这份时间拿回来；剩下的主要就是传输本身，本节的其余部分处理它。

trace 还能告诉你哪些 MoE kernel 值得融合。dispatch 之前 Megatron 会把 token 重排，让同一个专家的 token 挨在一起（`token_dispatcher.py#L668-L682`），combine 之后再排回去（`#L893-L900`）。带上 `--moe-permute-fusion`，做这件事的 permute 与 copy kernel 从步时的 11.45% 降到 7.14%，步时也跟着降。

每一项改动都按步时或 TFLOP/s 判断。trace 再告诉你这个数字为什么变：读一读 All-to-All 在步里留下了多少裸露份额。

什么时候用它：每个新配置用一次——在你要跑的那个节点数上追一步。如果 All-to-All 占比很大，先看 MaxVio，再带 `--moe-router-force-load-balancing` 重追：All-to-All 变小就先修均衡（3.3），没变小就去做下面的互联与重叠。如果 permute 与 copy kernel 占比很大，试试 `--moe-permute-fusion`。

我们追的那一步把 All-to-All 留在了单节点内、走 NVLink，而更大的运行不总能这样。当专家并行组跨两个节点、All-to-All 必须穿过节点间网络时，这一步会怎样？

互联（Interconnect）

跨节点时，All-to-All 离开 NVLink，走集群的网卡网络（比如 InfiniBand，或者 AWS 的 EFA），所以第一件要检查的事是 NCCL 到底有没有用上那张网。它可能在你明明要求了的情况下退回普通 TCP socket：我们两次运行设了 `FI_PROVIDER=efa`、也按名字指定了 OFI 插件，NCCL 仍记着 "Using network Socket"，那两次比我们在 EFA 上的 EP16 运行慢了约 25 倍。下图按顺序列出我们试过的设置。

> 图 3.7-b 让 NCCL 走上 EFA 的尝试顺序与结果（原站为交互图，此处从略）

图里，只有当主机的 RDMA 库排在库路径最前面时，NCCL 才走到 EFA；按路径或按名字指定插件，都还留在 socket 上。网通了之后，剩下的选择是专家并行规模 `--expert-model-parallel-size`，它决定 All-to-All 要走多远。专家张量并行取 1、rank 顺序取默认时，Megatron 用连续的 rank 组成每个 EP 组（`parallel_state.py#L921-L929`）。于是能整除单节点 GPU 数的 EP 规模会把 All-to-All 留在该节点内的 NVLink 上，更大的规模就摊到多个节点。我们的节点每台 8 张 H100，所以下图比较单节点 EP8 与跨两节点的 EP16。

> 图 3.7-c 单节点 EP8 vs 跨两节点 EP16 的 TFLOP/s（原站为交互图，此处从略）

摊到两个节点后，即便 EFA 正常工作，EP16 也比单节点 EP8 低 59%——尽管这两次运行在节点本身、以及每步累积的 micro-batch 数上也不一样。

```
# megatron/core/parallel_state.py#L920-L930（专家网格，EP 排在 TP 之后）
expert_order = _inject_gtp_remat_axis(order, after="ep")
expert_decoder_rank_generator = RankGenerator(
    tp=expert_tensor_parallel_size,
    ep=expert_model_parallel_size,
    dp=expert_data_parallel_size,
    pp=pipeline_model_parallel_size,
    cp=1,
    order=expert_order,
    rank_offset=rank_offset,
    gtp_remat=expert_gtp_remat_size,
)
```

```
# megatron/core/parallel_state.py#L1419-L1428（连续 rank 组成 EP 组）
for ranks in expert_decoder_rank_generator.get_ranks('ep'):
    group = create_group(
        ranks,
        timeout=timeout,
        pg_options=get_nccl_options("ep", nccl_comm_cfgs),
        group_desc="EXPERT_MODEL_PARALLEL_GROUP",
    )
    if rank in ranks:
        _EXPERT_MODEL_PARALLEL_GROUP = group
        _EXPERT_MODEL_PARALLEL_RANKS = ranks
```

退回 socket 会把跨节点运行拖慢到上面那个倍数，所以先看启动日志。设 `NCCL_DEBUG=INFO`，NCCL 会打印 "Using network" 和它选中的传输层。你要看到的是自家网的传输层（比如 EFA 上 OFI 插件对应的 Libfabric），出现 "Using network Socket" 就是退回去了。确认网通了，就按 2.1 用 nccl-tests 比较节点内与跨节点的 alltoall 总线带宽。然后分别在「装得进一个节点」和「跨两个节点」的 EP 规模下量每 GPU TFLOP/s。

什么时候用它：模型装得下时先在节点内用专家并行：从那里起步，加节点当数据并行副本，在定下来之前把两种摆法都按你的节点数量过一遍时间。

就算网是好的，穿网的 token 仍然压在关键路径上、让 GPU 干等。token 在路上的时候，GPU 能干点别的活吗？

重叠（Overlap）

我们在 3.5 块接线 已经看到，改架构能把 All-to-All 藏进一个 micro-batch 里。并行 block 让 dispatch 在注意力计算时就走，捷径连接的 MoE 还能把 combine 藏在旁边跑的稠密计算后面。可重接线会改变路由器看到的东西，而你也许不想动已经定好的架构。不重接线的话，裸露的 All-to-All 很难在一个 micro-batch 内藏起来。可一个训练步会跑很多 micro-batch，而它们的梯度求和之前，彼此的工作并不互相等待。

Megatron 就是借这份工作：`--overlap-moe-expert-parallel-comm` 让一个 micro-batch 的前向与另一个的反向同时跑，于是一个的 All-to-All 藏到另一个的计算后面（`model_parallel_config.py#L349-L352`）。流水调度本来也会把一个 micro-batch 的前向和另一个的反向配对，Ultra-Scale Playbook 有讲过，DeepSeek-V3 的 DualPipe 就是靠这个配对藏它的 All-to-All。下图画了一个 MoE 层在有无这种交织时的样子。

> 图 3.7-d 一个 MoE 层：micro-batch 顺序执行 vs 前向/反向交织（原站为交互图，此处从略）

按一个 micro-batch 接一个画的时候，这一层把四次 All-to-All 全裸露出来。交织的那版还打开了 `--delay-wgrad-compute`，Megatron 的 MoE 指南把它与这个重叠配对使用（`README.md#L434`）。这个开关把权重梯度的计算推后，于是反向的 dispatch 只等专家的输入梯度，并在该 micro-batch 自己那份延迟的专家权重梯度之下跑（`model_chunk_schedule_plan.py#L266-L268`），而另外三次 All-to-All 则跑在另一个 micro-batch 的计算之下。在我们只开了重叠开关的运行里，重叠把双节点吞吐提高了最多 35%。它也要花内存：rank 0 的 reserved 多了 5 到 10 GB，而峰值 allocated 几乎没动。

```
# megatron/core/pipeline_parallel/combined_1f1b.py#L54-L67（一个前向并一个反向）
"""Scheduler for 1f1b with no pipelining.

This function schedules micro-batches in a way that the forward pass of Transformer layers
for one micro-batch runs in parallel with the backward pass of another.
Each layer's forward and backward operations are co-scheduled to maximize the overlap of
their computations and communications.
EP A2A in forward step is hidden by the attention/mlp computation in the backward step,
and vice versa.
Assuming we have 4 microbatches, the schedule is as follows:
Phases 0: 1st microbatch forward
Phases 1: 1st microbatch backward + 2nd microbatch forward
Phases 2: 2nd microbatch backward + 3rd microbatch forward
Phases 3: 3rd microbatch backward + 4th microbatch forward
Phases 4: 4th microbatch backward
"""
```

打开重叠前后都用 `--log-throughput` 记每 GPU TFLOP/s，只有步变快才留下它。然后再看 trace，确认哪些 All-to-All 真的与计算重叠了。

什么时候用它：All-to-All 裸露、且内存允许时，加上 `--overlap-moe-expert-parallel-comm --delay-wgrad-compute`（Megatron MoE 指南推荐的那一对，见 `README.md#L434`），而不是 `--moe-shared-expert-overlap`。`--delay-wgrad-compute` 只在重叠打开、且用 Transformer Engine 层时才能跑（`transformer_config.py#L3631-L3635`、`arguments.py#L1956-L1958`）。用流水线并行时，这个重叠还需要虚拟流水 stage（`transformer_config.py#L3565-L3599`）。

重叠赢回时间但花内存。怎么把一次运行的内存看住、又不把这点时间还回去？

内存（Memory）

MoE 通过两处改变内存：权重和激活。权重装着所有专家，尽管每个 token 只用到其中 k 个；专家并行把这些专家摊开，每张 GPU 只留自己那份。而激活（一层为反向保留的中间结果）随 top-k 增长：dispatcher 对每个被选中的专家各拷一份 token 隐状态（`token_dispatcher.py#L559`），每份拷贝各自保留自己的专家激活，于是「专家收到更多 token」的那张 GPU 也存着更多。

常见的答案是激活重算：前向之后把激活丢掉，反向需要时再算一遍。Ultra-Scale Playbook 讲了它的两种经典形式——整层全量重算，和只重算注意力。全量重算把每层前向跑两遍，在 micro-batch 2 下让我们损失 24% 吞吐；而 3.7 重叠 要求把它关掉（`transformer_config.py#L3579-L3581`）。

更细的做法从「一层里每个模块为反向存了什么」出发。有些张量重建很便宜，比如归一化与专家激活函数的输出，几次逐元素运算就够；另一些又大又贵：进注意力的 query/key/value，以及进专家第一层矩阵乘的、每个 token 的 k 份拷贝。Megatron 区别对待这两类，下图标出它的每个选项作用在哪个被保存的张量上。

> 图 3.7-e 一层里被保存的张量，以及重算（蓝）与卸载（描边）选项分别作用在哪（原站为交互图，此处从略）

蓝色的标签是重算选项。Megatron 的指南重算归一化和专家激活（`--recompute-granularity selective --recompute-modules layernorm moe_act`）：把它们的廉价输出丢掉，在真正要读它们的那次反向之前重建（`random.py#L1042-L1064`）。描边的标签属于细粒度激活卸载，指南卸载注意力的输入与输出、以及专家的第一个输入（`--fine-grained-activation-offloading --offload-modules core_attn attn_proj expert_fc1`）。

卸载是在后续模块计算时，用一条单独的 CUDA stream 把这些大模块的输入拷到 pinned CPU 内存（`fine_grained_activation_offload.py#L1262-L1269`）。反向每做完一个被卸载的模块，就在第二条 stream 上开始拷回下一个模块的张量，于是它们在需要之前就已经在 GPU 上（`#L1332-L1342`）。重算付的是算力，卸载付的是主机与 GPU 之间的带宽，而这些拷贝被藏在计算后面。下图让三层走一遍前向与反向，分别保留全部保存的输入、全量重算、以及那种拆分，并追踪每种设置留在 GPU 上的保存输入。

> 图 3.7-f 三层模型里三种内存策略的峰值与保存量（原站为交互图，此处从略）

只有三层、一个 micro-batch、时长也是示意的：全量重算峰值最低但多一遍前向；细粒度拆分留得更多，却只多几次小重算，因为它的拷贝跑在计算之下。最后一层把它的大输入留在 GPU 上，因为它的反向在前向一结束就开始了，重新加载会把它卡住（`fine_grained_activation_offload.py#L630-L637`）。把模拟里的主机链路调慢，拷到 CPU 的拷贝就结束得晚，峰值随之上升；再慢一些，反向就要等它的重新加载。

```
# megatron/core/transformer/moe/experts.py#L319-L322（moe_act 需要 selective）
self.activation_recompute = (
    self.config.recompute_granularity == 'selective'
    and "moe_act" in self.config.recompute_modules
)
```

```
# megatron/core/transformer/moe/experts.py#L1130-L1141（重算激活，丢掉输出）
if self.activation_recompute:
    self.activation_checkpoint = tensor_parallel.CheckpointWithoutOutput()
    with moe_act_manager as fc1_output:
        bias_act_output = self.activation_checkpoint.checkpoint(
            bias_act_func, fc1_output, bias_parallel, permuted_probs
        )
else:
    with moe_act_manager as fc1_output:
        bias_act_output = bias_act_func(fc1_output, bias_parallel, permuted_probs)
output, output_bias = apply_module(self.linear_fc2)(bias_act_output, tokens_per_expert)
if self.activation_recompute:
    self.activation_checkpoint.discard_output_and_register_recompute(output)
```

NVIDIA 加这个功能时量过这笔交换：在 H100 上，一个 14 层、用 DeepSeek-V3 层形状的模型，卸载它的归一化输入和两个专家输入后，峰值 reserved 少 18%，吞吐损失 2%。

卸载只在它的拷贝还藏得住的时候省内存，而有三个设置决定它搬多少。`--activation-offload-fraction` 只按前向顺序卸载合格组的前一部分；`--min-offloaded-tensor-size` 跳过小张量，默认一百万个元素以下。靠后的流水 stage 同时进行的 micro-batch 更少、需要的内存也更少，所以 `--delta-offload-bytes-across-pp-ranks` 让每个 stage 比前一个少卸载一些。

重算与卸载缩小的是「每层保存的激活」。而一层也可能保留远超其激活所需的内存，这发生在你用 CUDA graphs 把这一步录下来之后：它重放录好的 kernel 序列、不要 CPU，于是每个 buffer 的大小必须事先定死。在不丢 token 的 MoE 里，一个 rank 收到的 token 数逐步变化，所以每层只能按你允许的最大负载预留专家 buffer：用 `--moe-expert-rank-capacity-factor` 配合 HybridEP 或 NCCL EP 后端时，这个上限就是「系数 × 完全均衡时的负载」。每层会一直占着这份预留直到反向跑完，而 NVIDIA 的 Megatron Core MoE 报告 指出，一层真正为反向留着的激活往往比它小十倍以上。

Megatron 的实验性分页暂存 `--moe-paged-stash` 把「一层算在多大的空间里」与「它存多大的空间」分开。所有层都在一块按预留大小分配的共享 buffer 里算，每层前向之后，另一条 stream 只把真实的 token 拷进小的定长页——和卸载把张量拷到 CPU 很像。它需要 `--use-transformer-engine-op-fuser` 和 FP8 或 NVFP4 专家，而且因为它自己已经持有专家激活，Megatron 拒绝把它与卸载 `expert_fc1` 或 `moe_act` 组合。

每项改动前后都比峰值内存与步时，因为内存不够的运行会在最忙那个 rank 上 OOM，而「拷贝比计算还久」的卸载会表现为反向变长。所以要看的峰值是最忙那个 rank 的峰值。Megatron 会打印数据并行 rank 0 的峰值（`common_utils.py#L407-L427`），并通过 2.4 的启动器把最后一个 rank 的峰值发到 Trackio。要找最忙的 rank，就在每个日志间隔取各 rank `torch.cuda.max_memory_allocated()` 的最大值，或者用 `--record-memory-history` 记逐 rank 快照。

什么时候用它：重叠放不进内存时，重算廉价模块（`--recompute-granularity selective --recompute-modules layernorm moe_act`）并卸载大输入（`--fine-grained-activation-offloading --offload-modules core_attn attn_proj expert_fc1`），而不是全量重算每一层——后者会挡掉重叠。步时变长就调低 `--activation-offload-fraction`。重算 `layernorm` 或 `moe_act` 不能与 FP8 的 delayed 配法一起跑

（transformer_config.py#L2422-L2433）；在 ncclep 后端下，重叠也不能与卸载 expert_fc1 一起跑（#L3606-L3618）。

到目前为止，你一直在不动路由器选择的前提下调系统。那如果让路由器把网络也考虑进去、代价付在训练上呢？

组限路由（Group-limited routing）

一个 token 的 top-k 专家可能落在专家并行组的任何位置。一旦这个组跨了好几个节点，一个 token 就可能需要多个节点上的专家，它的拷贝在 dispatch 与 combine 时都会穿过更慢的节点间网络。普通 top-k 路由不管专家住在哪，所以没什么能阻止一个 token 的专家散在很多节点上。

DeepSeek-V3 用组限路由补上这个约束：在挑之前先限定能看哪里，「以限制训练期间的通信开销」。它的专家放在 8 个节点上，对每个 token 先把每个节点上该 token 的 top-2 专家分数（含 3.3 的均衡偏置）加起来给节点打分，保留最好的 4 个节点，再在这些节点上的专家里取 top-k，于是每个 token 最多到达 4 个节点。把「设备」换成「节点」，下图把同一条规则画在标准 top-k 和 MoGE 旁边。MoGE（Mixture of Grouped Experts）是华为 Pangu Pro MoE 用的路由，它把专家平均分成每个设备一组，从每组里各取同样多的专家。用 8 张 GPU 做专家并行训练出来的 Intern-S1-Pro，就是用同样方式从 8 个组里各取 1 个。

> 图 3.7-g 标准 top-k / 组限路由 / MoGE 的专家选择对比（原站为交互图，此处从略）

逐设备比较：组限路由给每个 token 能到达的设备数封顶，而 MoGE 让每台设备负载均匀、把每个 token 都发给所有设备。

```
# megatron/core/transformer/moe/moe_utils.py#L713-L718（给组打分，保留最好的几个）
group_scores = (
    scores.view(num_tokens, num_groups, -1).topk(topk // group_topk, dim=-1)[0].sum(dim=-1)
)
group_idx = torch.topk(group_scores, k=group_topk, dim=-1, sorted=False)[1]
group_mask = torch.zeros_like(group_scores)
group_mask.scatter_(1, group_idx, 1)
```

```
# megatron/core/transformer/moe/moe_utils.py#L721-L728（在保留的组内取 top-k）
score_mask = (
    group_mask.unsqueeze(-1)
    .expand(num_tokens, num_groups, num_experts // num_groups)
    .reshape(num_tokens, -1)
)

masked_scores = scores.masked_fill(~score_mask.bool(), float('-inf'))
probs, top_indices = torch.topk(masked_scores, k=topk, dim=-1)
```

这个上限省下多少流量，取决于 dispatcher。Megatron 的 alltoall dispatcher 每个专家送一份 token 拷贝；而配 DeepEP 的 flex dispatcher（它的 kernel 跟着 DeepSeek-V3 的组限路由走）

每个节点送一份、再在节点内用 NVLink 转发。对我们那个 7B-A1B 模型的一层（64 专家、top-8、hidden 2048、bf16），摊在 2 到 8 个节点上，下图统计了两种 dispatcher 下、有无这个上限时穿网走掉的字节。

> 图 3.7-h 穿网字节数：`alltoall` vs DeepEP，有无节点上限（原站为交互图，此处从略）

用 `alltoall` 时，上限只是把 token 的拷贝挪到更少的节点上，因为每个被选中且不在本节点的专家仍然各拿一份；图里的路由器随机给专家打分、不管 token 落在哪个节点，于是平均下来穿网字节和不加上限一样多。用 DeepEP 时，token 到达的每个节点花一份拷贝，所以给节点封顶就等于给字节封顶。到达的节点更少，也意味着每个 token 能选的专家更少——这笔代价付在质量上而不是时间上，不过很小：Xue 等人用组限训练一个 13B 模型时，loss 只高了那么一点。

近期 MoE 倾向于去掉这个上限。V3 用了它的 DeepSeek 在 V4 里去掉了；GLM、Kimi、Nemotron、MiMo、Hy4 也都让每个 token 到达任何专家。InclusionAI 的 Ling 与 Ring 仍然保持 DeepSeek-V3 的那种形式（8 组、每 token 留 4 组），SK Telecom 的 A.X-K2 保留了一个更宽松的版本。

token 扇出到很多节点时这一步就慢，所以盯两个数。第一是每 token 到达的节点数，从路由 trace 与「哪个 rank 持有哪个专家」的映射离线算（A.1）。第二是每个 EP rank 收到的 token 数，用 `--save-tokens-per-expert-interval` 写出的计数求和（A.1）。

先试上面那些建议，因为它们不改训练；只有当跨节点 All-to-All 仍然占很大份额时才动这个上限：上限会改变 token 能挑哪些专家，也就改变了模型学到什么。它只在专家并行跨多个节点、且 dispatcher 按节点只送一份时才有用，也就是配 DeepEP 的 `flex` dispatcher。那时把 `--moe-router-num-groups` 设成节点数、`--moe-router-group-topk` 设成每个 token 最多能到几个节点，并配一个不是 `quantile_balancing` 的均衡器（它不能与上限一起跑）。否则就别开：单节点上它省不了流量，用 `alltoall` 时平均也省不了。

节点上限限制的是 token 的拷贝去哪，可路由偏爱的专家仍然照单全收所有送来的 token，持有它的那个 rank 就是最后一个跑完的。你该把这个专家的多余 token 挡在门外，还是把它拷到清闲的 rank 上？两种做法各付什么代价？

容量与副本（Capacity and replicas）

持有热门专家的 rank 最后跑完自己那份，其他 rank 在下一个同步点等它。默认情况下 Megatron 接受这个等待、保留每一个 token：不设 `--moe-expert-capacity-factor` 时路由是不丢 token 的（`transformer_config.py#L1009-L1011`），一个专家会收下路由器发给它的全部 token。

容量系数给这个数封顶。每个专家只保留概率最高的那些 token、直到容量上限（`transformer_config.py#L1024-L1028`），超出的 token 跳过这个专家。这换来一些速度（没有专家能以超过容量的 token 数拖住这一步），也封住内存（没有专家会为超过容量的 token 存激活）。要静态形状，再加上 `--moe-pad-expert-input-to-capacity`。

容量系数省出的内存还能换成更好的并行布局。NVIDIA 的 Vavre 等人把一个 Llama 3-8B upcycle 成 8 专家、top-2 路由的 MoE，在 128 张 GPU 上，容量系数 1 且不用张量并行跑到每 GPU 463 TFLOP/s，而张量并行 2 的不丢 token 训练是 392。作者把这份收益归给更小的显存占用——它让他们能用更少的模型并行。

另一个答案是保留所有 token，转而复制热门专家。下图给出处理热门专家的三种方式：不丢 token 路由、容量系数、以及按 ECHO 的方式放置的热门专家副本（ECHO 出自 Megatron Core 团队那份报告，在训练中复制热门专家）。

> 图 3.7-i 一个偏斜的玩具 batch：不丢 token / 容量系数 / 在空 rank 上放副本（原站为交互图，此处从略）

在一个 8 专家、4 rank 的偏斜玩具 batch 上，容量系数丢掉超出一个专家容量的选择（主要是热门专家的），并且按到达顺序丢——而 Megatron 默认会保留概率最高的那些。每个被丢掉的选择都跳过它的专家，于是画的这一步变短、模型输出也变了。把副本放在最闲的 rank 上，能让玩具里最忙的 rank 接近均值，且什么都不丢。

```
# megatron/core/transformer/moe/router.py#L1029-L1038（在 top-k 之后丢）
# Apply token dropping to probs and routing_map.
if self.config.moe_expert_capacity_factor is not None:
    probs, routing_map = apply_router_token_dropping(
        probs,
        routing_map,
        router_topk=self.topk,
        capacity_factor=self.config.moe_expert_capacity_factor,
        drop_policy=self.config.moe_token_drop_policy,
        pad_to_capacity=self.config.moe_pad_expert_input_to_capacity,
    )
```

```
# megatron/core/transformer/moe/moe_utils.py#L1079-L1112（每专家容量，保留最高概率）
# Calculate expert capacity
expert_capacity = get_capacity(
    num_tokens=num_tokens * router_topk,
    num_experts=num_experts,
    capacity_factor=capacity_factor,
)

# Create capacity mask based on drop policy
if expert_capacity > num_tokens:
    # No need to drop tokens if capacity exceeds the number of tokens
    capacity_mask = torch.ones_like(routing_probs).bool()
else:
    if drop_policy == "probs":
        _, capacity_indices = torch.topk(routing_probs, k=expert_capacity, dim=0, sorted=False)
        capacity_mask = torch.zeros_like(routing_probs).scatter(0, capacity_indices, 1).bool()
    elif drop_policy == "position":
        _, capacity_indices = torch.topk(
            routing_map.int(), k=expert_capacity, dim=0, sorted=False
        )
        capacity_mask = torch.zeros_like(routing_probs).scatter(0, capacity_indices, 1).bool()
    else:
        raise ValueError(f"Invalid drop_policy: {drop_policy}")

# Apply capacity constraints
if pad_to_capacity:
    final_map = capacity_mask
    final_probs = routing_probs * final_map
else:
    # Get exceed mask and maskout exceeded probs and indices
    final_map = torch.logical_and(routing_map, capacity_mask)
```

真实运行丢的比玩具少：容量系数 1.5 时，华为 Pangu 团队在一个 20B MoE 上看到约 6% 的 token 被丢，在他们的 718B 上约 8%。副本则能把各 rank 校得几乎完全一致。Wei 等人的 UltraEP 为每个 micro-batch、每一层重新规划副本，把三次训练（DeepSeek-V3、Qwen3-235B、GLM-4.5-Air）里最忙 rank 的负载从普通 Megatron-LM 的 1.30 到 2.86 倍均值，压到 1.01 到 1.03 倍。

副本在训练里比在服务里贵。每个副本前向要把专家权发送到它的 rank，反向要把梯度归约回原处，所以 ECHO 的规划器尽量少复制：它统计每个专家超出「每 rank 均值负载」的 token，找到清闲 rank 上的空位，把前者塞进后者，算完就丢掉副本。均衡的 rank 有两份回报：等得更少、buffer 更小——因为 CUDA graphs 会按最坏情况定它静态 buffer 的大小，也正是 3.7 内存 里分页暂存要绕开的那份最坏情况。

服务端有同样的问题但没梯度，所以它的副本留得比 ECHO 久得多（后者每次前向都重新规划）。DeepSeek-V3 按在线服务的负载、以约 10 分钟为间隔重挑它的冗余专家。DeepSeek 用来服务 V4.1-Flash 的引擎 DeepInfer 给每张 GPU 留出空槽放最忙专家的副本，按最近一个窗口统计每个专家的负载，prefill 每 1,000 步、decode 每 10,000 步重新放置一次，于是最慢的 GPU 保持在均值的 1.09 倍以内——就是 3.3 节引的那个数。

丢 token 会掉质量吗？公开的比较说会掉一些 loss。华为 Pangu 团队用容量系数 1.5 和不丢 token 各训了一个 20B MoE，不丢的那次全程 loss 更低，且差距随专家数增多而变大。Gale 等人做 MegaBlocks 时也一样：不丢任何 token 时 loss 最低。

下游基准可能掩盖这个差距：Vavre 等人 upcycle 的 Llama 3 MoE 在他们试的每个容量系数下、以及不丢 token 时，MMLU 都在一分以内。丢掉的 token 落在哪也很要紧：Meta 的 MobileMoE 带着容量系数预训练，微调时切回不丢 token——因为在指令-回答对里丢 token 会切掉内容。

带容量上限训练的模型，不必带着上限服务。Zoph 等人用比训练时更高的容量系数评估他们的 ST-MoE 时，比较里每个模型分数都更好。DeepSeek-V2 就是为这个切换设计的：训练时在每个设备上丢 token，但约 10% 的训练序列从不丢，于是推理时它可以自己决定丢不丢，还照样匹配它训练时的行为。

开了容量系数，token 是被静默丢掉的，所以要把「丢得多频繁」和验证 loss 一起看。`--moe-routing-trace-path` 写出的路由 trace 会跳过每一条「token 被丢」的记录（`router_trace.py#L353-L360`）。要数丢了多少，就在路由器上挂一个前向 hook，逐层算被丢的 token-专家分配占比：`1 - kept / (tokens * topk)`，其中 `kept` 是路由图里剩下的分配数。

什么时候用它：默认保持不丢 token。当一个热门专家拖住这一步、而清闲的 rank 还有地儿时，给它放一个副本就能把 rank 校平、又不动 loss（不过 ECHO 的复制目前还是一个未合并的 PR）。内存紧张、又能接受略高的 loss 时用容量系数；要静态形状还要加 `--moe-pad-expert-input-to-capacity`。微调时切回不丢 token，因为那时丢掉的 token 落在指令-回答对里。

不管丢不丢，专家留下的每个 token 都要过它的 GEMM，而到目前为止这些 GEMM 都跑在 bf16 里。换成 FP8 这样的更低精度，能让这份活更便宜、存的激活更小，又不伤 loss 吗？

精度（Precision）

MoE 层的大部分计算在矩阵乘里，而大部分激活内存在这些乘法为反向保存的输入里。相对 bf16，FP8 把两者的字节都减半，而且它的 Tensor Core 操作在 Hopper 与 Blackwell 上更快。这份收益能落到步时上多少，取决于乘法占这一步多大比例、以及量化那些张量要付多少——对大规模 MoE 训练，Megatron Core 团队报告端到端约 10% 到 25%。

不是每个操作都能吃这个低精度。下图给出 FP8 在一层里（前向与反向）跑到哪，沿用 DeepSeek-V3 训练时的切分：每个线性层的前向乘法与两次反向乘法都跑 FP8，而 embedding、输出头、归一化、注意力核心与路由器保持 BF16 或 FP32。

> 图 3.7-j 一层里 FP8 与 BF16/FP32 的分界（原站为交互图，此处从略）

省下的内存来自反向。权重梯度那次乘法要读前向保存的输入，把它留在 FP8 就把这份内存减半——对 DeepSeek-V3 来说约合每 GPU 16 GB 激活。而主权重、权重梯度和优化器状态保持更高精度。

路由器是要保护的部分。它的分数决定每个 token 的专家，那里的量化噪声会改变选择、让训练不稳，这也是 Megatron Core 报告把它留在 FP32 的原因。在 Megatron 里路由器不是 Transformer Engine 层，所以 FP8 根本到不了它，而 `--moe-router-dtype fp32` 设定它数学运算的精度

（router.py#L125-L130，3.1 精度）。量化后的分组 GEMM 需要每个专家的 token 数补齐到对齐尺寸，这由 --moe-router-padding-for-quantization 提供（transformer_config.py#L851-L854）。

「配法（recipe）」决定缩放系数跟数据跟得多细。每个张量一个缩放系数时，最大的离群值决定其余所有值的范围，所以 DeepSeek-V3 与 Ling 2.0 按 1×128 元素为一格缩放激活与梯度、按 128×128 为一块缩放权重。Megatron Core 报告推荐 Hopper 上用这种分块配法、Blackwell 上用 MXFP8（每 32 个元素一个缩放系数）。用 --fp8-format 与 --fp8-recipe 选，分块配法需要 Transformer Engine 2.3 或更新（fp8_utils.py#L799）。

FP8 也能过网。DeepSeek-V3 在 dispatch 前把 token 量化成 FP8——反正专家的第一层就是以 FP8 读它们；「MoE 下投影之前对激活梯度用了相似策略」，而「forward 与 backward 的 combine 组件」仍留在 BF16。在 Megatron 里，--moe-dispatch-fwd-dtype mxfp8 与 --moe-combine-bwd-dtype mxfp8 让 dispatch 载荷与经 combine 回传的梯度走 MXFP8。两者都需要 flex dispatcher 的 ncclep 后端（transformer_config.py#L2150-L2158），而 MXFP8 本身要 Blackwell 级 GPU（#L656-L661）。

长跑报告说 loss 撑住了。DeepSeek 用 FP8 训了一个 16B 和一个 230B 的 MoE，各约一万亿 token，二者相对 BF16 基线的误差都在 0.25% 以内。Ling-1T 在 900B token 上保持在同一量级（平均约 0.1%），同时 FP8 把它的 MFU 提高了约 15%。

再往下一档是 4 bit，头几个大 MoE 已经在那儿预训练了。NVIDIA 用 NVFP4（一种 4 位浮点格式：每 16 个元素一块配一个 FP8 缩放系数，再加一个逐张量的 FP32 缩放系数。）在 25T token 上训了 Nemotron 3 Super（120B 参数、12B 激活），大多数线性层的三次乘法都跑在里面。例外标出了 4 bit 哪里不够：网络最后 15%、注意力投影、embedding 与多 token 预测层留在 BF16，Mamba 的输出投影跑 MXFP8（它在 NVFP4 下下溢了）。这套配法还按二维块缩放权重（于是前向反向看到同一份量化权重）、对权重梯度乘法的输入做随机 Hadamard 变换、并对梯度做随机舍入。Nemotron 3 Ultra（550B、55B 激活）用同一套配法跟了 20T token；当 NVIDIA 从其 checkpoint 分叉出 BF16 运行对比时，与它们的 loss 差距平均在 0.4% 以内。

FP4 把保存的输入压到 BF16 的四分之一（FP8 是一半），而原生 FP4 乘法要 Blackwell 级 GPU。Hopper 没有 4 位 Tensor Core 指令，但浙江实验室的 Zhang 等人用一个 671B MoE 做了软件模拟的 MXFP4：把激活与 dispatch 流量存成 MXFP4，而乘法仍走 FP8，峰值激活内存降了 14.8%，收敛与一个强的 FP8 基线持平。省下来的内存让它少重算，比「装得下的最好 FP8 配置」还快 12.5%。

一次 FP8 运行的 loss 应该全程贴近 BF16，所以用相对差值与同配置的 BF16 运行比。DeepSeek 的 16B 与 230B 运行、以及 Ling-1T 都在 0.25% 以内，所以远超这个数、或者一直在拉大的差距，就指向配法出了问题。用 NVFP4 时，还要数专家层里恰好为零的权重梯度元素：在 Nemotron 3 Super 里它们到预训练结束时涨到全部参数的 7%（某些专家里低范数的通道缩到了零），而在 Nemotron 3 Nano 上做的一个更小的消融里，切回 BF16 就让这个计数回到基线。

什么时候用它：线性层占这一步大头时用 FP8——Hopper 上用分块配法、Blackwell 上用 MXFP8，并把路由器、归一化与注意力核心留在更高精度。在大 MoE 上，预期吞吐多约 10% 到 25%，loss 与

BF16 相差不到零点几个百分点。在 Blackwell 上，NVFP4 是再下一档，用 Nemotron 3 那套配法把最后几层、注意力投影和 embedding 留在 BF16。

该盯什么

部位	症状	该盯的信号
剖析	「慢」：这一步比应有的更久	先看步时或 TFLOP/s；再从 trace 看 All-to-All 在步里留下多少裸露
互联	多节点上「慢」：NCCL 退回 TCP socket 时，我们的 EP16 代理慢约 25 倍，42.0 vs 1.7 TFLOP/s	`NCCL_DEBUG=INFO` 下看 "Using network" 是否是你要的传输层：InfiniBand 看 IB，EFA 上的 OFI 插件看 Libfabric；Socket 表示回退了。然后用 nccl-tests 比较节点内与跨节点的 alltoall 总线带宽（2.1 节），以及节点内 vs 跨节点 EP 的 TFLOP/s
重叠	「慢」：All-to-All 没藏在计算后面	打开前后每 GPU TFLOP/s（`--log-throughput`）。裸露份额只用来解释变化
内存	「OOM」：某个 rank 爆了。「慢」：重算、或比计算还久的卸载重新加载，增加了步时	每次改动后比较步时与峰值内存。看最忙 rank 的峰值：各 rank `torch.cuda.max_memory_allocated()` 的最大值，或逐 rank `--record-memory-history` 快照。反向变长可能暴露比计算更久的卸载重载
组限路由	「慢」：每个 token 扇出到多个节点	每 token 到达的节点数（路由 trace 与专家到 rank 的映射），以及每个 EP rank 收到的 token 数（token 计数），A.1 节
容量与副本	「学不好」：token 被静默丢掉。「OOM」：某个 rank 爆了	逐层被丢的 token-专家分配占比 `1 - kept / (tokens * topk)`（路由器前向 hook），与验证 loss 并看。路由 trace 会跳过被丢的记录
精度	「慢」：相对 bf16 没提速。「学不好」：loss 比 bf16 高	与同配置 BF16 运行比全程相对 loss 差；DeepSeek 的 16B、230B 与 Ling-1T 都在 0.25% 以下。NVFP4 下还要数恰好为零的专家权重梯度元素：Nemotron 3 Super 里达到全部参数的 7%

到这里你已经拿到了一个 MoE 层的每个部件：从挑专家的路由器，到它们计算时用的精度。等训练结束、模型要被服务出去的时候，这些选择里还有哪些仍然要紧？

导航：[[2026-11-moe-handbook|全书目录]] · ← 第 3 章（下）块布局与训练 · 第 4 章 从训练到服务 →

第 4 章 从训练到推理服务

模型只训一次，却要在被别人用的整段时间里持续提供服务。到这为止，这本手册讲的是前半截：怎么把一个 MoE 训得快、训得好。这一章讲后半截。

推理服务用另一套数来评判模型。训练看每步耗时和 loss；服务看每次请求多快拿到答案、以及回答完这些请求要占多少张卡。模型是同一个，目标却换了，于是那些在训练时正确的选择，等训练结束就值得再看一眼。

MoE 需要重看的东西比稠密模型多。每个 token 只过少数几个专家，但每个专家通常都得留在显存里，等着某个 token 选中它——于是这个模型算力上便宜、显存上昂贵。一旦进入逐 token 解码，每个专家每步也只见到几个 token，于是「把专家摊到多少张卡上」本身就成了一个服务侧决策。注意力和专家（在目前这种布局里共用同一张卡）对硬件的需求也不一样。

本章逐条处理这些差异。先看 k，也就是路由器为每个 token 选的专家个数。接着看专家占多少显存、摊到多少张卡上、以及注意力和专家该分别在哪儿跑。最后是 RL 后训练：服务引擎写出训练器要学的样本，而两边可能对「每个 token 走了哪些专家」这件事各执一词。

第一个改动，是你在服务时设的一个数字。如果你用与训练时不同的专家个数来服务每个 token，它的输出会怎样？

用不同的 k 服务

训练定死了每个 token 用几个专家，但没人拦着你少用。从某个 token 的 top-k 里每砍掉一个被路由到的专家，就省一份算力；而每个专家仍然常驻显存，等着选中它的那些 token——所以显存一点没省。难处在于路由权重。做重归一化的路由器（3.1 节）会拿被选中专家的概率除以它们的和，于是专家分支输出的量级（增益）是按训练时的 k 定下来的。当你少服务几个专家时，幸存者要瓜分同一个总量，每个分到的权重都比训练时更大。所以更小的 k 同时改了两件事：哪些专家在跑，以及每个专家贡献多少。

Chen 与 Yao 用一个整数把这两个效应拆开。他们保留 top-k1 的专家，但把权重除以 top-k2 的 softmax 质量，k2 在服务时指定、不需要训练。让 k2 = k1，就是对幸存者做常规重归一化，LExI（一个为每层挑不同 k 的方法）也是这么做的；让 k2 等于专家总数，等于关掉重归一化；让它等于训练时的 k，则把增益锚回模型训练时见过的那个值。

下面这张图把一个模拟 token 送进一个形状模仿 Qwen3.6-35B-A3B 路由器的 softmax——256 个被路由专家、按 top-8 训练——然后用更小的 k 服务它，每种分母一行。跟着中间那栏看 k 下降时的变化：除以幸存者，每个被服务的专家权重比 top-8 时更大；除以训练时的 top 8，它保持训练时的权重；除以 top 16，它变得更小。右栏给出每一行实测的 MMLU 变化。

那个路由器很「平」：在 WikiText 上它的 top 8 专家平均只占 softmax 质量的 0.182，所以分母一动，增益就动得很远。以训练时 top-8 的 81.65% MMLU 为基准，top-4 除以四个幸存者的得分低 4.65 分，除以 top-8 质量低 3.10 分；除以 top-16 质量只低 0.35 分，这个降幅不显著（p =

0.66）。完全不重归一化的话，top-6 掉 27.45 分。作为回报，top-4 把每 token 激活的被路由参数从 1.006B 砍到 0.503B，相当于省掉每 token 全部参数的 17%。

分母有多要紧，取决于路由器有多平。Gemma-4-26B-A4B 的 128 个专家里 top 8 占了 0.302 的质量，在 top-6 上去掉重归一化，困惑度只涨 0.7%，而 Qwen 那个模型上是 356%。两个结果都来自 softmax 路由器。

服务时用比训练更多的专家，每 token 算力更贵，而质量未必跟着涨。Gu 等人在 OLMoE-1B-7B（按 top-8 训练）上做微调实验，服务 top-16 让九项 benchmark 的平均从 41.35 变成 41.09，落在他们报告的误差棒内。多出来的专家组合是模型从没一起训过的，所以他们的实验里，只在服务时调大 k 并没有帮上忙。

Megatron 就按 `--moe-router-topk` 服务，权重跟着你训出来的路由器走。默认情况下 softmax 路由器在 top-k logits 上做 softmax，于是更小的 k 就是在幸存者上重归一化，即 Chen 与 Yao 的 k2 = k1。加 `--moe-router-pre-softmax`，权重来自完整 softmax，永不重归一化。sigmoid 路由器在 k > 1 时除以自己的 top-k 和，任何路由器都会乘上 `--moe-router-topk-scaling-factor`（如果设了）。transformers 的 Qwen3-MoE 里，服务的 k 是 `num_experts_per_tok`，只有设了 `norm_topk_prob` 才把路由权重除以 top-k 和。想要别的分母，两种路由器都只需要打几行补丁：保留 top k1 的专家，除以 top-k2 的质量。

```
# megatron/core/transformer/moe/moe_utils.py#L913-L947
if score_function == "softmax":
    if use_pre_softmax:
        scores = torch.softmax(logits, dim=-1, dtype=torch.float32)
        ...
    else:
        scores, top_indices = compute_topk(logits, topk, num_groups, group_topk)
        probs = torch.softmax(scores, dim=-1, dtype=torch.float32)
elif score_function in ("sigmoid", "sqrtsoftplus"):
    ...
    probs = scores / (scores.sum(dim=-1, keepdim=True) + 1e-20) if topk > 1 else scores
if scaling_factor:
    probs = probs * scaling_factor
```

> 提案：几行补丁——保留 top k1 的专家、除以 top-k2 的质量，打在 Megatron 里或者 transformers 的路由器里；然后再做下面的服务端 k 扫描。

除了困惑度，还要在有标注的任务上给每个服务的 k 打分，因为两者可能打架：35B 模型在 top-6 时，以训练时的 top-8 质量为分母困惑度最低，而 MMLU 最高的却是以 top-16 质量为分母。

什么时候用它：当你需要降低每 token 的被路由专家算力、又能把所有专家留在显存里时——服务一个更小的 k，并把幸存者的权重除以一个比它们自身更大的质量。在 Qwen3.6-35B-A3B 上取 top-4、除以 top-16 质量，MMLU 与训练模型的差距保持在 0.35 分内。分母一定要在你自己模型上验证，因为它有多要紧取决于你的路由器有多平。

砍 k 省的是算力，每个专家仍然占着显存。当限制你服务的是显存而不是算力时，怎么让专家少占点？

低位宽专家

训练一结束，权重就不动了，描述它们在服务时成本的就两个数：在显存里占多少字节，以及每解码一个 token 要读多少字节。MoE 里被路由的专家握着大部分参数，所以它们的精度决定了模型的大半体积；只把它们存成 4 bit、其余保持更宽的格式，就能拿到大部分的节省。

有些前沿 MoE 已经是这么发的。OpenAI 用 MXFP4 量化 gpt-oss 的 MoE 权重做后训练，折合每参数 4.25 bit。这些权重要占全部参数的 90% 以上，于是 120b 的模型能塞进一张 80 GB 卡。Moonshot 在 SFT 与 RL 阶段把 Kimi K3 的被路由专家量化成 MXFP4，其余保持高精度。DeepSeek-V4 在后训练里对 MoE 专家权重做 MXFP4 量化感知训练（前向时把权重圆整到目标格式），GLM 团队则在 GLM-5 的 SFT 阶段用 INT4 量化感知训练。这四家的量化都发生在预训练之后，所以你预训练的配置不用动。

下面这张图比较 gpt-oss-20b 的三种存法，以逐 token 解码为前提：全部 bf16、只有被路由专家 4 bit、以及所有线性层 4 bit。每根柱是模型在显存里的大小，蓝色块是一个 token 读到的权重。对比第一行到第二行，看模型缩小了多少、而一个 token 的读取缩小了多少；再看第三行把注意力也放进 4 bit 后多出了什么。

把专家存成 4 bit，gpt-oss-20b 从 39.0 GiB 缩到 12.8 GiB，小三倍；但每解码一个 token 读到的权重字节只从 6.72 GiB 降到 3.45 GiB，大约一半。batch-1 解码时，一个 token 每层只读 32 个专家里的 4 个，却要读全部注意力和 unembedding，所以专家一小，那些 bf16 权重就占了它读取量的三分之二。把注意力也放进 4 bit，读取再降四分之一，而显存只多省 0.9 GiB——因为注意力本来就没几个参数。

公开的 checkpoint 两种范围都有。ZAYA1-8B 的 MXFP4 版本只量化被路由专家，Kimi K2 Thinking 和 Qwen 的 GPTQ-Int4 版 Qwen3.5 也是（用 int4 而非 MXFP4）。Qwen 的 GPTQ-Int4 版 Qwen3 30B MoE 与腾讯的 Hunyuan-A13B 把注意力也放进 int4，只留路由器、embedding 和输出头不量化。

Megatron 通过 ModelOpt 导出：`examples/post_training/modelopt/quantize.py` 用 `--export-quant-cfg` 接一个 ModelOpt 配置名，比如 `FP8_DEFAULT_CFG` 做 FP8 导出，选项里还有 `--weight-only` 和 `--sync-expert-weight-amax`。要在 4 bit 下训练，设 `--fp4-format e2m1`，在 Blackwell 上走 Transformer Engine 的 NVFP4，需要 Transformer Engine 2.7 以上，且不能与 `--fp8-format` 同用。

```
# megatron/post_training/arguments.py#L56-L61（--export-quant-cfg）
group.add_argument("--export-quant-cfg", type=str, default=None,
    help="Specify a quantization config from mtq.config.choices.")
# examples/post_training/modelopt/quantize.py#L159-L176（--weight-only / --sync-expert-weight-ama
x）
group.add_argument("--weight-only", action="store_true", help="Disable input quantization.")
group.add_argument("--sync-expert-weight-amax", action="store_true",
    help="Synchronize expert weight amax across experts.")
# megatron/core/transformer/transformer_config.py#L735-L743（--fp4-format）
fp4: Optional[Literal['e2m1']] = field(default=None, metadata={"argparse_meta": {"arg_names": ["--
fp4-format"]}})
```

把每个专家的量化误差对着它的 token 数画出来（token 数用 3.0 节的逐专家统计），这样「因为很少被路由而变差」的专家会自己跳出来。然后拿量化导出的模型和 Megatron 在同一段文本上比 loss。

什么时候用它：当服务受限于显存、或者受限于每 token 要读的字节数时——在后训练里把被路由专家量化到 4 bit，其余保持高精度，像 Kimi K3 那样。

把专家压到更低位宽，缩的是每张卡装的东西，但没回答该把它们摊到多少张卡上。一旦请求是一个 token 一个 token 地来，还该按训练时那样铺开吗？

服务端的专家并行

训练只为一种步挑专家并行度：一大批 token 一起过。服务跑两种步。prefill 步处理 prompt，DeepSeek 的引擎允许一步里每卡最多 32,768 个 token。decode 步只承载在飞请求的下一个（或几个）token，于是每个专家每步只见到很少的 token。把专家摊到更多卡上对后者有帮助：每卡持有更少的专家、每步读更少的权重，而 All-to-All 给每个专家攒出更大的批次。

DeepSeek 在服务 DeepSeek-V4.1-Flash 时，把两个阶段往相反的方向配。prefill 跑 EP2 到 EP8，也就是显存够用的最小规模；decode 跑 EP64 到 EP128，也就是手里显卡允许的最大规模。两个阶段里注意力都做数据并行，每个 EP rank 一个副本，且都不用张量并行或上下文并行。

下面这张图重画了 DeepSeek 自己对 decode 两种设置的比较，每个点对应一种「每卡请求数」。每卡 64 个请求时，EP64 每卡持 7 个被路由专家，每卡每秒约 15,700 token、每用户约 246 token/s；EP8 每卡持 52 个专家，只有约 12,600 与 198，所以 EP64 下两个数都高约 25%。每卡 256 个请求时，两种设置几乎打平，约 26,300 与 24,900 token/s/卡。

宽度是有代价的。DeepSeek 给 prefill 与 decode 的卡数大致相当，所以 decode 开到 EP64 以上时，最小可跑部署就是 64 张卡做 prefill、加 64 张做 decode。同一个 EP 实例的所有卡还构成一个通信域，一张卡挂了整个实例就挂，重启时 DeepSeek 会按还健康的卡重新调整 EP 组的大小。

Megatron 在 coordinator 模式下用专家并行服务（这是默认模式），direct 模式拒绝大于 1 的 EP。专家并行下，只有所有 EP rank 都在 decode 时，一步才会走 CUDA graph：只要有一个 rank 的 batch 里有 prefill，所有 rank 都退回 eager 模式。做 RL 时，`--rl-inference-expert-model-parallel-size` 给 rollout 引擎单独的 EP 大小，它被建成一个独立的推理模型，训练器的权重再 refit 进去。

```
# docs/mcore-inference-user-guide.md#L277-L278（EP 需要 coordinator 模式）
- *Required* for: HTTP serving (`serve`), expert parallelism (`EP > 1`),
  multimodal inputs, streaming, and the lifecycle controls
# megatron/core/inference/batch_dimensions_utils.py#L156-L161（一个 prefill rank 把整个 EP 组拖回 e
ager）
"""Adjust CUDA graph batch dimensions for expert parallelism.
All-reduce-max the token count and non-decode flag across the EP group.
If any rank has a prefill (non-decode) step, all ranks fall back to eager mode.
"""
# megatron/training/config/rl_config.py#L140-L141（rollout 的 EP 大小）
rl_inference_expert_model_parallel_size: int | None = None
```

什么时候用它：当你要的是 decode 速度、也出得起宽部署的卡时——decode 用你的卡允许的最宽专家并行，prefill 用塞得下的最窄规模，像 DeepSeek 对 V4.1-Flash 那样；再给宽 decode 组配冗余专家，让它最慢的那个 rank 贴近均值。

更宽的专家组改变了每张卡干多少专家活，但每张卡仍然把注意力和专家并排跑，哪怕两者对它的要求并不一样。到了推理阶段，共用显卡还是你该选的摆法吗？

注意力与前馈解耦

在共置（colocated）布局里，专家并行组里的每张卡都要跑每层的两半——注意力和专家——但解码时这两半要的资源不同。注意力参数很少，却得为每个在飞请求保留 KV cache；专家握着大部分参数，却不保存任何按请求的状态。于是服务可以把注意力和专家放到不同的卡上，各成一个池，用小批次来回传递，让一个池在算的时候另一个池在算上一批。（Megatron 的 `megatron/core/inference/disaggregation` 沿的是另一条线：把 KV 与 SSM 状态从 prefill 引擎交给 decode 引擎。）MegaScale-Infer 就是这样把每层里的注意力与前馈拆开的，它报告每卡吞吐最高达到此前服务系统的 1.90 倍。

下面这张图左边画 FastAFD 部署 Qwen3-235B-A22B 的方式（28 张注意力卡 + 4 张专家卡），右边画 4 张卡上的共置专家并行。看左边对比两种布局下每张卡装了什么：没有专家的注意力卡能放下更多请求的 cache，而专家卡不保存任何请求的 cache，每卡请求数的柱状把两个池都算进去了。看右边跟踪一个 decode 步，看第二个 micro-batch 如何在第一批去专家那里时把注意力池占住。共置那一行也用两个 micro-batch 掩盖它的 All-to-All，vLLM 把这种做法叫 dual-batch overlap。

这笔收益有多大，取决于你拿什么比。按论文发表时的口径，FastAFD 的布局每卡吞吐是共置 EP4 的 1.41 倍。美团的 Qiu 等人用解析模型复现了这个增益，然后在同一个模型里按同样的 72 卡预算重新给两种布局配了卡，比值掉到 0.99 倍。在他们建模的所有设置里，同构 AFD 只在少数情形下提高了固定预算的吞吐。Step-3 的头条数字——同样 20 token/s 的解码目标下每卡每秒 4,039 token，对 DeepSeek-V3 的 2,324——比较的是两个不同的模型，因此并不能把布局这个因素单独隔出来。

训练在这里决定一件事：专家池要服务的模型有多稀疏。Step-3 把稀疏度算成「激活专家数（含共享）/ 被路由专家数」，并推导出专家池在「仍是算力受限、且通信能与计算重叠」的前提下所能服务的最稀疏 MoE。这个下界随模型的宽度和深度上升；在 DeepSeek-V3 的宽度与深度、H800 上它是 0.058，而 Step-3 大约按 0.08 训练。更稀疏的模型就得退回大规模专家并行，或者用 3.7 节那种分组受限路由之类的限制。

什么时候用它：当你的推理引擎能把注意力和专家跑成两个独立显卡池（像 MegaScale-Infer 和 Step-3 的服务系统那样），并且这种布局能打过一个按同样卡数重新配过的共置专家并行布局——因为正是这种比较，让论文里的增益在 Qiu 等人的模型里缩到了打平。如果你打算按这种方式服务，在定专家数和 top-k 之前，先按你的宽度、深度和硬件算出 Step-3 那个下界，并让激活比例高于它。

注意力和专家跑在哪儿是服务侧的决策，但在 RL 后训练里，服务引擎还负责写出训练器要学的 rollout。那么，你能指望训练器把每个 token 送到和 rollout 引擎一样的专家上吗？

RL 里的路由

RL 后训练把样本生成和梯度计算分开。rollout 引擎用它自己的 kernel 生成样本，在 off-policy 或异步 RL 下，用的还是比训练器落后一次或更多更新的权重；而训练器要把每个 token 重算一遍来求梯度。这些差异会挪动 token 概率，但在 MoE 里，它们还会改变一个 token 走了哪些专家——于是训练器求的是一条样本根本没走过的路径。

每个成因都被单独测过。Ma 等人用 SGLang 在 Qwen3-30B-A3B 上生成回答，再喂给 Megatron：约 10% 的路由器选了不同的专家，94% 的 token 至少在一层上不一样。GSPO 的作者 Zheng 等人在 Qwen3-30B-A3B-Base 上测了另一个成因：一次梯度更新后，同一条 rollout 里约 10% 被激活的专家变了。两个 10% 数的不是一回事：一个是跨引擎不一致的路由器，一个是一次更新里会变的激活专家。

Ma 等人的修法是 R3：把 rollout 选中的专家记下来，在训练器里重放。这抹掉了路由那一份差距，但抹不掉其余的：训练-推理 KL 从 1.5e-3 降到 7.5e-4，接近稠密 Qwen3-8B 的 6.4e-4。在 Megatron 里，`--moe-enable-routing-replay` 让推理引擎记录每个请求的 top-k；你的 RL 框架随后在 `REPLAY_FORWARD` 前用 `set_replay_data` 把这些索引交回训练器。（第三个动作 `REPLAY_BACKWARD`，是在激活重算于反向传播中重做前向时，再把索引重放一遍。）记录时要把引擎的异步调度关掉，因为引擎不接受两者同时开。重放并不冻结路由器：在 `REPLAY_FORWARD` 里它会在被重放的索引处取自己当前的分数，于是梯度流过的是 rollout 选中的专家，而不是它自己会选的专家。

```
# megatron/core/inference/inference_request.py#L786-L787（每个请求的 rollout 索引）
# routing_indices is reconstructed from per-block storage when a request finishes.
routing_indices: Optional[np.ndarray] = None
# docs/api-guide/router_replay.md#L70-L80（先 set_replay_data，再 REPLAY_FORWARD）
1. Enable & Instantiate: one `RouterReplay` instance per MoE router layer.
2. Record: `RouterReplay.set_global_router_replay_action(RouterReplayAction.RECORD)`.
3. Forward Replay: `set_replay_data(list_of_tensors)` then `REPLAY_FORWARD`.
4. Backward Replay: `REPLAY_BACKWARD` under recomputation.
# megatron/core/transformer/moe/router_replay.py#L13-L15（三种动作）
RECORD = "record"
REPLAY_FORWARD = "replay_forward"
REPLAY_BACKWARD = "replay_backward"
```

公开的 RL 配方在这件事上两边都有。DeepSeek-V3.2、MiMo-V2-Flash 和 Instella-MoE 在训练器里重放 rollout 的路由；Kimi K2.5 与 Mellum 2 则没有重放，而是在 loss 里处理这个不一致——把概率比漂得太远的那些 token 的梯度屏蔽掉。

重放到底能不能换成 reward，仍然有争议。在 Ma 等人于 Qwen3-30B-A3B 上的比较里，不带重放的 GRPO 在三种设置里全崩，带截断重要性采样的 GRPO 在两个里崩了一个，而带重放的没有一次崩，他们报告训练稳定性和性能都有显著提升。后来 Zheng 等人的一项 Qwen 研究在 on-policy 训练里没看到 R3 的收益；在 He 等人的实验里，重放在 Moonlight-16B-A3B 上不如朴素 GRPO，不过在 Qwen 的 30B-A3B 上不是。开销在 rollout 侧很小，存路由带来的额外延迟不到 3%。而用过重放的 GSPO 作者列出了额外显存与通信、以及可能限制模型容量的问题，最终放弃它，改用序列级重要性比。

TRL 不用重放来处理同一个不一致。在它的 `GRPOConfig` 里，`importance_sampling_level="sequence"` 给出 GSPO 的序列级比值；默认开启的 `vllm_importance_sampling_correction` 会在训练器与 vLLM 的概率之间施加重要性采样比，并把比值落到设定区间外的序列屏蔽掉。

重放剩下的那一部分，是 kernel 自身贡献的差距，Megatron 用 `--batch-invariant-mode` 去治它：文档把 batch-invariant kernel 说成训练与推理 log 概率不一致的解法，称其为「RL 中真实存在的误差与不稳定来源」。它需要 flash attention 3 或 4，MoE 训练还需要 `alltoall` 分发器，Megatron 也警告说它「会显著影响速度」。在我们那个 64 专家 MoE 上（较早的 Megatron commit），开着它跑两次，12 步里打印的 loss 与梯度范数完全一致，而它带来的每步耗时成本分辨不出来，混在 run 与 run 之间的波动里。至于 rollout 权重比训练器落后一次更新，那是另一个成因，batch-invariant 模式管不着。

DeepSeek 的生产服务引擎 DeepInfer 提供同样的选项：设 `DEEPINFER_BATCH_INVARIANT=1` 时，模型结果在不同 batch 组成下保持一致；再设 `DEEPINFER_ENABLE_MTP_VARLEN=0` 并给每个请求固定种子，采样就是确定性的——仅限 NVIDIA 显卡。

StepFun 给 Step 5 Preview 做的 RL 两种补救都用。rollout 用当前权重时，它把训练与推理的算子对齐到 log 概率逐位相同，开销不到 10%。rollout 陈旧时，它重放被路由的专家——索引与系数都重放，路由器冻结——actor 与参考策略都这么做。这比 Megatron 的重放走得更远，后者会在被重放的索引处用路由器当前的分数重算权重。

NVIDIA 给 Nemotron 3 Super 与 Ultra 的 NeMo-RL 配方，在「rollout 最多落后一次更新」的异步 RL 里冻结路由矩阵，但仍然更新 Megatron 加到分数上、只用于选专家的那个均衡偏置。那个偏置每步都动，所以选择在冻结矩阵下仍会漂——图里那行「冻结」并没有体现这种漂移。

量化 rollout 的激活会放大同一个差距。Qwen 的 Zhuge 等人跑 NVFP4 rollout（权重和激活都是 4 bit），对上一个 BF16 训练器，训练在约 150 步后崩了。他们比较了「只量化权重」与「只量化激活」两组 rollout，把崩溃归到量化后的激活上，而不是权重。

会咬人的不一致，表现为一次学得很差或者直接崩掉的 RL 实验。信号是训练-推理概率差，Megatron 的 RL 循环往 W&B 记两个指标：`mean_inf_train_prob_abs_diff`（训练器与 rollout 概率的平均绝对差）和 `max_piold_to_inf_prob`（训练器比 rollout 概率的最大比值）。想单独看路由那一份，就把 Megatron 推理引擎记下的 `routing_indices` 和训练器自己的 top-k 离线比一比。

什么时候用它：当一次 MoE 的 RL 里，训练器与 rollout 引擎在路由上不一致时——不管是因为两个不同的引擎，还是在 off-policy / 异步 RL 下 rollout 权重落后于训练器。重放能消掉这个不一致，但它换来的 reward 收益有争议：Ma 等人报告有增益，而 Qwen 那项 on-policy 研究没看到。先量一下有

多少路由器不一致，再比一次带重放和不带重放的学习曲线，然后决定留不留。如果你还要量化 rollout，先只量化权重试试——在 Zhuge 等人的实验里，是量化后的激活导致了崩溃。

到这里，你已经把一个 MoE 从第一次稠密基线带过了服务、又带进了一个 RL 循环。这一路上的选择里，哪些可以当作已经定了，哪些还开着？

导航：[[2026-11-moe-handbook|全书目录]] · ← 第 3 章（下）系统工程 · 第 5 章 结语与推荐参数 →

第 5 章 结语

我们从一次稠密实验出发，把 MoE 层一个设置一个设置地拆开，然后跟着训练好的模型走进推理服务和 RL 后训练。现在你可以先验一遍集群再去信它，用同一批数据和同一个启动脚本训出稠密基线与 MoE，并且让两次实验可复现到能相互比较的程度。第 3 章给每一层里的每个部件——从路由器的分数到 All-to-All——都配了一个该盯的信号和一条建议。附录 A.2 反向走一遍：从面板上的一个症状，倒推回背后的那个配置。

下面这段把各章推荐的参数集中起来，每条后面注释了为它说话的节号：被注释掉的行只在某个条件下成立，或者用来替换上面那一行；尖括号里的值由你自己定。从你自己的启动脚本出发，一组一组往里加，每加一组都跟上一次实验比——每条推荐都来自它自己的那套环境，收益并不保证能叠加。

按节整理的全部推荐参数

```
# 2.1 测硬件
--gpu-sniff-test-interval <n>               # 2.1：每 n 步做一次 GPU 与 EP All-to-All 检查

# 2.2 构建镜像（AWS P5 上的 EFA）
# 2.2、3.7：先装 EFA 库，否则 NCCL 会悄悄退回 socket
export LD_LIBRARY_PATH=/opt/amazon/ofi-nccl/lib:/opt/amazon/efa/lib:${LD_LIBRARY_PATH:-}
export NCCL_NET_PLUGIN=ofi                  # 2.2：按名字选；传路径会让 NCCL 留在 socket 上
export FI_PROVIDER=efa                      # 2.2
export FI_EFA_USE_DEVICE_RDMA=1             # 2.2
export NCCL_SOCKET_IFNAME='^lo,docker0,veth_def_agent'  # 2.2：按你的网卡改
export NCCL_DEBUG=INFO                      # 2.2、3.7：要看到 "Using network Libfabric"

# 2.5 你的第一个 MoE
# --deterministic-mode                      # 2.5：要比实验或追 NaN 时开；
# export NVTE_ALLOW_NONDETERMINISTIC_ALGO=0 #   同时去掉 --cross-entropy-loss-fusion

# 3.0 监控
--moe-per-layer-logging                     # 3.0、3.5：逐层路由损失
--save <dir> --save-interval <m>            # 3.0：token 统计与 dump 落在这里
--save-tokens-per-expert-interval <n>       # 3.0、3.3：逐专家 token 数、MaxVio
--log-throughput --log-params-norm          # 3.0：TFLOP/s、参数范数
--log-num-zeros-in-grad --log-max-attention-logit  # 3.0：运行健康度
# --moe-routing-trace-path <dir> --moe-routing-trace-max-training-iters <n>
# --cuda-graph-impl none                    # 3.0：有界路由追踪；hook 会跳过 CUDA graph
# --save-activations-interval <n>           # 3.0、3.4：专家输出范数；抽样稀疏一些

# 3.1 路由器打分
--moe-router-dtype fp32                     # 3.1：bf16 的并列值会让专家翻转
# --moe-router-score-function sigmoid       # 3.1、3.3：专家偏置需要它
# --moe-router-pre-softmax                  # 3.1：top-1 时做 softmax，否则路由器没梯度
# --moe-z-loss-coeff 1e-3                   # 3.1：logits 持续上漂的 softmax 路由器

# 3.2 路由策略
# --moe-num-hash-layers <n>                 # 3.2：只为复现 DeepSeek-V4；
#                                           #   要加载固定表，不是占位符

# 3.3 负载均衡
--moe-router-load-balancing-type aux_loss --moe-aux-loss-coeff 1e-3  # 3.3 辅助损失
# --moe-router-force-load-balancing         # 3.3、3.7：只用于对照实验，绝不用于训练
# --moe-router-load-balancing-type global_aux_loss  # 3.3 作用域：领域专家，步时 +5.8%
# 3.3 无损失偏置，替换上面那行 aux_loss：
# --moe-router-enable-expert-bias --moe-router-score-function sigmoid
# --moe-router-bias-update-rate <rate>      # 按专家数调；超过 128 后 1e-3 太粗
# --moe-router-load-balancing-type seq_aux_loss --moe-aux-loss-coeff 1e-4  # DeepSeek-V3
# --moe-router-load-balancing-type quantile_balancing  # 3.3 分位数法：不要 aux loss，
#                                           #   也不支持路由器融合或分组受限路由

# 3.4 专家
# 2.5 的形状；3.4 数量与大小：专家更细有帮助，但有上限
--num-experts 64 --moe-ffn-hidden-size 1024 --moe-router-topk 8
# --moe-latent-size <l>                     # 3.4 潜空间 MoE：专家更多，通信量不变
# --moe-use-norm-before-up-proj             # 3.4：Kimi K3 对潜空间数值爆炸的修法
# --moe-shared-expert-intermediate-size 1024  # 3.4：被路由的宽度，top-k 减一
# --activation-func-clamp-value 10          # 3.4 激活：专家范数发散的场合
# --activation-func-tanh-clamp-scale 4 --activation-func-tanh-clamp-scale-linear 25
```

```
#                                           # 3.4：Kimi K3 的 SiTU-GLU，需 Megatron Core v0.20+

# 3.5 层间布局
# --moe-layer-freq "([0]+[1]*15)"           # 3.5：首层留稠密，少几次 All-to-All
# --moe-shortcut-connection --moe-shortcut-parallel  # 3.5 层间连线：混合栈；
#                                           #   不支持 FSDP、CUDA graph 与完整重算

# 3.6 训练
# --moe-use-upcycling --moe-upcycling-granularity 8  # 3.6：只第一次跑，配 2.5 的两个补丁
# --optimizer muon --muon-extra-scale-factor 0.2     # 3.6 优化器：1.0 会发散；不支持 FSDP
# --moe-router-skip-muon                    # 3.6：路由器留在 Adam，需 Megatron Core v0.20+
# --mtp-num-layers 1 --mtp-loss-scaling-factor 0.1   # 3.6 MTP
# --pipeline-model-parallel-layout <layout> # 3.6：把 MTP 层放进 PP 布局
# --timing-log-level 2 --timing-log-option all       # 3.6：找出最慢的 PP stage
--rerun-mode validate_results               # 3.6 稳定性：默认值，保持开着

# 3.7 系统工程
# --profile --profile-step-start <a> --profile-step-end <b> --profile-ranks <r>
#                                           # 3.7 性能剖析：每个新配置只剖一步
# --moe-permute-fusion                      # 3.7：permute 与 copy kernel 显眼时用
--expert-model-parallel-size 8              # 3.7 互联：EP 控制在一个节点内
# --overlap-moe-expert-parallel-comm --delay-wgrad-compute  # 3.7 重叠：暴露出来的
#                                           #   All-to-All；不支持完整重算与 PP 下的 VPP
# --recompute-granularity selective --recompute-modules layernorm moe_act
# --fine-grained-activation-offloading --offload-modules core_attn attn_proj expert_fc1
#                                           # 3.7 显存：重叠放不下时用
# --activation-offload-fraction <f>         # 3.7：步时变长就把这个比例调低
# --moe-token-dispatcher-type flex --moe-router-num-groups <nodes> --moe-router-group-topk <g>
#                                           # 3.7 分组受限：跨节点 EP，需 DeepEP
# --moe-expert-capacity-factor <cf> --moe-pad-expert-input-to-capacity
#                                           # 3.7 容量：显存吃紧；要微调就改回 dropless
# --fp8-format <fmt> --fp8-recipe blockwise # 3.7 精度：Hopper；Blackwell 上用 mxfp8
# --moe-router-padding-for-quantization     # 3.7：给量化 GEMM 对齐计数
# --moe-dispatch-fwd-dtype mxfp8 --moe-combine-bwd-dtype mxfp8  # 3.7：ncclep，Blackwell

# 4 从训练到推理服务
# --fp4-format e2m1                         # 4 低位宽：Blackwell 上的 NVFP4，不与 FP8 同用
# --rl-inference-expert-model-parallel-size <n>  # 4：rollout 引擎自己的 EP 大小
# --moe-enable-routing-replay               # 4 RL 路由：rollout 与训练器不一致时
# --batch-invariant-mode                    # 4：kernel 那一份不一致；代价是变慢
```

还有大量问题没定。我们对大规模训练 MoE 的了解，大多来自开源权重模型的技术报告，而报告写的是一个团队最终交付了什么：很少写它试过又放弃了什么，也很少写是哪一次实验定下了一个选择。上面许多推荐都建立在某一家、某一套环境里的单次消融上，每一条都值得在别的规模、别的数据、别的集群上重跑一遍。如果你跑了一个，把配置和日志一起发出来，包括负面结果——下一个团队就是这样从你学到的东西起步，而不是从零开始。你可以在本文的讨论区分享你的结果，或者问我们。

希望这篇长文能替你下次训 MoE 省下几天——我们自己是为它花掉了几天的。

致谢

感谢 Antoine Bosselut、Martin Jaggi 与 Swiss AI 团队提供了本文背后的一部分算力，以及一路上那些有启发的讨论。也感谢 Thibaud Frere 在制图上给予的帮助。

导航：[[2026-11-moe-handbook|全书目录]] · ← 第 4 章 从训练到服务 · 附录 A 监控细节与排查索引 →

附录 A：监控细节与排障索引

> 说明：原站的图是交互组件，这里无法复现；本文按节内顺序编号引用。原文用的 Megatron 版本固定在 commit `d6316eb45`。

A.1 监控细节

本附录给出第 3 章各处「该盯什么」背后的命令与算法。路由器统计描述工作量的分布，路由 trace 保留每一次专家选择，激活转储显示专家产出了什么；run 级日志把这些测量与 loss、时间、内存放在一起看。

每条配方都针对 Megatron commit `d6316eb45`。每次抓取都要连配置与 rank 布局一起保存：它们决定局部专家 id、token 位置与路由分数该怎么解读。trace 与激活只在短的诊断运行里采，与吞吐测量分开跑。

> 代码位置一览（原文内嵌代码，这里只给位置，按需去仓库读）：
> 辅助损失上报 `router.py#L691-L700`、z-loss 上报 `router.py#L805-L811`、逐层标量写盘 `moe_logging.py#L392-L404`、给每个路由器挂 hook `router_trace.py#L217-L221`、路由图转 top-k id `router_trace.py#L353-L365`、重算 logits 并存 bf16 `router_trace.py#L383-L395`、参数范数 `training.py#L5387-L5393`、梯度零元素计数 `clip_grads.py#L282-L288`、最大注意力 logit `qk_clip.py#L30-L40`、NaN 梯度中止 `param_and_grad_buffer.py#L396-L406`。
> 作者的三个脚本：`router_watch.py` 读 token 计数与路由 trace、在活的 router 上查 fp32 flip；`expert_watch.py` 读激活转储；`expert_watch_hooks.py` 在激活发生时现场采样。

路由器统计

逐层损失

用 2.4 的 Trackio 启动器开 `--moe-per-layer-logging`，就能逐层记录辅助损失与 z-loss。Megatron 只记录配置里启用的损失，并且已经除掉了它们的系数。一个用专家偏置做均衡的运行，只要启用了辅助损失或 z-loss，照样会报这两项。

辅助损失在概率很平时也能读到 1，哪怕 token 计数很不均（3.3），所以直接用 token 计数来衡量负载。z-loss 描述 logits 的量级：把记录值开平方，得到 token 间 logsumexp 的 RMS。

专家计数与 MaxVio

要保存每个本地专家处理的 token 数，把下面两个开关加到训练命令：

```
--save run/ \
--moe-grouped-gemm \
--save-tokens-per-expert-interval 100
```

分组线性层上的 hook 会写出 `run/tokens_per_expert/rank*.jsonl`。每条记录标明迭代步与层，`tpe` 列表里每个被捕获的 micro-batch 一个计数向量。解码层与 MTP 层有不同的身份。这些向量描述的是 dispatch 之后的本地专家负载（`activation_logging.py`）。

对一个专家计数向量 c，MaxVio 衡量最忙专家相对均值的超出量：

```
MaxVio = max(c) / mean(c) - 1
```

比如计数 `[2, 2, 4, 0]` 的均值是 2、MaxVio 是 1：最忙专家收到的负载是平均的两倍。零计数的专家也计入均值，必须留在算式里。完全均衡时是 0；全零向量的 MaxVio 无定义。

算之前先定聚合范围：

- micro-batch：把一个专家并行（EP）组内的本地专家向量按全局专家 id 顺序拼起来。
- global batch：把各 micro-batch 与各独立数据副本上每个专家的计数相加，再从得到的向量算 MaxVio。
- 序列：用逐次的路由选择与序列边界，见「路由诊断」。

把各 micro-batch 的 MaxVio 取平均，并不会得到 global-batch 的 MaxVio。专家可以在每个 micro-batch 里都不均、在整批上却均衡（3.3）。聚合还必须保留层的身份：流水 stage 与 MTP 深度都要分开。交错流水的 chunk 会复用局部层 id，而保存的记录区分不了它们（`activation_logging.py#L131-L134`），所以抓取时把交错关掉。激活重算会让 hook 被调用两次，所以要么关掉重算再抓，要么保留原始前向的标识。

`router_watch.py` 这个读脚本按一张显式的「rank → 全局专家 id」映射把计数文件拼起来。两个 rank 各持四个专家时，把 `{"0": [0, 1], "1": [2, 3]}` 存成 `experts.json`，然后跑：

```
python3 scripts/router_watch.py counts run/tokens_per_expert \
    --num-experts 4 --ranks 0 1 \
    --expert-map experts.json --pool-ranks
```

`--pool-ranks` 声明所选文件是同一层的独立工作（已去掉张量并行副本）。要逐 micro-batch 测量，就得保留能跨 rank 对上「同一次前向」的标识。

每个 rank 的负载

把一个 EP rank 上所有专家的计数加起来，再拿最忙的 rank 与均值比。这个数与专家 MaxVio 并看：几个忙专家如果摆在同一张 GPU 上，工作就会挤在那儿（3.3、3.7）。

开了容量上限或补齐时，分组 GEMM 的计数描述的是这些操作之后的工作量。要量原始需求，用路由器未被补齐的选择。

路由诊断

抓取专家选择

短抓取时把下面这些开关加到训练命令，把 `trace_steps` 换成要抓的迭代数：

```
--moe-routing-trace-path trace/ \
--moe-routing-trace-max-training-iters trace_steps \
--cuda-graph-impl none
```

hook 在 dispatch 之前记录被选中的专家 id。加 `--moe-routing-trace-capture-logits` 会额外存 logits 文件；加 `--moe-routing-trace-capture-hidden-states --moe-routing-trace-dump-weights` 会存隐状态与路由器权重。这些抓取都要写盘，而 Python hook 需要 eager 执行。

专家 id 按路由图的顺序排列，所以某个指标需要「主专家」时，要按重建出来的权重排序。丢 token 可能导致整条记录被跳过，这种缺口按缺失观测处理（`router_trace.py`）。

从 trace 算计数与 MaxVio

被选中的专家 id 本身就能给出 token 计数与 MaxVio，不必抓 logits。读一个持有 64 专家的 rank：

```
python3 scripts/router_watch.py trace trace/ --num-experts 64 --ranks 0
```

每条 `trace_call` 记录里有单次调用的 `counts` 与 `maxvio`。`rank_captured_step` 记录则在每层、每 rank、每步内把计数求和、再从和算 MaxVio。这些汇总量只覆盖被捕获的 token；要做 global-batch 测量，需要该批所有原始前向与独立副本，并去掉张量并行副本。拼接抓取时用「专家计数与 MaxVio」那节的聚合规则。因丢分配而被跳过的记录，在这些总量里仍然是缺失的。

逐序列均衡

逐序列均衡需要每个 token 行属于哪个序列。完整、未切分的输入按 `[sequence, batch, hidden]` 排布、batch 为 B 时，第 `s * B + b` 行属于序列 b。只有这种布局成立时，才给读脚本传 `--sequence-length S --batch-size B`。

在每个序列内统计各专家被分到的次数（含零次的专家），再从该向量算 MaxVio。然后按层汇总这些逐序列的值。

对打包序列，把定长布局换成真实的序列边界。上下文并行或序列并行还需要 token 位置，才能把各 rank 上的片段拼起来。跨观测地跟踪同一个 token 还需要它的身份——两个训练步里同样的行号，通常指的是不同 token。

混合权重

logits 文件支持对「施加在被选专家上的权重」做近似重建。hook 会重算这些 logits、以 bf16 存下。重建权重时要保留记录下来的选择，并使用该次运行的打分函数、归一化方式与路由缩放系数。

一次 64 专家、缩放系数 1 的 softmax 抓取：

```
python3 scripts/router_watch.py trace trace/ \
    --num-experts 64 --ranks 0 --score softmax --scale 1
```

如果该次运算是先对全部专家做 softmax 再选，加 `--pre-softmax`；否则 softmax 只作用在选中的 logits 上。用 sigmoid 和 sqrt(softplus) 时，按该次运行对选中分数的归一化来处理。专家偏置影响选择，但不要算进混合权重。

记最终被选权重（含缩放系数）为 w，令 q 为 w 除以其和。每个指标先按 token 算、再按层汇总：

- 最大权重：`max(w)` 是对专家输出的最大倍率。要跨不同缩放系数比较集中度，用 `max(q)`。
- 选中权重熵：`-sum(q * log(q))`，范围 0 到 `log(k)`，k 个权重越接近熵越大。top-1 时为 0。
- Max1/Max2：最大权重除以第二大的。接近 1 说明前两名权重相近；top-1 时无定义。
- 估计的混合 RMS：`sqrt(sum(q*q))`，在「各专家输出 RMS 相等且互不相关」的假设下，估计混合结果相对单个专家的 RMS（3.1）。用 w 就把路由缩放系数也算进去了。
- top-k 概率质量：对 pre-softmax 路由，在缩放前把被选中的全 softmax 概率加起来。原始 sigmoid 与 sqrt(softplus) 分数不构成全专家上的概率，解读方式不同。

举例：选中权重 `[0.75, 0.25]` 的最大权重是 0.75，Max1/Max2 是 3，熵约 0.562（自然对数）。

每个 token 到达的节点数

要衡量通信扇出，把被选专家 id 映射到目的节点，再数每个 token 到达了几台不同节点。按层报均值与最大值（3.7）。

给读脚本传 `--node-map nodes.json`。该文件把全局专家 id 映射到节点名，例如 `{"0": "node-a", "1": "node-a", "2": "node-b"}`。对复制出来的专家，要抓 dispatcher 实际选中的那个目的地。要只数远端节点，还需要 token 的源节点，才能把它排除。

把扇出换算成字节，需要隐层宽度、dtype 与 dispatcher 的拷贝策略：每个专家一份拷贝、与每个目的节点一份拷贝，流量完全不同。

被丢掉的分配

有容量上限时，从路由器最终的 routing map 量丢弃量。N 个有效 token、K 个存活的分配：

```
dropped_fraction = 1 - K / (N * topk)
```

这数的是「token-专家分配」的损失。一个 token 可能只保住了它的一部分专家。算的时候要排除被 mask 的 token 和为量化 GEMM 补的行。这个测量要用活 hook，因为存下来的 trace 可能跳过含丢弃的记录。`ExpertWatch` 能抓这张 map。

精度翻转

fp32 flip rate 衡量「当路由器用 fp32 计算时，被选专家集合多久变一次」。两次计算必须用同一次前向的输入、权重、偏置与选择规则。比较的是集合，这样单纯顺序变化不算翻转。

对普通的 token-choice 路由，`install_fp32_flip_hook` 在活状态上做这个比较。把它挂到选定 MoE 层的 router 上、跑一次 eager 前向：

```
import torch
from scripts.router_watch import install_fp32_flip_hook

previous_tf32 = torch.backends.cuda.matmul.allow_tf32
handle = install_fp32_flip_hook(moe_layer.router, emit=print)
try:
    torch.backends.cuda.matmul.allow_tf32 = False
    output = model(tokens)
finally:
    handle.remove()
    torch.backends.cuda.matmul.allow_tf32 = previous_tf32
```

关掉 TF32 就得到 fp32 精度的参考矩阵乘。hook 在一批有界的有效 token 上比较选中的专家集合。它支持哪些设置见 docstring，不支持的路径会被拒绝。要关掉 CUDA graphs 跑，并且别放在计时测量里。

两次计算都从活输入的现有精度出发，所以结果隔离出的是「该输入精度下的路由分歧」；它对 loss 的影响要另做实验（3.1）。

做离线比较时，要保留被捕获前向的精确状态与门控输入，包括抖动。内置的权重转储只在每个 router 第一次被调用时快照，因此不跟踪后续的权重与均衡偏置更新。隐状态与重算的 logits 以 bf16 存，快照还不含 router 的线性偏置（如果有）。所以从这些文件做的重建应标注为「近似」，并说明抓取步与缺失状态。

专家激活

抓取激活

加 `--save run/ --save-activations-interval 100` 可以抓受支持线性模块的输入输出，含分组专家的 token 计数。每个模块的条目会在它下一次被抓时被覆盖，所以留下的是最后一次抓到的 micro-batch。hook 在抓取时会把这些张量拷到 CPU，所以间隔要稀疏、运行要短，并关掉 CUDA graphs 和激活重算。

逐专家输出范数

原生转储里的 `experts.linear_fc2/output0` 含本地专家那几行。用模块的 `input1` token 计数把这些行切开，转成 fp32，算每个 token 输出向量的 L2 范数。这些范数的均值描述该专家的输出量级。把每个范数除以输出宽度的平方根就得到 token RMS，便于跨宽度比较。

`expert_watch.py` 做这些计算。在有 PyTorch 的环境里跑：

```
python3 scripts/expert_watch.py \
    --unpadded --expert-tp-size 1 activations.pth
```

这两个选项断言「抓到的行不含补齐」「专家张量并行是 1」。脚本报每个专家的 token 数、范数之和与平均范数，还会报「收到 token 的那些专家里」相对中位数的最大与最小平均范数比（3.4、3.6）。

空专家没有平均输出范数，所以脚本把它们单列出来。中位数为零时这些比值也无定义。把一个专家所有行拼起来再取范数，会把输出量级和 token 数混在一起；对逐 token 范数取平均则把这两者分开了。

这个中位数覆盖的是一个文件里的专家。要做整层比较，先把 EP 分片拼起来再算中位数。多次观测可以合并：把每个专家的 `sum_l2` 与 `rows` 相加再相除，得到均值。专家张量并行大于 1 时，要先合并部分输出向量——它们的单个范数不能相加来还原完整向量的范数。这些计算都要排除补齐行。

分组 fc2 的输出可能已经含了路由权重，取决于实现把权重施加在哪一步。于是小范数也可能只反映小的路由权重。比较时把「施加权重的位置」固定住，并把混合权重与范数放在一起看。

现场采样时，`ExpertWatch` 在它的 `routed_experts` 输出里报同样的逐专家范数统计，含 `local_max_to_median` 与 `local_min_to_median`。这些比值只覆盖本地专家；算整层中位数前要跨 EP rank 拼统计量。见「共享与路由分支的贡献」那节的配置。

潜在升维

要检查潜在升维，读 `fc2_latent_proj/output0`，比较它的输入与输出 RMS（3.4）。用 RMS 才能抵掉潜在宽度与模型宽度的差异。开了 `--moe-use-norm-before-up-proj` 时，再在归一化之前采一次，定位增长从哪开始。

共享与路由分支的贡献

要比较两个分支，抓「过完可选 sigmoid 门之后的共享输出」与「逆置换、合并之后的路由输出」，有潜在升维时把它算进去。每个 token 算 `norm(shared) / norm(routed)`，分母为零的单列。这个比值比较两个分支的量级；两向量相加时仍可能互相加强或抵消。

原始的 fc2 转储发生得更早：共享 fc2 在门之前，路由 fc2 在合并之前。要在需要的点上现场采样，就把 `ExpertWatch` 套在一次诊断前向上：

```
from scripts.expert_watch_hooks import ExpertWatch

with ExpertWatch(moe_layer, emit=print):
    output = model(tokens)
```

这个 helper 支持：eager 执行、all-to-all dispatcher、张量并行与专家张量并行都是 1、关掉共享专家重叠。样本必须未补齐，且关闭量化与专家补齐。把 hook 的范围限制在那次诊断前向上，并把每 rank 的 JSON 输出连同步与 micro-batch 身份一起存。

对标准 MoE，helper 用「总输出减去门后的共享输出」还原路由分支。这个减法会带上原始加法里的舍入误差，所以当路由分支相对共享分支很小、又需要准确值时就别用减法，直接在相加之前抓。对潜在 MoE，

helper 直接抓升维的输出。分支汇总会包含补齐位置，这正是这个比较要求未补齐样本的原因。

挂 hook 之前想先在 CPU 上验一遍，跑 `python -m unittest scripts.test_router_watch scripts.test_expert_watch`。

运行健康

Loss 与梯度信号

把 LM loss 和梯度范数与路由器、专家的测量放在一起。下面这些开关提供 run 级信号：

- `--log-throughput`：训练吞吐。
- `--log-params-norm`：参数范数。
- `--log-num-zeros-in-grad`：整个模型梯度里零元素的数量。
- `--log-max-attention-logit`：最大注意力 logit，需要 Transformer Engine 2.9 或更新。

要查专家级的下溢（3.7），在梯度累加之后、优化器更新之前，数专家权重梯度里恰好为零的元素。把这个专家级的数与同配置的 BF16 运行比。

NaN 检查默认开启。检出 NaN 会中止这一步并触发重跑处理（3.6）；`--no-check-for-nan-in-loss-and-grad` 关掉它。

时间与内存

加 `--timing-log-level 2 --timing-log-option all` 得到逐 rank 的前向与反向耗时。按流水 stage 给 rank 分组，定位慢的 stage（包括持 MTP 的那个，3.6）。然后用 3.7 的工具剖析那个 stage。

跨运行比较时，比同一段「预热之后」的区间。测峰值内存时，在区间开始重置 CUDA 峰值统计、在区间结束读每个 rank 的 `torch.cuda.max_memory_allocated()`，取各 rank 的最大值作为最高峰值。

A.2 排障索引

第 3 章是一节一节顺着讲的：从一个开关讲到它对这次运行做了什么。运行出问题时你是反着走：先在看板上看到一个症状，再去找它背后的配置。本附录就是那张反向索引，一行一个症状，各自指回正文里解释它的那一节。

不过在怪某个设置之前，先用 2.5 末尾那两项检查把技术栈本身排除掉。

怎么读这张索引

每行带一个标签、症状（以及最先动的那个信号）、还有该去读的小节。标签按「怎么坏」给运行分类：`slow`（慢）、`OOM`（爆内存）、`blows up`（炸掉）、`learns badly`（学不好）、`won't start`（起不来，指

第一步之前就停）。先找你的标签，再看最先动的信号，那一行会告诉你该去哪。（引言里那张图把这些行都画了，只差一行 `global_aux_loss` 设置——它没有起始信号。）

标签	症状	该读
`slow`	吞吐下降	3.3、3.7 剖析
`slow`	单节点一步里一半时间都是 All-to-All	先 3.3 均衡，再 3.7 重叠
`slow`	辅助损失接近 1，可有一张 GPU 很慢	3.3 负载不均的代价：概率平坦时辅助损失恒为 1，与负载无关，要看 MaxVio
`slow`、`learns badly`	用无损失偏置跑了很久，仍不均衡	3.3 无损失偏置：固定步长滞后，改用分位数均衡
`slow`、`learns badly`	只有前几层有几个专家过载	3.2 哈希层、3.5 稠密层
`slow`	每个 token 的 All-to-All 都到达所有节点	3.7 互联与组限路由
`slow`	换成更多、更小的专家后吞吐下降	3.4 数量与宽度
`slow`、`OOM`	开 MTP 后一步慢很多，或流水并行下多了几层就 OOM	3.6 MTP
`slow`	剖析里 All-to-All 是裸露的	3.5 块接线、3.7 重叠
`slow`	多节点慢约 25 倍（我们的 EP16 代理），且 NCCL 日志说 "Using network Socket"	3.7 互联
`slow`	换成 `global_aux_loss` 后一步变慢	3.3 范围
`slow`	分位数均衡下，global batch 的 MaxVio 低、micro-batch 的高	3.3 分位数
`slow`	加 `--moe-latent-size` 后一步变慢	3.4 潜在 MoE：两个 $d \times \ell$ 投影与更小的专家 GEMM；LatentMoE 报的吞吐是服务侧的，并发 1 时低 12%
`slow`	在路由 top-k 之上加共享专家后推理变慢	3.4 共享专家
`slow`	全量重算还开着	3.7 内存
`slow`	均衡、重叠、融合都做了，TFLOP/s 还是不够	3.7 精度
`OOM`	某个 rank OOM	3.3（不均衡）、3.7 内存
`OOM`	峰值内存看着没问题，却有一个 rank OOM	3.7 内存（Megatron 只打数据并行 rank 0）
`blows up`	一个 NaN，之前什么都没有在动（梯度范数、参数范数、z-loss 都平）	3.6 稳定：没有预警；在另一张 GPU 上重跑能区分坏卡与可复现的 NaN
`blows up`	同一节点上两次退出码 16	3.6 稳定：重跑需要换一张 GPU
`blows up`	Muon 运行在最初几步就发散	3.6 优化器（更新尺度、`--muon-extra-scale-factor`）

标签	症状	该读
`blows up`	z-loss 或路由 logits 在尖峰前一路爬升	3.1 精度（若 NaN 之前 z-loss 反而平，则是 3.6 稳定问题）
`blows up`	专家输出范数在增大，或它的 max/median 比在尖峰前爬升	3.4 激活（clamp）
`blows up`	潜在 MoE 出现激活爆炸或尖峰	3.4 潜在（升维前归一化）
`blows up`	NaN 前几步梯度范数与负载均衡损失一起跳	3.6 稳定、3.3。我们的 7B-A1B
`blows up`	Muon 下的 loss 尖峰，从附近 checkpoint 续训常能躲开	3.6 优化器（bf16 里做正交化）
`learns badly`	RL reward 停滞或塌掉，同时训推概率差在变大	第 4 章 RL 路由
`learns badly`	loss 平，但 MaxVio 或某专家输出范数偏离该层中位数（塌缩、死专家、流氓专家）	3.6 稳定、3.3
`learns badly`	top-k 权重全相等（门很平）	3.1 尖锐度
`learns badly`	开着辅助损失却出现死专家	3.3 辅助损失（系数太小）
`learns badly`	`global_aux_loss` 且每个 global batch 超过 256 个 micro-batch	3.3 范围 的开关（bf16 计数器）
`learns badly`、`blows up`	最前面几个 MoE 层路由塌缩或发散	3.5 稠密层
`learns badly`	设了容量系数，loss 落后于不丢 token 的运行	3.7 容量与副本（丢弃量不会记日志）
`learns badly`	Muon 下专家一层一层塌缩	3.6 优化器
`learns badly`	用 `--moe-router-pre-softmax`（top-k 权重不再归一化），loss 高于归一化的运行	3.1 输出尺度
`learns badly`	专家没有按领域分化	3.3 范围
`learns badly`	系数调大后 LM loss 反而上升	3.3 辅助损失
`learns badly`	专家变得更小之后 loss 不再改善	3.4 数量与宽度
`learns badly`	用专家选择（expert choice）路由训的模型，推理时表现不同	3.2 谁在选

标签	症状	该读
`learns badly`	upcycle 出来的 MoE 领先了一阵之后就学得慢	3.6 upcycling
`won't start`	`--moe-use-upcycling` 报 "unexpected keyword argument 'model'" 或缺 `pre_mlp_layernorm` 键	3.6 upcycling（两个上游 bug）
`won't start`	开 `--overlap-moe-expert-parallel-comm` 后断言全量重算	3.7 重叠与内存

信号比症状重要，因为同一个症状可能来自完全不同的部位。我们那几个 NaN 之前，一直在看的信号（梯度范数、参数范数、z-loss）都是平的，追下去的那个 NaN 生在分组专家 GEMM 的反向里。这类 NaN，用 Megatron 在另一张 GPU 上的重跑就能区分「坏卡」与「数据或模型真产生了 NaN」（3.6 稳定）。一步变慢而辅助损失接近 1，应该改看 MaxVio：Megatron 的辅助损失在路由器概率平坦时恒等于 1，与负载无关，所以一张过载的 GPU 可以藏在它后面（3.3 负载不均的代价）。

代码、配置与数据

原文的这部分是一份逐条来源清单：每个数字与结论对应到具体配置文件、代码行或论文，例如 Qwen3-30B-A3B 的 config.json、DeepSeek-V3 的 config.json、DeepSeek-V4-Pro 推理代码里的 `Gate`（`model.py#L556`，`layer_id < n_hash_layers` 时走哈希路由）、datatrove 的 executor 等。完整清单见原文页面底部的「Code, configs and data」。

导航：[[2026-11-moe-handbook|全书目录]] · ← 第 5 章 结语与推荐参数